Robustness-Aware Evaluation and Enhancement of Mutation-Based Fuzzing for Bug Discovery

***Abstract*—Fuzzing is a powerful technique for discovering software vulnerabilities by generating test inputs and executing the program on them. Mutation-based fuzzing dominates in practice due to its broad applicability and ease of deployment: a fuzzer iteratively mutates the current input to produce the next one. Despite rapid progress in fuzzer design, rigorous evaluation remains challenging due to the inherent randomness of the fuzzing process, which can also limit bug detection. In particular, few have treated randomness as an independent evaluation dimension, even though it is central to fuzzer reliability and performance. Quantifying this randomness to assess robustness, and based on this understanding, developing methods that systematically enhance bug detection capability and robustness, are therefore essential.**

We address both needs. We propose an evaluation framework that models fuzzing as a Markov chain and formalizes robustness in terms of chain variances. We design an MC simulation procedure to generate fuzzing trajectories. Based on these trajectories, we develop estimators for the target objective. We then introduce evaluation metrics derived from these estimators and their deviation from the true objective, including variance-based measures and mean squared error. We establish principled fuzzer comparison via computable bounds, even when ground-truth metrics are unavailable. The resulting robustness characterization depends on the number of trials M, the trial length T, and a spectral parameter δ. Finally, we propose novel splitting-based simulation methods that increase bug detection capability by branching trajectories after bug triggers when the bug rate is low. We prove that splitting can also achieve smaller variability than the MC baseline, but the reduction depends on the associated improvement in bug detection capability. Extensive experiments with 7 fuzzers on 10 widely used FuzzBench benchmarks validate our insights, and ground-truth experiments on Magma demonstrate concrete security impact: splitting turns unreliable detections of real CVEs into reliable ones (e.g., CVE-2019-19926, triggered in 5 of 5 trials with splitting versus 0 of 5 without) while never finding fewer ground-truth bugs. Together these provide practical tools for evaluating and improving fuzzer performance.

1. Introduction

Fuzzing, iteratively generating inputs and feeding them to a program to trigger bugs, is among the most effective techniques for discovering software vulnerabilities [15], [17], [32]. Yet fuzzing is inherently stochastic: when we evaluated AFL++ on poppler across 20 independent 23-hour campaigns, the number of unique bug signatures per trial ranged from 1 to 13, a 13-fold difference. This variability is not an anomaly; it is the norm across fuzzers and benchmarks [15], [20], [32]. It raises a fundamental question: *which result should a practitioner trust, and how many trials are enough?*

Despite extensive work on fuzzer evaluation [15], [17], [32], the community lacks a principled answer. Existing approaches measure effectiveness via bug counts or code coverage, and assess stability via confidence intervals or ad-hoc variance estimates. However, these heuristic measures have no theoretical guarantee that they capture the true underlying stochastic variation of the fuzzing process [32]. Moreover, code coverage, the most common proxy, does not reliably correlate with bug-finding ability [32]. A rigorous, bug-centric framework for quantifying fuzzer robustness remains an open problem.

We observe that variability in fuzzing plays a dual role. On one hand, it undermines reliability: practitioners must run many expensive trials to draw sound conclusions. On the other hand, variability reflects how a fuzzer traverses the input space, jumping between regions due to stochastic mutations. When bugs are clustered, a fuzzer that stays in a bug-rich region finds more bugs; one that drifts away wastes effort. This insight suggests that *controlling* variability, not merely measuring it, can directly improve bug-finding performance. Yet no principled tools for this exist.

This paper addresses both challenges: principled bug-centric robustness evaluation and robustness-aware enhancement. We model the fuzzing process as a Markov chain over the input space, where randomness is rooted, and define robustness as the effort-normalized variance of the bug detection rate. This formulation yields two key results. First, we prove that the sample variance is a reliable proxy for true robustness with explicit finite-sample accuracy, providing a theoretically grounded evaluation criterion; empirically, $M \geq 20$ trials of $T \geq 12$ hours suffice. Second, inspired by rare-event simulation, we propose *splitting*: when a trial discovers a bug, we fork the trajectory into descendant chains that continue exploring the bug-rich region. Splitting is a black-box plug-in that wraps any existing fuzzer without modifying its internals. Via the interdependence between descendant chains, it reduces variability in these regions while concentrating effort where bugs cluster, improving bug discovery.

1.1. Contributions

1) **Robustness metric with theoretical guarantees.** We model fuzzing via Markov chains (input) and stochastic processes (output) and introduce a variance-based robustness measure grounded in the bug detection rate (BDR). We prove that the effort-normalized sample variance converges to the true robustness σ^2 and reveal a direct relationship between robustness and effectiveness (Theorem 4).
2) **Splitting-based enhancement.** We propose a splitting method that forks fuzzing trajectories upon detecting a bug. We prove that splitting never observes fewer bug-triggering events than the MC baseline, and that it increases the BDR whenever the forked trajectories are at least as productive as the original ones—the clustered-bug regime (Theorem 3); in most such regimes it simultaneously reduces variance (Theorem 2), achieving a best-of-both-worlds outcome. Unlike power scheduling [1], [19], splitting does not modify fuzzer internals and composes with any mutation-based fuzzer.
3) **Comprehensive empirical validation.** We evaluate 7 fuzzers on 10 FuzzBench benchmarks (23-hour campaigns; 20 trials per pair for the baseline, 10 root trials for splitting), demonstrating that splitting improves the weighted BDR in *every* fuzzer–benchmark pair (median $+26\%$, up to $+126\%$ for AFL++; Wilcoxon $p = 3.6 \times 10^{-13}$) and reduces effort-normalized variance in 68 of 70 pairs over the 6–23 h window, with terminal reductions of median $3.2\times$ and up to $231\times$ ($p = 2.6 \times 10^{-12}$). We additionally validate on Magma [12] with ground-truth bug identifiers, showing concrete security impact (e.g., AFL++ triggers CVE-2019-19926 in $5/5$ vs. $0/5$ trials under splitting vs. baseline). Both offline and online splitting implementations are provided.

Paper organization. Sections 2–5 cover related work, the stochastic formulation, the evaluation framework, and its theoretical guarantees; Section 6 introduces splitting; Section 7 reports the FuzzBench and Magma experiments; Sections 8–9 distill practical guidance and conclude.

2. Related Work

Mutation-based fuzzing. Input generation broadly falls into grammar-based [10], [13], [34] and mutation-based approaches; the latter dominates, iteratively modifying inputs guided by feedback such as bug signals [24], execution time [27], divergence [11], [26], or coverage [16], [25], with recent LLM-guided variants [22], [36]. Our framework applies to any mutation-based fuzzer as a black-box plug-in.

Fuzzer evaluation. Reliable evaluation is crucial, yet the community lacks consensus on metrics. Klees et al. [15] find that higher coverage does not necessarily yield more bugs; Li et al. [17] advocate richer metrics including unique bug counts; Hazimeh et al. [12] observe that bug counts are subject to substantial randomness; subsequent studies refine practices further [5], [14], [29], [35]. Schlögel et al. [32] conclude from a large-scale study that existing methodologies still lack rigor—surveying prior work using 1–40 trials—particularly regarding how many trials are needed. Our work directly addresses this gap with a theoretically grounded criterion for trial selection and a formal robustness metric.

Robustness in fuzzing evaluation. Prior work approaches robustness through confidence intervals of code coverage [15] or median/percentile statistics [17], [32]. These methods rely on empirical guidelines, employ indirect proxies (code coverage rather than bug detection), and account only for across-trial variability while ignoring within-trial variability. Moreover, none treat variance as an independent evaluation dimension with theoretical backing. Our framework jointly considers inter-trial variability, intra-trial variability, and computational cost, all grounded in bug-centric measures with provable convergence guarantees.

Markov chain modeling in fuzzing. Böhme et al. [1] first model coverage-based greybox fuzzing as a Markov chain over coverage states, deriving the AFLFast energy schedules. Our modeling differs fundamentally: we define the chain over the *input space*, treating coverage and bug-triggering as output functions of it—advantageous because input-level mutation transitions are time-homogeneous for a fixed mutation operator, whereas coverage transitions depend on the growing corpus and are not necessarily Markovian, and because any program behavior (bugs, coverage, sanitizer signals) is a deterministic function of the input chain.

Benchmarking randomness. Madadi et al. [20] empirically show that fuzzer benchmarking outcomes vary substantially with trial count, duration, and benchmark selection. We complement their findings with a theoretical framework that formalizes this randomness through Markov chain theory, provides convergence guarantees, and proposes splitting to reduce the variability they identify.

Rare-event simulation and splitting. Splitting methods originate in rare-event simulation [2], [8], [9], where trajectories are progressively biased toward target regions using hand-tuned importance functions. Our adaptation differs in two respects: (1) splitting triggers directly on the bug-detection output signal, requiring no user-specified importance function; and (2) we analyze splitting's effect on both variance and bug-detection rate in the fuzzing context (Theorems 2, 3).

3. Problem Formulation

We model the fuzzing process as a Markov chain over the input space, with bug-triggering as an output function. This formulation enables a principled treatment of both effectiveness (bug detection rate) and robustness (variance of that rate), as the source of randomness is the input string.

Let the target program be denoted by f with input domain D. Each input $x \in D$ has length l_x (in bits). For a given fuzzer R, define $\{X_R(w,t)\}_{t\geq 0}$ as the input Markov chain, where $X_R(w,t)$ denotes the input string at time t when the process starts from an initial input w. For any fuzzer R, the state space of $X_R(w,t)$ is D. Since D is finite, we use the counting measure and write $\int h(x)\,dx := \sum_{x\in D} h(x)$ for any function $h : D \to \mathbb{R}$.

Upon execution, the program produces trace output $f(x)$ and behavioral output $g(x)$. Let B_f denote the set of abnormal behaviors indicating bugs or vulnerabilities, and define the set of buggy inputs as $B = \{x \in D \mid g(x) \in B_f\}$. We define the output stochastic process as $Y_R(w,t) = \mathbf{1}_{\{g(X_R(w,t))\in B_f\}}$, which indicates whether the current input at time t triggers a bug under fuzzer R. The state space of Y_R is $\Sigma = \{0,1\}$.

A fuzzer $R \in \mathcal{R}$ mutates an input $x \in D$ to produce $R(x) \in D$ through bit-level operations such as flipping, deleting, inserting, or modifying bytes. Each mutation corresponds to a transition in the input Markov chain from x to $R(x)$, with transition probability denoted by $P_R^X(x, R(x))$. The transition matrix of X_R is $\{P_R^X(i,j)\}_{i,j\in D}$.

Let π_R^X denote a stationary distribution of the input Markov chain X_R, i.e., $\pi_R^X(j) = \sum_{i\in D} \pi_R^X(i)\, P_R^X(i,j), \quad j \in D$. At stationarity, the induced transition law of the output process is $E_{x\sim\pi_R^X}\big[P_R\big(\mathbf{1}_{\{g(x)\in B_f\}}, \mathbf{1}_{\{g(R(x))\in B_f\}} \,\big|\, x\big)\big]$. We denote the corresponding 2×2 matrix by $\{P_R(m,n)\}_{m,n\in\{0,1\}}$. The stationary distribution of the output process Y_R induced by π_R^X is then given by $\pi_R(1) = \sum_{x\in D} \pi_R^X(x)\, \mathbf{1}_{\{x\in B\}} = \pi(1), \pi_R(0) = 1 - \pi_R(1) = \pi(0)$. We emphasize that Y_R is a deterministic function of the input chain and is therefore not, in general, a Markov chain itself (a function of a Markov chain is Markov only under lumpability conditions). The 2×2 matrix $\{P_R(m,n)\}$ is the stationary-induced aggregate transition law of the output process, which we use as a tractable two-state surrogate; its adequacy is validated empirically in Section 7.6, where the output autocorrelation of all evaluated fuzzers exhibits the geometric decay this surrogate predicts.

Without loss of generality, we start from a single seed and defer the multi-seed case to Appendix D. Given T mutation rounds, the fuzzer starts from seed w, executes each input to observe $g(\cdot)$ and record buggy inputs, and mutates to produce the next input. Let H_R^T denote the set of bugs discovered and I_R^T the set of executed inputs; this generates realizations $\{X_R(w,t)\}_{t=1}^T$ and $\{Y_R(w,t)\}_{t=1}^T$.

Table 1 summarizes the key notation used throughout.

TABLE 1: Summary of Key Notation

Symbol	Meaning
D	Input domain (finite set of bit strings)
f, g	Program trace and behavioral output functions
B_f, B	Set of abnormal behaviors; set of buggy inputs
$R \in \mathcal{R}$	A fuzzer from the fuzzer set
$X_R(w,t)$	Input Markov chain at time t from seed w
$Y_R(w,t)$	Output process: $\mathbf{1}\{g(X_R(w,t)) \in B_f\}$
P_R^X, $P_R(\cdot,\cdot)$	Transition matrices of input and output chains
π_R^X, π_R	Stationary distributions of input and output chains
T	Total mutation rounds (horizon)
M	Number of independent trials (replications)
P_R	BDR (scalar): $\lim_{T\to\infty} N_T/T$, with $N_T = \sum_t Y_R(w,t)$
$\widetilde{P}_R(T)$	Finite-horizon BDR estimator
σ^2	Effort-normalized asymptotic variance (robustness)
$S_R(T)$, $\widetilde{S}_R(T)$	Finite-horizon variance; sample variance estimator
$\gamma_{M,R}$, δ_R	Spectral gap; relaxation time ($\delta_R = 1/\gamma_{M,R}$)
k_t^m	Active paths at time t in trial m (splitting)

3.1. Modeling

Mutation scope. The input Markov chain X_R captures *any* mutation operator, not only simple bit-flips: splicing, dictionary-based insertion, and havoc-mode multi-byte operations all correspond to transitions with specific probabilities $P_R^X(x, R(x))$. The framework only requires that, given the current input x, the next input $R(x)$ is drawn from a fixed distribution—determined by mutation rules that are fixed in code and hence independent of the time step t. For stateful fuzzers whose mutation rules take internal state G_t as input: if G_t depends only on X_R^t (e.g., coverage delta), the paradigm applies verbatim; if G_t depends on G_{t-1}, we expand the state as $X_{t+1}^R = \mathcal{R}(X_t^R, G_t)$ and use the same *Markov structure*. The output depends only on the input component X_t^R either way.

Scope and limitations. LLM-based fuzzers whose mutation distributions change unpredictably over time fall outside the scope of our current framework. We leave their treatment to future work. The modeling requires two standard assumptions from the Markov chain community [4] (details in Appendix B.1): (i) *time-homogeneity* of the mutation kernel, satisfied when mutation rules are fixed in code as argued above; and (ii) *geometric ergodicity* of the output chain, which excludes only degenerate fuzzers trapped forever in a single input region—the random restarts built into practical fuzzers ensure it. We empirically examine these assumptions in Section 7.6: the output process exhibits approximately geometric autocorrelation decay across all evaluated fuzzers, supporting geometric ergodicity, while a windowed analysis shows that the two-state output surrogate's parameters drift slowly between the early and late campaign phases as easy bugs deplete; campaign-length estimates should accordingly be read as phase-averaged.

Bug detection rate. We define the *bug detection rate* (BDR), which quantifies the fraction of executed inputs that trigger bugs: $P_R = \lim_{T\to\infty} \frac{|H_R^T|}{|I_R^T|} = \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^T Y_R(w,t)$ and its finite-horizon counterpart $P_R(T) = E[\frac{|H_R^T|}{|I_R^T|}] = E[\frac{1}{T}\sum_{t=1}^T Y_R(w,t)]$. This measure is monotone in $|H_R^T|$, the total bug count used in existing evaluation [15], [17], [32], while additionally accounting for the effort over T rounds. Since $|H_R^T|$ depends on realizations of the underlying Markov chains, it is a random variable whose variability has not been previously quantified. We therefore introduce robustness measures that capture the stability of BDR via variance.

Robustness. We consider both terminating and non-terminating regimes, and begin with the non-terminating regime, focusing on the asymptotic behavior of the Markov chains. We define the performance measure in the non-

terminating regime as

$$S_R = \lim_{T\to\infty} Var(\frac{|H_R^T|}{|I_R^T|});$$

$$\sigma^2 := \lim_{T\to\infty} T \cdot Var(\frac{|H_R^T|}{|I_R^T|}) = \lim_{T\to\infty} Var(\frac{1}{\sqrt{T}}\sum_{t=1}^{T} Y_R(w,t)),$$

where $Var(A)$ denotes the variance of random variable A. This variance-based criterion serves as a natural measure of robustness, and the scaling by T captures the effort required to achieve a given level of variability. Including this factor also reflects work efficiency and computational cost, particularly relevant in fuzzing. In practice, mutations run for finitely many rounds, so we also define a finite-horizon measure:

$$S_R(T) = Var(\frac{|H_R^T|}{|I_R^T|});$$

$$\sigma^2(T) = T \cdot Var(\frac{|H_R^T|}{|I_R^T|}) = T \cdot Var(\frac{1}{T}\sum_{t=1}^{T} Y_R(w,t)).$$

Remark 1 (Extension to coverage-oriented benchmarks)**.** *On targets where bugs are unknown or extremely rare, one can replace B_f with a set of coverage targets B_c (e.g., uncovered branches) and define $Y_R(w,t) = \mathbf{1}\{g(X_R(w,t)) \in B_c\}$; since Y_R remains a deterministic function of X_R, all analyses carry over, and the robustness metric quantifies stability of coverage-target discovery, keeping the framework informative even when BDR $= 0$.*

Remark 2 (BDR vs. unique bug counts)**.** *Practitioners ultimately care about the number of* unique *bugs found. We use BDR instead for three reasons: (a) the unique-bug set Z_t depends on both X_t and the history Z_{t-1}, so analyzing it requires a joint chain (X, Z)—a direct extension of our framework; (b) the unique-bug rate $|Z_T|/T \to 0$ as $T \to \infty$ (the bug set is finite), so its variance degenerates (observed on Magma), blocking* ***effort-normalized****, steady-state analysis; (c) BDR and unique-bug discovery are positively correlated in practice—fuzzers that revisit vulnerable regions also discover neighboring bugs—which we validate empirically in Section 7.5 using Magma's ground-truth identifiers.*

4. Evaluation Framework

Figure 1 provides an overview of our framework. All experiments use real fuzzer executions on real programs. The term "Monte Carlo" refers to the evaluation methodology (repeated independent trials), not a synthetic simulator.

We now describe how to estimate the robustness measures of Section 3 from fuzzing data, via (1) a simulation procedure generating M independent trajectories, (2) point estimators for BDR and variance, and (3) evaluation metrics quantifying robustness from complementary perspectives.

Simulation algorithm. Algorithm 1 presents the simulation pseudocode based on a Markov chain (MC) framework: M independent runs, each replicating the fuzzing process for T mutation steps from an initial input w. The

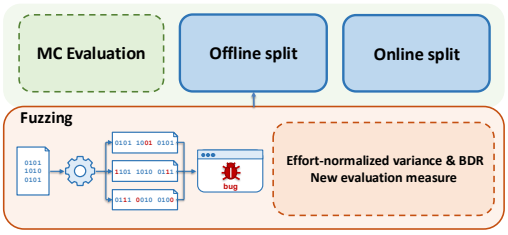


Figure 1: Overview of the framework. Real fuzzer executions generate M independent trajectories, from which we compute effort-normalized variance (robustness) and the bug-detection rate. The MC procedure evaluates fuzzers; splitting enhances them, implemented online or offline depending on prior-data availability.

fuzzer R iteratively mutates the current input according to its transition matrix and executes the results, so the m_{th} run generates one trajectory of the input–output process $\{(X_R^m(w,t), Y_R^m(w,t))\}_{t=1}^T$; repeating this M times yields M i.i.d. sample trajectories.

Algorithm 1: Monte Carlo Simulation Algorithm

Input: replications M; run length T; input seed w; start time $t = 1$; fuzzer R; executed-input and bug-triggering-input sets I_R^T, H_R^T.;
for $1 \le m \le M$ **do**
  $N_T^m = 0$, $H_R^{m,T}, I_R^{m,T} = \emptyset$
  for $1 \le t \le T$ **do** `// Simulation`
    Mutate $s_t = X(w,t)$ with fuzzer R and generate $X(w,t+1)$;
    Add $X(w,t+1)$ to I_R^T
    The software executes $X(w,t+1)$ and generates the feedback; `// Env;`
    Observe $y_R^m(t) = Y(w,t+1)$;
    if $y_R^m(t) = 1$ **then**
      Add $X(w,t+1)$ to H_R^T and update $N_T^m = N_T^m + 1$;
    end
    $s_t = X(w,t+1)$
  end
end
Output: $\{H_R^{m,T}\}_{m=1}^M$, $\{I_R^{m,T}\}_{m=1}^M$, $\{N_T^m\}_{m=1}^M$, $\{y_R^m(t)\}_{m,t}$

Using this classical simulation algorithm, our contribution lies in the design and analysis of the estimators built on the M simulated trajectories, which quantify fuzzing robustness and guide the joint choice of M and T.

Point estimators. We construct two point estimators. The first, $\tilde{P}_R(T)$, is an estimator of the bug detection rate P_R and is defined as follows:

$$\widetilde{P}_R(T) = \frac{1}{M}\sum_{m=1}^{M}\frac{N_T^m}{T} = \frac{1}{MT}\sum_{m=1}^{M}\sum_{t=1}^{T} Y_R^m(w,t),$$

where N_T^m denotes the terminal value of the fuzzing path in the m-th replication, i.e., $N_T^m = \sum_{t=1}^{T} Y_R^m(w,t)$. It is proportional to the common bug-count metric $\sum_m N_T^m$, while accounting for the effort T.

The second, our novel sample-variance estimator $\tilde{S}_R(T)$, captures variability across replications, guides the choice of T and M, and quantifies robustness:

$$\tilde{S}_R(T) = \frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - \frac{1}{T}\sum_{t=1}^{T}\mathbf{1}_{\{X_t^m \in B\}})^2,$$

where in computations we use the unbiased $1/(M-1)$ normalization.

Evaluation metrics. Assessing robustness requires second-moment quantities; we propose several metrics from complementary perspectives. We first evaluate the stability of $\widetilde{P}_R(T)$ using the effort-normalized sample variance $T \cdot \widetilde{S}_R(T)$, which captures cross-replication variability and depends only on simulation outputs, hence is directly computable.

We next quantify the accuracy of this estimator through

$$A_R(T) = \left| T \cdot \tilde{S}_R(T) - \sigma^2 \right|, \qquad B_R(T) = \left| T \cdot V_R(T) - \sigma^2 \right|,$$

where $V_R(T) = \mathbb{E}\big[(\tilde{P}_R(T) - \mathbb{E}[\tilde{P}_R(T)])^2\big]$ denotes the population variance of the estimator.

We also propose new metrics based on mean square errors and variability, and defer them to Appendix B.2.

Lastly, we also evaluate bug-detection ability via $\widetilde{P}_R(T)$ (BDR), equivalently the total bug count $N(T) = \sum_{m=1}^{M} N_T^m$.

5. Evaluation Results

In this section, we evaluate the fuzzers using the proposed metrics and their theoretical guarantees. We defer detailed proofs to the full version of this paper in our publicly available artifact.[1]

Theorem 1 (Informal; formalized as Theorem 4 in the Appendix)**.** *Ignoring constants, set* $U_{A,R}(T) = O\Big(\frac{1}{M} + \frac{1-\gamma_{M,R}}{\gamma_{M,R}^2} + \frac{1}{T}\frac{1}{\gamma_{M,R}^2} + \frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}\Big)$*. With high probability (at least* $1 - 2|\mathcal{R}|\exp\{-cT^2/M\}$*,* $c > 0$ *absolute; the* M*-dependence reflects the deviation radius* $1/M$ *demanded as* M *grows),* $A_R(T) = |T \cdot \widetilde{S}_R(T) - \sigma^2| \le U_{A,R}(T)$ *for every fuzzer* $R \in \mathcal{R}$ *simultaneously. Hence the log-computable* $T \cdot \widetilde{S}_R(T)$ *estimates the true robustness* σ^2 *to within the explicit margin* $U_{A,R}(T)$*, and ranking fuzzers by it ranks them by* σ^2 *up to these margins. The second term—the price of initial-distribution bias—vanishes as the spectral gap grows and sharpens by a further* $1/T$ *factor (see the proof in the artifact); larger-gap fuzzers are thus certified reliably from fewer and shorter trials.*

Theorem 4 guides the selection of the minimum M and T required for reliable evaluation and strong performance.

1. https://anonymous.4open.science/r/split-fuzzing-artifact/README.md

Here, $\gamma_{M,R} = \frac{1}{\delta_R}$ is the spectral parameter of the transition matrix P_R and can be estimated using data-driven methods to evaluate robustness.

Practical interpretation of Theorem 1. If you run $M \ge 20$ independent trials for $T \ge 12$ hours each, the effort-normalized sample variance $T \cdot \widetilde{S}_R(T)$ reliably approximates the true robustness σ^2; fuzzers with smaller σ^2 produce more stable results per unit of compute. The spectral gap $\gamma_{M,R}$ (equivalently, $\lambda_2 = 1 - \gamma_{M,R}$) controls how fast the estimate converges and can itself be estimated from trajectory data (Algorithm 3, Appendix; implemented with `PyDTMC`) with sample complexity $O(1/((1-\lambda_2)\varepsilon^2))$—highly accurate at our experimental scale. Section 7.6 reports $\hat{\gamma}_{M,R}$ for representative pairs and confirms $A_R(T) \le U_{A,R}(T)$ throughout.

The robustness estimator is also a function of $\widetilde{P}_R(T)$, the bug detection rate (BDR), which reflects the effort-normalized capability of discovering bugs. This reveals a direct relationship between robustness and effectiveness, both of which depend on the mutation effort.

6. Enhancement of Fuzzing

The evaluation results in Section 5 reveal that robustness varies substantially across fuzzers and is jointly determined by the fuzzer–benchmark pair. This motivates a natural follow-up: can we *improve* robustness and bug detection simultaneously? We propose a splitting-based method that enhances any existing fuzzer by **reallocating mutation effort** toward bug-rich regions of the input space. The core idea is simple: when a trial discovers a bug, we fork the trajectory into descendant chains that continue exploring from that promising state. Splitting applies as a black-box plug-in without modifying fuzzer internals. Within the rare-event-splitting literature, our method is novel in that it is guided directly by bug signals and requires no handcrafted importance function to decide when to split.

6.1. Relationship to Power Scheduling

How does splitting relate to existing effort-allocation strategies such as power scheduling (AFLFast [1], MOpt [19]) and corpus prioritization? While both concentrate effort in promising regions, they differ fundamentally in mechanism, trigger signal, and guarantees; Table 2 summarizes.

Beyond the table, two distinctions deserve emphasis. First, splitting improves both BDR (Theorem 3) and robustness (Theorem 2) under explicitly stated clustered-bug conditions, whereas power scheduling's analytical results [1] concern path discovery under a coverage model and say nothing about bug-detection variance. Second, our data contain a direct head-to-head on the same ancestor fuzzer: splitting-wrapped AFL attains a higher terminal BDR than plain AFLFast (power-scheduled AFL) and plain MOpt on *all ten* benchmarks at the same wall-clock budget (median gains $+25\%$ and $+22\%$; Wilcoxon $p = 0.002$ each). Splitting also composes with power scheduling: it improves AFL++ (which ships a sophisticated power schedule) by up to 126% in BDR.

TABLE 2: Splitting vs. power scheduling

	AFLFast / MOpt	Splitting (Ours)
Trigger signal	Coverage-based heuristic	Bug-triggering output
Mechanism	Re-prioritize fixed seed pool	Fork new chain from current state
Modifies fuzzer	Yes, internal scheduler	No, black-box plug-in
Theoretical guarantee	Path-hitting under a coverage Markov model [1]; none on bug detection or variance	On BDR and variance: Thm. 2, 3
Effect on robustness	Not analyzed	Reduces variance in the clustered-bug regime
Composable	Within-fuzzer scheduler	Yes, wraps any fuzzer

Practical overhead. Splitting requires only the ability to fork/snapshot the fuzzer's process state and access to the bug-triggering signal (sanitizer report). The fork operation introduces negligible overhead: in our Magma online-splitting campaigns (Section 7.5), the monitor-derived fork dead-time has a median of 60 seconds per 12-hour campaign (0.14% of wall-time; mean 0.16%), with a worst case of 1.1% in 5 of the 187 online trials that completed the full 12 hours. This is because a fork occurs only upon bug detection (a rare event relative to the total number of mutations), and modern operating systems implement copy-on-write forking efficiently. Splitting does *not* require re-executing the entire prefix of the trajectory; the descendant chain inherits the parent's current state and continues from there.

6.2. Methodology based on Splitting

Splitting algorithm. We present the proposed method in Algorithm 2. We retain the MC simulation framework of Section 4 but modify the procedure: whenever a bug is detected, we *split* the run into multiple descendant runs, each initialized at the current input with a randomly sampled seed, and each continuing to fuzz independently (our implementations additionally gate splits by a sparsity-zone signal; Section 7.2 and Appendix A). This produces one trajectory of input-output pairs $\{(X_R(w,t), Y_R(w,t))\}_{t=1}^{T}$ per run; repeating M times yields M i.i.d. sample trajectories.

Point estimators. Generally, the fuzzing path splits whenever a bug is detected. We use the estimator

$$\hat{P}_R^m(T) \;=\; \frac{1}{T}\sum_{t=1}^{T}\frac{1}{k_t^m}\sum_{s=1}^{k_t^m} Y_{s,t},\; \tilde{P}_R(T) = \frac{1}{M}\sum_{m=1}^{M}\hat{P}_R^m(T),$$

where k_t^m denotes the total number of active fuzzing paths at time t along the m_{th} run.

With these definitions in place, we define the sample-variance estimator as $\tilde{S}_R(T) \;=\; \frac{1}{M}\sum_{m=1}^{M}\Big(\tilde{P}_R(T) - \hat{P}_R^m(T)\Big)^2$, $V_R(T) = E[(\tilde{P}_R(T) - E[\tilde{P}_R(T)])^2]$. We also define the total CPU-hours as $\mathrm{CPU}_R(T) \;=\; \sum_{m=1}^{M}\sum_{t=1}^{T} k_t^m$, and estimate σ^2 by $\hat{\sigma}^2(T) \;=\; \mathrm{CPU}_R(T)\cdot \tilde{S}_R(T)$.

Algorithm 2: Splitting Algorithm

Initialization: Maximum path length T, input seed w, $t = 1$, active paths k_t ($k_1 = 1$), fuzzer F, mutation speed $\Delta + 1 \in \mathcal{N}^+$ (default $\Delta = 0$), splits at time t: $n_t \in \mathcal{N}^+$, bug count N_t ($N_0 = 0$), bug-triggering inputs E_t, new paths TMP_t;
for $1 \le t \le T$ **do**
  for *each* $s_t = X(w,t) \in [k_t]$ **do**
    // Simulation
    Mutate $X(w,t)$ with fuzzer F and generate $X(w,t+1),\ldots,X(w,t+\Delta+1)$;
    Execute and observe $Y(w,t+1),\ldots,Y(w,t+\Delta+1)$;
    if $\exists\, 1 \le i \le \Delta+1$ *s.t.* $Y(w,t+i) = 1$ **then**
      Add $X(w,t+i)$ to E_t; update $N_t \leftarrow N_t + 1$ (with N_t initialized to N_{t-1} at the start of round t);
      Split $X(w,t+i)$ into n_t copies $\rightarrow tmp_t$
    else
      Continue with s_t; $tmp_t = \emptyset$
    end
    Add tmp_t to TMP_t ;
  end
  Update $[k_{t+1}] = TMP_t \cup [k_t]$;
  for *each sample path* $s_t \in [k_{t+1}]$ **do**
    Update path weight
  end
end
Output: E_T, N_T and $\{k_t\}_{t=1}^{T}$

6.3. Effectiveness of Splitting

A practically important question is when and how much splitting outperforms MC. The intuition is simple: splitting is most effective when bugs are clustered, enabling thorough exploitation of bug-likely regions—there it reduces variability *and* improves the bug detection rate (best-of-both-worlds). If instead the bug set is so sparse that MC almost fails while splitting discovers substantially more bugs, it may increase variability due to the extra effort across bug regions.

In practice, practitioners need not know a priori whether bugs are clustered. Splitting is triggered *reactively* upon observing a bug, so it automatically concentrates effort only when and where bugs actually appear. If the bug structure turns out to be sparse (no clusters), splitting still guarantees non-regression of the bug-event count (Theorem 3), and the practitioner can monitor variance online and revert to MC if variance increases. Our FuzzBench experiments (Section 7) show that this clustered regime is the common case there: splitting reduces effort-normalized variance in 68 out of 70 FuzzBench fuzzer–benchmark pairs. On Magma's ground-

truth targets, where a handful of planted bugs saturate quickly and trial outcomes often coincide, the variance metric frequently degenerates to zero and this benefit does not transfer (Section 7.5).

6.3.1. Variance Comparison. We consider effort-normalized variance, defined by

$$\mathcal{E}(T) \;:=\; \mathrm{Var}(\text{estimator at horizon } T) \times \mathrm{CPU}(T),$$

and note that $\hat{\sigma}^2(T) = \mathrm{CPU}_R(T)\,\tilde{S}_R(T)$ is an empirical estimator of this quantity. Let $\mathcal{E}_{\mathrm{split}}(T)$ and $\mathcal{E}_{\mathrm{MC}}(T)$ denote the corresponding quantities for Splitting and MC, respectively.

Theorem 2. *In the clustered regime—where the autocovariance contributions to σ^2 are dominated by bug-region visits and splitting increases k_t primarily inside those regions—the ratio satisfies $\frac{\mathcal{E}_{\mathrm{split}}(T)}{\mathcal{E}_{\mathrm{MC}}(T)} \lesssim \frac{T+|B_{split}|}{2T}$, where $|B_{split}|$ is the number of bug-triggering events observed by splitting and $\lesssim$ hides lower-order terms.*

In other words, splitting is most beneficial when it increases k_t primarily in clustered bug regions, thereby reducing the dominant covariance contributions by more than the corresponding marginal increase in CPU cost. We scope the analysis device explicitly: our implementation forks *independent* descendants, and the clustered-regime mechanism above—averaging the dominant within-cluster covariance over the k_t paths (see the case study in the artifact's full version)—applies to independent forks; the coupled-noise construction in the full analysis (Assumptions A1–A3, artifact) is a stylized device quantifying an *additional* reduction available to anti-correlated descendants, which our implementation does not exploit.

6.3.2. Bug Detection Rate Comparison. We denote by $|B_{\mathrm{MC}}|$ the number of bug-triggering events observed by MC. Notably, $|B_{\mathrm{split}}| - |B_{\mathrm{MC}}| = \sum_{t=1}^{T}\sum_{s=2}^{k_t} Y_{s,t} \geq 0$, which means that **splitting never observes fewer bug-triggering events than MC** (the descendant paths can only add events). We note that the bug detection rate of MC is straightforward, $R_{\mathrm{MC}} = \frac{|B_{\mathrm{MC}}|}{T}$. Meanwhile, the bug detection rate of Splitting is $R_{\mathrm{Split}} = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{k_t}\sum_{s=1}^{k_t} Y_{s,t}$.

Theorem 3. *We derive that $R_{Split} - R_{MC} \leq \frac{1}{2T}\Big(|B_{split}| - |B_{MC}|\Big)$, and $R_{Split} \geq R_{MC}$ holds whenever $\sum_{t\in\{k_t=2\}} Y_{2,t} \geq \sum_{t\in\{k_t=2\}} Y_{1,t}$.*

In the clustered regime, we have that $\sum_{t\in\{k_t=2\}} Y_{2,t} - Y_{1,t} > 0$ as splitting starts from or is conditioned on the bug zone. This implies that **splitting increases the bug detection rate in this clustered regime**; outside it, the theorem still guarantees non-regression of the total bug-event count.

6.3.3. Discussions. The derivation bounds the BDR increment by $\frac{1}{2T}\big(|B_{\mathrm{split}}| - |B_{\mathrm{MC}}|\big)$, i.e., relative increment $\frac{|B_{\mathrm{split}}|-|B_{\mathrm{MC}}|}{2|B_{\mathrm{MC}}|}$, while the variance-increment ratio is $\frac{T+|B_{\mathrm{split}}|}{2T}$. **Together these reveal a practically important trade-off between bug-detection gains and robustness under splitting.** When $|B_{\mathrm{split}}|-|B_{\mathrm{MC}}|$ is positive but small, splitting improves upon MC in both bug detection and variance, **achieving best-of-both-worlds**; when it is large, splitting may increase variance in exchange for a more substantial bug-detection gain. The choice depends on practitioners' priorities. Detailed guarantees and proofs are in the artifact's full version.[1]

These theoretical results provide conservative performance guarantees for the splitting methods, establishing a baseline level of improvement. They also offer tools for understanding and interpreting the experimental phenomena in the next section.

7. Numerical Experiments

In this section, we present a comprehensive numerical study of existing fuzzers to examine the proposed evaluation and enhancement framework. We instantiate the variance-based evaluation on FuzzBench using real fuzzing logs, implement the Monte Carlo method of Section 4 alongside an end-to-end deployment of the splitting method (Section 6), and compare both on variance and bug detection rate. We further conduct an ablation study that performs splitting on the fly to isolate the splitting mechanism and facilitate practical adoption. We present representative results here and defer the full results and code to the artifacts.[1]

7.1. Experimental Setup and Data

We evaluate seven widely used mutation-based fuzzers AFL [37], AFLFast [1], AFL++ [6], AFLSmart [28], MOpt [19], LibFuzzer [18], and Honggfuzz [33], following [32]. We test them using Algorithm 1 and Algorithm 2 on 10 well-known program benchmarks from FuzzBench [23]: arrow, ffmpeg, grok, libhevc, libhtp, matio, openh264, php, poppler, and stb, each run for up to $T = 23$ hours. These fuzzers and benchmarks are widely used in practice and in fuzzer evaluation studies [32], making them representative of real-world settings. Moreover, some benchmarks include multiple programs, further enhancing experimental coverage.

Experimental protocol. Fix a benchmark–fuzzer pair (b, f). Each *trial* is a full fuzzing campaign of length $T = 23$ hours. The Monte Carlo (non-splitting) arm comprises $M{=}20$ independent trials per pair; the splitting arm comprises $M{=}10$ independent root trials per pair (7 and 9 for two pairs, libFuzzer/libhevc and Honggfuzz/openh264, whose remaining trials failed infrastructure checks), with descendant paths weighted by $1/k_t$ as in Section 6. This 10-vs-20 design approximately compute-matches the arms: the splitting arm's total CPU (roots plus branches) averages $1.00\times$ the non-splitting arm's 460 CPU-hours (median $1.07\times$; 57/70 pairs within $\pm 20\%$). The compute-matched MC arm thus doubles as the natural "more independent instances at equal total CPU" baseline. FuzzBench records measurements every 15 minutes; we align both arms on a common 6-minute analysis grid, i.e., $\delta = 0.1$ hours and $\mathcal{G} = \{0, \delta, 2\delta, \ldots, T\}$, by forward-filling. All "$\min$ / $\max$ over time" operations in this section are taken over recorded timestamps in $\mathcal{G}$.

TABLE 3: Bug signature statistics per benchmark. Counts are per-trial unique signatures over the 23-hour campaigns, aggregated across all 7 fuzzers and 20 trials. FuzzBench commit: `90e59b6`. Bugs/h = median count divided by the 23-hour horizon.

	arrow	ffmpeg	grok	libhevc	libhtp	matio	openh264	php	poppler	stb
Median/trial	39	7	1	29	3	10	4	1	12	18
Max/trial	47	36	3	34	6	16	7	7	22	22
Bugs/h	1.70	0.30	0.04	1.26	0.13	0.43	0.17	0.04	0.52	0.78

7.1.1. Bug Definition and Deduplication. Sanitizer-based bug oracle. We adopt FuzzBench's default bug-discovery setup and compile each benchmark with LLVM sanitizers. In our experiments, the targets are instrumented with AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan), so that memory-safety violations and undefined-behavior violations deterministically raise *fatal* runtime reports. We treat each fatal sanitizer report as evidence of a bug-triggering behavior.

What we count as a *bug*. A bug is *not* an individual failing input. Many distinct inputs can trigger the same underlying defect, and counting failing inputs would therefore overestimate bug-finding progress. Instead, we count *deduplicated failure signatures*. For every fatal sanitizer report, we form a signature from (i) the sanitizer-reported error class and (ii) a canonicalized stack-trace signature (normalized to be stable across repeated occurrences). Within each trial, we count each distinct signature at most once. A bug is discovered when its signature appears for the first time in that trial, and any subsequent occurrences of the same signature do not increase the bug count, regardless of how many different inputs trigger it.

Scope. Signature-based deduplication is a practical proxy for root-cause bugs. A single defect can manifest as multiple distinct signatures (over-counting), and multiple defects may occasionally collapse to the same signature (under-counting). Our claims are therefore scoped to discovering unique sanitizer failure signatures; we additionally validate this proxy using benchmarks with ground-truth bug identifiers (Magma [12]) in Section 7.5. We also note the bridge to the theory: the theoretical BDR counts bug-triggering *events*, while FuzzBench logs expose only deduplicated signature counts, so the FuzzBench figures track unique-signature rates; Magma's monitor exposes raw trigger events, on which the weighted $\widehat{P}_R$ operates directly, and the two quantities are positively correlated across trials (Spearman $\rho = 0.55$; Section 7.5).

Clarification on figure y-axis. We note that the bug-detection-rate figures in Section 7.3.3 plot *cumulative unique signatures divided by elapsed wall-clock hours* (bugs per hour), the standard per-unit-compute normalization used in FuzzBench [23]. This quantity can exceed 1 because it is a rate per hour, not a per-execution probability: the theoretical BDR $\widetilde{P}_R(T)$ is the per-mutation rate and equals the plotted rate divided by the fuzzer's hourly execution throughput. FuzzBench does not log per-trial execution counts—throughput is an innate property of each fuzzer rather than an evaluation knob—so we report the time-normalized rate throughout. For example, on LIBHEVC, the value $\approx$1.3 bugs/hour at t=23h corresponds to a median of 29 unique signatures over the 23-hour campaign. For readers who prefer familiar absolute counts, Table 3 reports per-trial unique signature counts for each benchmark.

7.2. Implementation

The MC simulation (Algorithm 1) directly mimics the fuzzing process. Appendix A details how we implement the splitting method (Algorithm 2) for real-world fuzzers: (1) compute a sparsity signal from the aggregate bug curve of a *prior, completed non-splitting campaign* (historical data), (2) declare a sparsity-zone entry once the signal stays below a stage threshold with persistence, and (3) execute a split only when a running trial observes a new bug after zone entry. Because the schedule is fixed from historical data before the splitting campaign runs, the splitting trials remain i.i.d. and use no information from their own future. An online version requiring no history follows in Section 7.4.

7.3. Experiment Results

Herein, we present results for the fuzzers and benchmarks, focusing on robustness (effort-normalized variance) and effectiveness (bug detection rate). For each benchmark–fuzzer pair, we compute the sample variance of $P(t)$ across repeated trials and report its *CPU-hour–normalized* form, $\hat{\sigma}^2(T) = \mathrm{CPU}_R(T)\, S_R(T)$, so that robustness is compared under a matched compute budget. Smaller normalized variance indicates more stable (and thus more reliably estimable) outcomes per unit of compute. We also report $\tilde{P}(t)$, which is likewise effort-normalized and therefore comparable on a fair basis. Larger $\tilde{P}(t)$ indicates more bugs detected per unit, reflecting stronger bug-detection capability. We present a subset of results in this section and defer the full results to the artifacts.

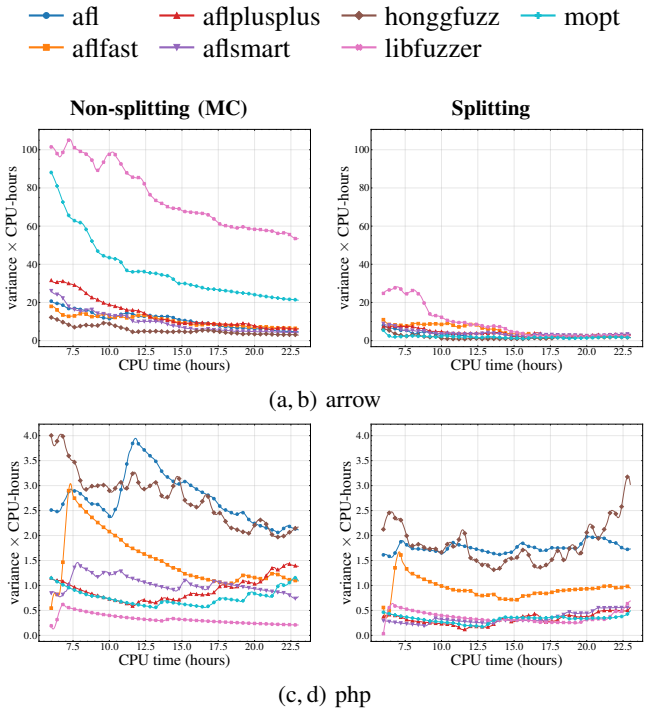
7.3.1. Monte Carlo Simulation. We first present the baseline variability under standard Monte Carlo evaluation (*MC* or *non-splitting*). Figure 2 reports the CPU-hour–normalized variance across fuzzers for two representative benchmarks; all ten appear in the artifact.

Each row corresponds to one benchmark; the left column shows Monte Carlo (non-splitting) and the right column shows splitting, with one curve per fuzzer, and within each subfigure each curve corresponds to one fuzzer. The horizontal axis shows CPU running hours, and the vertical axis shows variance normalized by consumed CPU hours (variance $\times$ CPU-hours). We emphasize that comparisons should be made *within* each benchmark panel: the per-panel y-axis range is chosen for readability, and the salient signal is the *relative* separation and decay behavior of curves over time. The consistent decay observed on most benchmarks aligns with the theoretical evaluation results in Section 5, where the evaluation criterion decreases with T. Within the same

benchmark, different fuzzers exhibit distinct variance levels and decay rates, suggesting varying degrees of robustness and providing guidance for fuzzer selection.

Across benchmarks, we observe pronounced heterogeneity in how variance evolves over time: on LIBHEVC, LIBHTP, and STB, the across-trial variance of the bug-detection ratio typically decreases as the campaign progresses, whereas on PHP and POPPLER it remains roughly flat or increases in later stages. This indicates that robustness is not an intrinsic property of a fuzzer alone; rather, it is jointly determined by the fuzzer–benchmark pair and can change materially with the elapsed fuzzing time.

These results confirm the value of normalized variance as a robustness criterion: it distinguishes among fuzzers on the same benchmark and generalizes across all ten benchmarks.



(a, b) arrow

(c, d) php

Figure 2: Variance ($\times$ CPU-hours) across the seven fuzzers under Monte Carlo (non-splitting, left) and splitting (right), for two representative benchmarks (one per row). The remaining eight benchmarks are shown in the artifact.

7.3.2. Splitting Simulation. We then evaluate the same criterion, normalized variance, under *splitting*. Figure 2 places each benchmark's Monte Carlo (left column) and splitting (right column) results side by side, enabling a direct visual comparison.

A salient pattern is that splitting changes not only the *level* but also the *temporal profile* of the variance. On multiple benchmarks (e.g., arrow, libhevc, and libhtp), the variance trajectories under splitting contract earlier and stay within a lower range for a substantial portion of the run; while many curves then plateau, some benchmarks such as ffmpeg and poppler instead fluctuate within a narrow band with a mild late upward drift.

Although variance reduction is the dominant outcome, splitting is not universally variance-improving. In our 7×10 benchmark–fuzzer experiments, we observe only a small number of corner cases where splitting yields higher normalized variance than standard Monte Carlo evaluation; concretely, these cases correspond to libFuzzer on grok and php. This observation is consistent with our theoretical discussion: Theorem 2 bounds the variance in a manner that increases with the number of bugs discovered under splitting, and Section 6.3.3 explains how sufficiently large gains in discovered-bug set size can move the process into a regime where splitting trades robustness for larger bug-detection improvements. We therefore interpret these corner cases as evidence of a quantifiable trade-off.

The relative performance of fuzzers on the same benchmark also differs, and their separation differs from the MC ordering in Figure 2: splitting's impact is heterogeneous across fuzzers.

The performance of the same fuzzer also varies across benchmarks. On representative benchmarks like matio and openh264, honggfuzz's variance curves under splitting drop to a much smaller range and stabilize earlier in time, indicating improved robustness. On other benchmarks like php and poppler, the variance continues to fluctuate.

7.3.3. Comparisons. Next, we compare the Monte Carlo and splitting methods on robustness, and then on bug-detection effectiveness.

Variance comparisons. Fixing the fuzzer and comparing its CPU-hour–normalized variance across benchmarks, the results fall into two stylized cases: (1) the majority, where splitting yields *lower* normalized variance, and (2) corner cases where it is *higher*.

Specifically, Figure 3 illustrates representative fuzzer aflplusplus on five representative benchmarks; the full ten-benchmark grid is in the artifact. In the majority regime (for each of the six fuzzers afl, aflfast, aflplusplus, aflsmart, honggfuzz, and mopt on all ten benchmarks, and for libFuzzer on eight of the ten benchmarks), splitting reduces the CPU-hour-normalized variance averaged over the displayed $[6, 23]$-hour window—68 of the 70 pairs; at the terminal 23-hour snapshot alone the count is 66 of 70 (AFLFast/stb ends 6% higher and Honggfuzz/php 38% higher). Among the winning pairs, the terminal variance reduction has median $3.2\times$ and reaches $231\times$ (Honggfuzz on libhevc; two further pairs collapse the split variance to numerically zero). Across the 70 pairs, a Wilcoxon signed-rank test on the terminal effort-normalized variance confirms the reduction is systematic ($p = 2.6 \times 10^{-12}$). The complete set of comparison figures is in the artifact. Meanwhile, the time-resolved curves show that the magnitude of variance reduction is not uniform: splitting often yields the largest gains in the mid-phase of the run, while the improvement can diminish (or partially rebound) later.

Occasionally, the rare corner regime of libFuzzer on grok and php (figures in the artifact) shows increased variance,

consistent with Section 6.3.3: when splitting substantially enlarges the effective bug set, the theory predicts a potential increase in variability—a robustness–effectiveness trade-off, not an anomaly. In most regimes splitting operates within the threshold where it reduces variance; a practical response is to monitor variance online and revert to MC once it exceeds the baseline.

Bug detection rate comparisons. Motivated by this phenomenon and the discussion in Section 6.3.3 regarding the benefits of splitting for bug detection, we next compare the bug-detection capability of fuzzers with and without splitting. Notably, this capability is reflected by the effort-normalized bug rate, which corresponds to bug counts per unit of compute—an evaluation measure commonly used in fuzzing—and thus justifies our choice.

Specifically, Figure 4 compares the time-resolved bug-detection rate (BDR) for aflplusplus: in each per-benchmark panel, the solid curve is the splitting arm and the dashed curve the non-splitting baseline. Across all 70 pairs, the terminal weighted BDR under splitting (per-trial terminal counts) is strictly greater than non-splitting in every pair, with median relative improvement $+26\%$ and up to $+126\%$ for AFL++ (on poppler); a Wilcoxon signed-rank test across pairs gives $p = 3.6\times10^{-13}$, and per-pair Mann–Whitney tests on per-trial terminal bug counts favor splitting significantly ($p < 0.05$) in 50 of 70 pairs (35 after Holm correction; median Vargha-Delaney $\hat{A}_{12} = 0.85$, a large effect), with zero pairs significantly favoring non-splitting. For the rare cases when the splitting has higher variance, i.e., libFuzzer on grok and php, splitting also has the most substantial improvements of bug detection rate (up to $+800\%$ on libFuzzer/php; figures in the artifact). On budget accounting: the arms are approximately compute-matched in total (Section 7.1) and compared via the $1/k_t$-weighted estimator at matched wall-clock time, the quantity our theory targets; within a single trial, raw signatures per consumed CPU-hour are generally lower under splitting, as descendants deliberately duplicate effort inside bug regions.

Importantly, and consistent with the theory in Section 6.3.3, splitting yields a higher bug-detection rate in every evaluated fuzzer–benchmark pair, and hence greater effectiveness. Moreover, the improvement in bug-detection rate is more pronounced in cases where splitting exhibits higher variance, indicating that the cost of increased variance brings corresponding gains in bug discovery, thereby illustrating the enhancement. When the improvement is less pronounced, splitting achieves a best-of-both-worlds outcome, as it improves bug-detection capability while also reducing variance, again demonstrating its uplift.

7.4. Ablation Study

To isolate the effect of the splitting operation itself, without the historical sparsity zone of Section 7.2, we conduct an ablation with an *on-the-fly* variant whose split decisions use only data from the running experiment. Beyond evaluation, this variant is independently practical: it applies to new targets with no logged history. We present representative results; the full set is in the artifacts.

Implementation. The online variant retains the offline structure of Section 7.2 but computes the sparsity signal from a *prefix-only* statistic available at the current time, instead of the tail-based signal over a completed historical trajectory—removing the dependence on logged data. The ablation thus preserves the splitting mechanism, modifies only the sparsity score, and is broadly deployable; details are in Appendix A.2.

Ablation results. We conduct the ablation study on three representative fuzzers, aflplusplus, aflsmart, and mopt, and four benchmarks, arrow, php, poppler, and stb; complete results are reported in the artifacts. Notably, these choices are sufficiently general to cover a variety of fuzzers and benchmarks. Figure 5 reports both (i) normalized variance and (ii) bug detection rate over time for the fuzzer aflplusplus on the arrow and poppler benchmarks for illustration (php, stb, and the mopt and aflsmart results are in the artifact). The figures share the same layout as those in Sections 7.3.2 and 7.3.3.

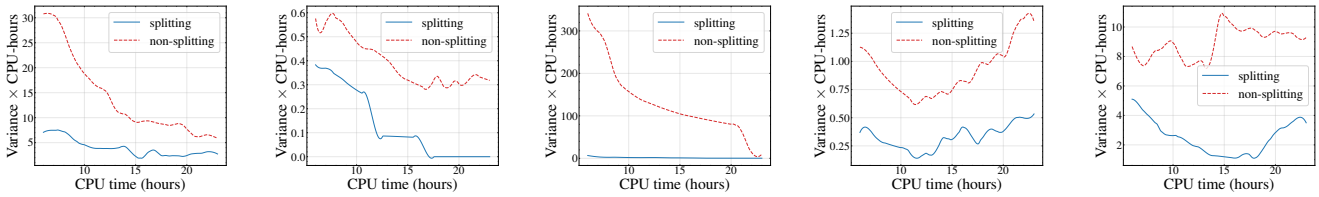
Across the evaluated subset, the online variant preserves the qualitative behavior of splitting. It reduces normalized variance relative to MC (non-splitting) in all twelve fuzzer–benchmark settings (terminal and window-averaged), and improves the terminal bug-detection rate over MC in all twelve. This indicates that triggering splitting upon each new bug is itself an effective enhancement compared to Monte Carlo, and achieves best-of-both-worlds. At the same time, the offline-informed policy is more stable, with lower window-averaged normalized variance than the online variant in 11 of the 12 settings (8 of 12 at the terminal snapshot), showing the advantage of leveraging prior knowledge when it is available.

Together, these results show that *offline guidance is not required* for splitting to enhance performance, but that *it adds a measurable robustness gain* by stabilizing when and how aggressively a run is split.

7.5. Ground-Truth Validation with Magma

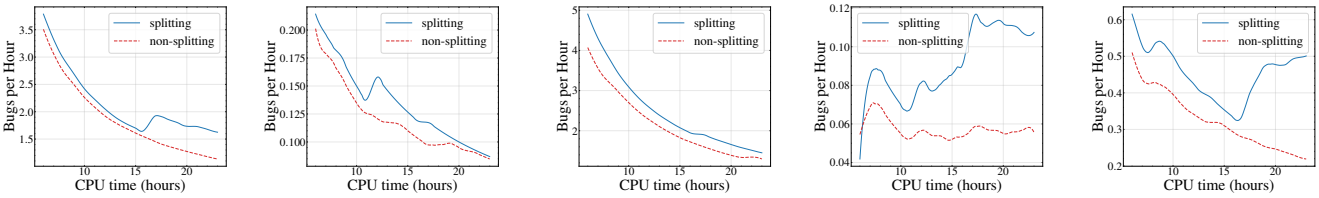
To verify that our signature-based deduplication faithfully tracks true root-cause bugs, and to demonstrate concrete security impact, we conduct additional experiments on Magma [12], a benchmark whose targets are seeded with known, individually identified bugs, each tied to a real CVE or canonical Magma ID. Unlike the signature proxy, Magma's monitor reports exactly which planted bug is *reached* and *triggered* on every execution, providing an unambiguous ground truth.

Setup. We evaluate five of the seven mutation-based fuzzers (AFL, AFLFast, AFL++, MOpt, Honggfuzz; the five supported by Magma's harness in our setup) on eight Magma programs (libpng, libtiff, libsndfile, poppler, sqlite3, openssl, libxml2, php). Each fuzzer–program pair is run for $M{=}5$ independent trials of $T{=}12$ hours under both the baseline Monte Carlo (non-split) and splitting (Online)



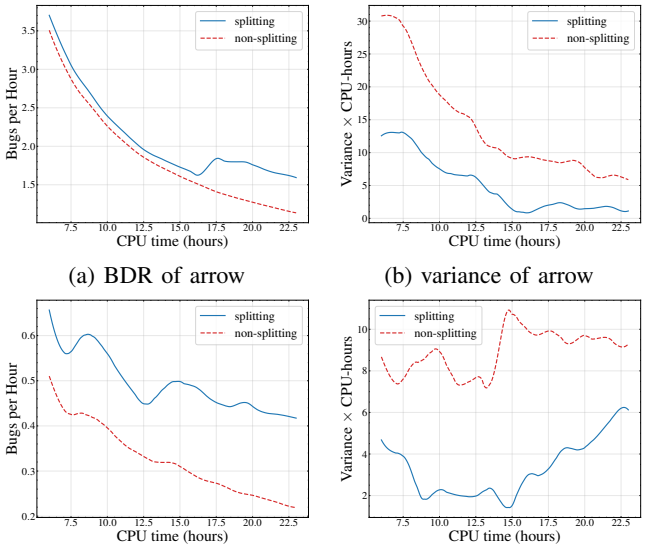
(a) variance of arrow (b) variance of grok (c) variance of libhevc (d) variance of php (e) variance of poppler

Figure 3: Comparisons of variance (splitting vs. Monte Carlo) for fuzzer aflplusplus on five representative benchmarks. The full ten-benchmark grid is in the artifact.



(a) BDR of arrow (b) BDR of grok (c) BDR of libhevc (d) BDR of php (e) BDR of poppler

Figure 4: Comparisons of bug detection rate (bugs per hour) for fuzzer aflplusplus on five representative benchmarks. Y-axis shows cumulative unique signatures divided by elapsed hours. The full ten-benchmark grid is in the artifact.



(a) BDR of arrow (b) variance of arrow

(c) BDR of poppler (d) variance of poppler

Figure 5: Ablation study for aflplusplus on arrow and poppler; php and stb are in the artifact.

configurations.[2] Magma's built-in monitor records the set of ground-truth bugs triggered in each trial, from which we compute—*aggregated over all trials*—the mean and total (union) number of distinct bugs, the weighted bug-detection rate $\widehat{P}_R$, and its effort-normalized variance. Both arms run for the same 12-hour wall-clock budget; the splitting estimators weight descendant branches by $1/k_t$ and the variance is effort-normalized, so both metrics account for the additional compute that splitting's branches consume.

Results. Table 4 summarizes, per benchmark, how splitting compares to non-split across the five fuzzers; the complete per-fuzzer table is in Table 7 (Appendix). Across all 40 fuzzer–program pairs, splitting *ties or improves* the unique-bug count in every case, with no regressions on either measure: 25 wins / 15 ties on mean unique bugs (Wilcoxon signed-rank across cells, $p = 1.2 \times 10^{-5}$) and 18 wins / 22 ties on total (union) unique bugs ($p = 9.6 \times 10^{-5}$). It raises the weighted detection rate $\widehat{P}_R$ on 39 of the 40 pairs ($p = 3.9 \times 10^{-8}$); the sole exception, AFLFast on libtiff, is a near-tie (1.992 vs. 2.000). Every benchmark is a clean win-or-tie—sqlite3 is a clean sweep on total unique (5/5), and even libtiff, the only target where the splitting estimator marginally trails non-split on $\widehat{P}_R$ for one fuzzer (the trade-off of Section 6.3.3), is a net win on both unique-bug measures. With M=5 trials, per-cell comparisons have limited power; per-trial Mann–Whitney tests (tie-corrected normal approximation) are individually significant ($p < 0.05$) in 8 of 40 cells—notably sqlite3/AFL++ ($p = 0.006$)—and in no cell do they favor non-split.

Robustness on Magma: where the variance metric saturates. The variance picture on Magma differs sharply from FuzzBench, and we report it in full. Each Magma target plants a handful of bugs, several triggered within minutes in every trial, so trial outcomes often coincide exactly: in Table 7 the non-split effort-normalized variance is *exactly zero* in 17 of 40 cells, and splitting's variance is strictly larger in 21 of 40 (in 14 of these the baseline is zero, so any extra exploration registers as an increase), smaller in 16, tied in 3. In this saturated, few-bug regime the effort-normalized variance is uninformative as a robustness criterion—the degeneracy

2. Two splitting cells (AFL/libtiff and AFLFast/libtiff) completed 4 of 5 trials; their statistics are over the completed trials, with no imputation.

Remark 2 anticipates as $|Z_T|/T \to 0$—and the clustered-regime premise of Theorem 2 does not bind. We therefore scope the variance-reduction findings to FuzzBench, and use Magma for what its ground truth uniquely establishes: splitting never finds fewer real bugs, converts unreliable CVE detections into reliable ones, and raises the weighted detection rate in 39 of 40 pairs.

Security impact. Beyond aggregate counts, splitting converts *unreliable* detections into reliable ones and surfaces bugs the baseline never finds. On sqlite3, AFL++ triggers CVE-2019-19926 (SQL014) in all $5/5$ splitting trials but $0/5$ non-split trials. On poppler, MOpt discovers three bugs only under splitting—CVE-2019-14494 (PDF001), CVE-2019-7310 (PDF011), and PDF008—lifting its total from 3 to 6; Honggfuzz additionally triggers CVE-2016-6302 (SSL020) on openssl (total $1\to2$) and CVE-2016-10269 (TIF006) plus CVE-2019-7663 (TIF009) on libtiff (total $4\to6$)—precisely the clustered-bug regions where Theorem 3 predicts the largest bug-detection benefit.

Detection-rate regime and proxy validity. The $1/k_t$-weighted bug-detection rate $\widehat{P}_R$ ranges over $[0.40, 8.97]$ across pairs—a weighted count of bug-triggering events per run, not the (far smaller) per-execution probability; hence $\pi(1) \ll 1$ for every pair, the rare-bug regime where splitting is provably beneficial (Section 6.3.3).

Ties concentrate on saturated pairs where both methods already recover every quickly-reachable bug (e.g. php, where all five fuzzers tie on total (union) unique-bug counts yet splitting still raises $\widehat{P}_R$). Finally, these ground-truth results support the signature-based proxy and the BDR-vs-unique-bug assumption (Remark 2) in two ways. First, the direction of the splitting effect on planted Magma bugs matches the effect measured with deduplicated signatures on FuzzBench. Second, across all 398 Magma trials, the number of bug-triggering events in a trial correlates positively with the number of unique ground-truth bugs it finds (Spearman $\rho = 0.55$, $p = 6.7 \times 10^{-33}$), directly supporting the premise that fuzzers which trigger bugs more often also discover more distinct bugs.

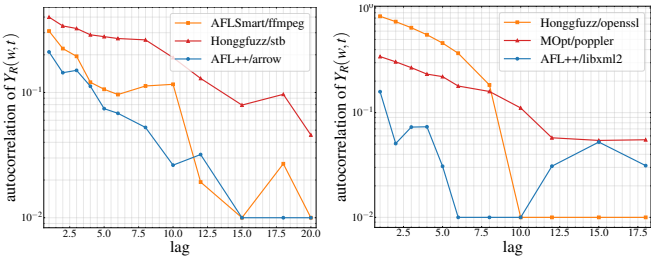
7.6. Empirical Validation of Modeling Assumptions

To empirically examine the time-homogeneity and geometric ergodicity assumptions discussed in Section 3.1, we analyze the autocorrelation structure and spectral properties of real fuzzer trajectories from our experiments.

Autocorrelation analysis. We compute the empirical autocorrelation of the bug-triggering output $Y_R(w,t)$—the indicator that the current execution triggers a bug—across increasing lags for six representative fuzzer–benchmark pairs, spanning three distinct fuzzers on three distinct benchmarks in each suite: AFLSmart/FFMPEG, Honggfuzz/STB, and AFL++/ARROW on FuzzBench, and Honggfuzz/OPENSSL, MOpt/POPPLER, and AFL++/LIBXML2 on Magma. Figure 6 plots the autocorrelation against lag on a logarithmic vertical axis. In every case the autocorrelation decays with lag toward the estimation-noise floor, exhibiting approximately geometric (exponential) decay. The pattern holds across both suites and for the adaptive schedulers AFL++ and MOpt, supporting geometric ergodicity even under complex scheduling.

TABLE 4: Magma ground-truth validation (summary): five fuzzers on each of eight Magma programs, $M{=}5$ trials of 12 h, splitting vs. non-split (MC). *Win/Tie/Lose* (denoted by W/T/L) counts, over the five fuzzers, how often splitting wins/ties/loses on mean unique bugs. Σ mean uniq and mean $\widehat{P}_R$ are summed / averaged over the five fuzzers (*splitting*/*non-split* denoted by s/n); **bold** marks the better value. The full per-fuzzer breakdown is in Table 7.

Benchmark	W/T/L	Σ mean uniq (s/n)	mean $\widehat{P}_R$ (s/n)
libpng	4/1/0	**11.0** / 9.0	**4.80** / 1.76
libtiff	4/1/0	**19.1** / 15.8	**3.93** / 2.96
libsndfile	1/4/0	**24.4** / 24.0	**6.55** / 4.04
poppler	3/2/0	**14.2** / 11.2	**5.03** / 2.00
sqlite3	4/1/0	**11.6** / 9.0	**4.16** / 1.76
openssl	4/1/0	**8.2** / 6.6	**4.36** / 1.32
libxml2	4/1/0	**12.4** / 10.4	**4.88** / 2.08
php	1/4/0	**13.0** / 12.0	**5.62** / 2.32
All 8	**25/15/0**	**113.9** / 98.0	**4.92** / 2.28



(a) FuzzBench (b) Magma

Figure 6: Empirical autocorrelation of $Y_R(w,t)$ versus lag (log vertical axis) for six representative fuzzer–benchmark pairs (three per suite). The approximately geometric decay toward the estimation-noise floor supports geometric ergodicity.

Spectral gap estimation. Using Algorithm 3, we estimate the spectral gap $\hat{\gamma}_{M,R}$ and relaxation time $\hat{\delta}_R = 1/\hat{\gamma}_{M,R}$ from the same trajectory data, by forming the empirical $2{\times}2$ output transition matrix and setting $\hat{\gamma}_{M,R} = 1 - |\hat{\lambda}_2|$ from its second eigenvalue $\hat{\lambda}_2$. Table 5 reports the results. The estimated relaxation times $\hat{\delta}_R$ are moderate, ranging from 1.2 to 6.0, so the bounds in Theorem 4 remain informative on these chains (loosest for Honggfuzz/openssl, whose small gap yields $U_{A,R} = 17.3$ against $A_R = 0.16$). These times are in units of the measurement grid (15-minute snapshots on FuzzBench; 60-second snapshots on Magma), i.e., roughly 20–90 wall-clock minutes on FuzzBench: they quantify mixing of the *snapshot-level output surrogate*, not per-execution mixing. Fast within-trial mixing at this scale is fully compatible with the large cross-trial spread that motivates this work (1–13 unique bugs for AFL++/poppler): that spread is driven by slow cross-trial corpus path-dependence, which the surrogate

TABLE 5: Estimated spectral parameters from trajectory data via Algorithm 3 (non-split chains): second eigenvalue $\hat{\lambda}_2$ of the empirical 2×2 output transition matrix, spectral gap $\hat{\gamma}_{M,R} = 1 - |\hat{\lambda}_2|$, relaxation time $\hat{\delta}_R = 1/\hat{\gamma}_{M,R}$ (from full-precision $\hat{\gamma}_{M,R}$). $U_{A,R}$ is the Theorem 4 bound at the campaign horizon (T=23 h, M=20 FuzzBench; T=12 h, M=5 Magma); $A_R = |\, T \cdot \widetilde{S}_R(T) - \sigma^2 \,|$ is the observed deviation, with σ^2 estimated by the two-state plug-in $\hat{\pi}_1(1-\hat{\pi}_1)(1+\hat{\lambda}_2)/(1-\hat{\lambda}_2)$. $A_R \le U_{A,R}$ in every case.

Fuzzer / Benchmark	$\hat{\lambda}_2$	$\hat{\gamma}_{M,R}$	$\hat{\delta}_R$	$U_{A,R}$	A_R
FuzzBench					
AFLSmart / ffmpeg	0.33	0.67	1.48	1.24	0.00
Honggfuzz / stb	0.41	0.59	1.68	2.59	0.30
AFL++ / arrow	0.22	0.78	1.28	0.39	0.12
Magma					
Honggfuzz / openssl	0.83	0.17	5.96	17.27	0.16
MOpt / poppler	0.34	0.66	1.53	0.66	0.07
AFL++ / libxml2	0.16	0.84	1.19	0.28	0.01

does not model pointwise but which the framework captures directly through the cross-trial variance $\widetilde{S}_R(T)$ and the early/late parameter drift quantified next. We also verify that the theoretical bound $U_{A,R}(T)$, evaluated at the campaign horizon ($T = 23$ h for FuzzBench and $T = 12$ h for Magma), is consistent with the empirical $A_R(T)$ of Theorem 4 in all cases: $A_R(T) \le U_{A,R}(T)$ for every pair (last two columns).

Time-homogeneity window check. The input-level homogeneity argument of Section 3.1 does not by itself pin the two-state *output* surrogate over time: as easy bugs deplete, the firing rate $\hat{\pi}_1$ falls and the surrogate's parameters can drift even though the mutation rules are static. Re-estimating the output transition matrix separately on the early and late halves of every trial, we find $\hat{\gamma}$ rises from early to late for the six pairs of Table 5 ($|\Delta\hat{\gamma}|$ from 0.08 to 0.86, exceeding twice the trial-bootstrap standard error)—driven on FuzzBench by the late decline of $\hat{\pi}_1$ (e.g., Honggfuzz/stb: $0.39 \to 0.05$) and, on the two near-saturated Magma pairs, by pinning at the firing state ($\hat{\pi}_1 \ge 0.998$ late)—while two additional adaptive-scheduler pairs, AFL++/poppler and MOpt/libxml2, are statistically consistent with homogeneity ($|\Delta\hat{\gamma}| = 0.003$ and 0.063). The drift is thus slow, systematic, and concentrated in the output projection (easy-bug depletion) rather than abrupt regime switching: strict homogeneity is an idealization, the campaign-length estimates of Table 5 are phase-averages, and the framework accommodates the drift via windowed estimation (Appendix B.1).

8. Implications for Fuzzing Practice

We distill our findings into actionable guidance for practitioners and evaluators.

How many trials are enough? Based on Theorem 4, the effort-normalized sample variance $T \cdot \widetilde{S}_R(T)$ reliably estimates true robustness σ^2 when M and T are sufficiently large; our experiments validate that $M \ge 20$ trials of $T \ge 12$ hours give stable estimates. Use $M \ge 20$ for definitive rankings; $M = 10$ may suffice for exploratory assessment. By this standard our Magma validation (M=5) falls below even the exploratory threshold; we accordingly treat its per-cell comparisons as directional and base conclusions on aggregate tallies across all 40 cells.

TABLE 6: Fuzzer ranking from the MC (non-splitting) arm: mean over the 10 benchmarks of the terminal (23 h) bug-detection rate (bugs/h; higher is better) and of the effort-normalized variance (variance × CPU-h; lower is better). Parenthesized rank, 1 = best.

	Hongg.	AFL	MOpt	AFLFast	AFLSm.	AFL++	LibF.
BDR	0.595 (1)	0.551 (2)	0.542 (3)	0.537 (4)	0.529 (5)	0.511 (6)	0.280 (7)
Var	28.7 (7)	11.7 (5)	10.7 (4)	3.4 (1)	12.1 (6)	5.9 (2)	7.4 (3)

When to use splitting? Splitting is most beneficial when bugs are clustered in the input space (Theorem 2): if bugs arrive in bursts followed by long silent periods—precisely, if the sparsity signal $\rho(t) < \theta$ persists for H hours (Appendix A)—the fuzzer is in a sparse tail where splitting can reallocate effort to the last-discovered bug region. On FuzzBench, splitting improved the weighted BDR in every pair (median $+26\%$, up to $+126\%$ for AFL++) and reduced effort-normalized variance in 68 of 70 pairs over the 6–23 h window, with terminal reductions of median $3.2\times$ and up to $231\times$; on Magma's saturated ground-truth targets the variance metric degenerates, but splitting still never finds fewer real bugs (Section 7.5).

Fuzzer selection. Evaluate fuzzers on effectiveness ($\widetilde{P}_R(T)$) and robustness ($T \cdot \widetilde{S}_R(T)$) simultaneously: a high-BDR, high-variance fuzzer may behave unreliably across deployments—Honggfuzz ranks first in effectiveness but last in robustness, while AFLFast pairs mid-pack effectiveness with the best robustness (Table 6).

Integration of splitting. Splitting is a black-box plug-in requiring only (1) the ability to fork/snapshot the fuzzer's process state and (2) access to the bug-triggering signal; overhead is negligible (median 0.14% of wall-time in our Magma campaigns). Both variants beat MC in our ablation, and the offline variant is more stable (lower windowed variance than online in 11 of 12 settings): prefer it when historical data exists, and fall back to the online variant for new targets.

Actionable recommendations. Future evaluations should (1) report $M \ge 20$ trials; (2) report both BDR *and* variance; (3) state per-trial unique-signature counts; and (4) consider splitting, which composes with any strategy.

9. Conclusion

We develop a principled framework for evaluating fuzzing robustness via computable effort-normalized variance, and a black-box splitting method that branches fuzzing in bug-prone regions, improving BDR while typically reducing variance; experiments on FuzzBench and Magma validate both, with security impact on real CVEs.

Ethical Considerations

This work introduces new security-oriented metrics for assessing mutation-based fuzzing techniques. We conduct empirical evaluations of existing fuzzers on widely adopted, publicly accessible benchmark suites (e.g., FuzzBench), focusing on mature, legacy versions of these programs. All experiments are performed exclusively in non-production environments.

The study does not reveal previously unknown vulnerabilities in deployed real-world systems, nor does it create risks of negative impact on individuals. Consequently, responsible disclosure is not required, and the research poses minimal risk of infringing on individual rights. Overall, the work follows established responsible research standards and deliberately avoids any activities that could harm software users or operational systems.

Open Science

In compliance with the open science policy, we make all research artifacts of this paper available. The artifacts comprise (i) the complete experiment data underlying every figure and table (FuzzBench per-trial logs for both the Monte Carlo and splitting arms, and the Magma ground-truth monitor data), (ii) the implementation of the offline and online splitting methods, (iii) all analysis and plotting scripts that reproduce every figure, table, and statistical test in the paper, and (iv) the full version of this paper with all technical proofs, additional theoretical results, and the complete per-fuzzer figure grids. The artifacts are available anonymously at https://anonymous.4open.science/r/split-fuzzing-artifact/README.md and will be archived in a permanent repository upon publication.

References

[1] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2016.

[2] Frédéric Cérou and Arnaud Guyader. Adaptive multilevel splitting for rare event analysis. *Stochastic Analysis and Applications*, 25(2):417–443, 2007.

[3] Michael CH Choi. Metropolis–hastings reversiblizations of non-reversible markov chains. *Stochastic Processes and their Applications*, 130(2):1041–1073, 2020.

[4] Douglas Down, Sean P Meyn, and Richard L Tweedie. Exponential and uniform ergodicity of markov processes. *The Annals of Probability*, 23(4):1671–1691, 1995.

[5] Sarah Elder, Nusrat Zahan, Rui Shu, Monica Metro, Valeri Kozarev, Tim Menzies, and Laurie Williams. Do i really need all this work to find vulnerabilities? an empirical case study comparing vulnerability detection techniques on a java application. *Empirical Software Engineering*, 27(6):154, 2022.

[6] Andrea Fioraldi, Dominik Christian Maier, Heiko Eißfeldt, and Marc Heuse. AFL++ : Combining incremental steps of fuzzing research. In *USENIX Workshop on Offensive Technologies*, 2020.

[7] James M Flegal and Galin L Jones. Batch means and spectral variance estimators in Markov chain Monte Carlo. *The Annals of Statistics*, 38(2):1034–1070, 2010.

[8] Paul Glasserman, Philip Heidelberger, Perwez Shahabuddin, and Tim Zajic. Splitting for rare event simulation: analysis of simple cases. In *Proceedings of the 28th conference on Winter simulation*, pages 302–308, 1996.

[9] Paul Glasserman, Philip Heidelberger, Perwez Shahabuddin, and Tim Zajic. Multilevel splitting for estimating rare event probabilities. *Operations Research*, 47(4):585–600, 1999.

[10] Patrice Godefroid, Adam Kiezun, and Michael Y Levin. Grammar-based whitebox fuzzing. In *Proceedings of the 29th ACM SIGPLAN conference on programming language design and implementation*, pages 206–215, 2008.

[11] Jianmin Guo, Yu Jiang, Yue Zhao, Quan Chen, and Jiaguang Sun. Dlfuzz: Differential fuzzing testing of deep learning systems. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 739–743, 2018.

[12] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. Magma: A ground-truth fuzzing benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 4(3):1–29, 2020.

[13] Samuel Jero, Maria Leonor Pacheco, Dan Goldwasser, and Cristina Nita-Rotaru. Leveraging textual specifications for grammar-based fuzzing of network protocols. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 9478–9483, 2019.

[14] Ling Jiang, Hengchen Yuan, Mingyuan Wu, Lingming Zhang, and Yuqun Zhang. Evaluating and improving hybrid fuzzing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 410–422. IEEE, 2023.

[15] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pages 2123–2138, 2018.

[16] Caroline Lemieux and Koushik Sen. Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*, pages 475–485, 2018.

[17] Yuwei Li, Shouling Ji, Yuan Chen, Sizhuang Liang, Wei-Han Lee, Yueyao Chen, Chenyang Lyu, Chunming Wu, Raheem Beyah, Peng Cheng, et al. UNIFUZZ: A holistic and pragmatic metrics-driven platform for evaluating fuzzers. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2777–2794, 2021.

[18] LLVM Project. Libfuzzer: A library for coverage-guided fuzz testing. https://llvm.org/docs/LibFuzzer.html.

[19] Chenyang Lyu, Shouling Ji, Chao Zhang, Yuwei Li, Wei-Han Lee, Yu Song, and Raheem Beyah. MOPT: Optimized mutation scheduling for fuzzers. In *28th USENIX security symposium (USENIX security 19)*, pages 1949–1966, 2019.

[20] Moein Madadi et al. In bugs we trust? on measuring the randomness of a fuzzer benchmarking outcome. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2026.

[21] Charles C Margossian, Matthew D Hoffman, Pavel Sountsov, Lionel Riou-Durand, Aki Vehtari, and Andrew Gelman. Nested $\hat{R}$: Assessing the convergence of Markov chain Monte Carlo when running many short chains. *Bayesian Analysis*, 1(1):1–28, 2024.

[22] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, volume 2024, 2024.

[23] Jonathan Metzman, László Szekeres, Laurent Simon, Read Sprabery, and Abhishek Arya. Fuzzbench: an open fuzzer benchmarking platform and service. In *ESEC/FSE*, 2021.

[24] Lama J Moukahal, Mohammad Zulkernine, and Martin Soukup. Vulnerability-oriented fuzz testing for connected autonomous vehicle systems. *IEEE transactions on reliability*, 70(4):1422–1437, 2021.

[25] Stefan Nagy and Matthew Hicks. Full-speed fuzzing: Reducing fuzzing overhead through coverage-guided tracing. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 787–802. IEEE, 2019.

[26] Theofilos Petsios, Adrian Tang, Salvatore Stolfo, Angelos D Keromytis, and Suman Jana. Nezha: Efficient domain-independent differential testing. In *2017 IEEE Symposium on security and privacy (SP)*, pages 615–632. IEEE, 2017.

[27] Theofilos Petsios, Jason Zhao, Angelos D Keromytis, and Suman Jana. Slowfuzz: Automated domain-independent detection of algorithmic complexity vulnerabilities. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, pages 2155–2168, 2017.

[28] Van-Thuan Pham, Marcel Böhme, Andrew E. Santosa, Alexandru Razvan Caciulescu, and Abhik Roychoudhury. Smart greybox fuzzing. *CoRR*, 2018.

[29] Meng Ren, Zijing Yin, Fuchen Ma, Zhenyang Xu, Yu Jiang, Chengnian Sun, Huizhong Li, and Yan Cai. Empirical evaluation of smart contract testing: What is the best choice? In *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, pages 566–579, 2021.

[30] Gareth Roberts and Jeffrey Rosenthal. Geometric ergodicity and hybrid Markov chains. *Electronic Communications in Probability*, 2:13–25, 1997.

[31] Gareth O Roberts and Jeffrey S Rosenthal. Variance bounding Markov chains. *The Annals of Applied Probability*, 18(3):1201–1214, 2008.

[32] Moritz Schloegel, Nils Bars, Nico Schiller, Lukas Bernhard, Tobias Scharnowski, Addison Crump, Arash Ale-Ebrahim, Nicolai Bissantz, Marius Muench, and Thorsten Holz. Sok: Prudent evaluation practices for fuzzing. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1974–1993. IEEE, 2024.

[33] Robert Swiecki. honggfuzz: Security oriented, feedback-driven, evolutionary fuzzer. https://github.com/google/honggfuzz.

[34] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Superion: Grammar-aware greybox fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 724–735. IEEE, 2019.

[35] Mingyuan Wu, Ling Jiang, Jiahong Xiang, Yuqun Zhang, Guowei Yang, Huixin Ma, Sen Nie, Shi Wu, Heming Cui, and Lingming Zhang. Evaluating and improving neural program-smoothing-based fuzzing. In *Proceedings of the 44th International Conference on Software Engineering*, pages 847–858, 2022.

[36] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.

[37] Michał Zalewski. American fuzzy lop. http://lcamtuf.coredump.cx/afl/.

Appendix A. Implementation Details

A.1. Offline Splitting Implementation (Section 7.2)

Modeling.. We fix a benchmark–fuzzer pair (b, f). Let $N_m(t)$ denote the cumulative bug count in trial m of a *prior, completed non-splitting campaign* at time $t \in [0, T]$—the historical data available to the offline policy—with across-trial mean curve $N(t) = \frac{1}{M}\sum_{m=1}^{M} N_m(t)$ and overall mean bug rate $r_{\text{tot}} = N(T)/T$. The decision of *when* to initiate Algorithm 2 is guided by $N(\cdot)$ and r_{tot}; the *execution* of a split is triggered on an individual running trial.

Sparsity score.. We adopt splitting in regimes where bugs are relatively sparse. To determine whether bug discovery has entered a "sparse tail" after time t, we use the **tail-average rate estimator** and **relative sparsity ratio**

$$r_{\text{tail}}(t) \;=\; \frac{N(T) - N(t)}{T - t}, \rho(t) \;=\; \frac{r_{\text{tail}}(t)}{r_{\text{tot}} + \varepsilon}, \qquad \varepsilon > 0,$$

where $\rho(t) \ll 1$ indicates the remaining discovery rate is far below the overall average. To overcome temporary fluctuations, we impose a persistence (anti-spike) criterion over a future horizon H:

$$\rho_H(t) \;=\; \max_{u \in [t, \min\{t+H, T\}]} \rho(u).$$

All predicates are evaluated on the experiment's discrete logging grid, and $\rho_H(t)$ only where the future interval is observable (no extrapolation).

Richness-adaptive earliest detection time.. The earliest detection time $t_{b,f}$ at which sparsity detection begins varies across pairs (b, f); we compute it from the richness score

$$\widetilde{B}_{b,f} = \log\big(1 + N(T)\big).$$

For each fixed fuzzer f, with reference quantiles $q_{\text{low}} < q_{\text{high}}$ of $\{\widetilde{B}_{b,f}\}_b$ across benchmarks, we normalize

$$u_{b,f} = \min\Big\{1, \max\Big\{0, \frac{\widetilde{B}_{b,f} - q_{\text{low}}}{q_{\text{high}} - q_{\text{low}}}\Big\}\Big\},$$

and map to $t_{b,f} \in [t_{\min}, t_{\max}]$ by $t_{b,f} = t_{\min} + (t_{\max} - t_{\min}) \cdot u_{b,f}^{p}$, a fuzzer-conditioned, monotone, continuous schedule.

Zone-entry time and ordered stages.. Given a nonincreasing threshold list $\Theta = (\theta_1, \ldots, \theta_K)$ for the K stages at which Algorithm 2 is applied, the first zone-entry time is

$$z_{\theta_1} \;=\; \min\Big\{t \in [t_{b,f}, T) : \rho_H(t) \le \theta_1\Big\},$$

and for $k \ge 2$,

$$z_{\theta_k} \;=\; \min\Big\{t \in [\max\{t_{b,f}, z_{\theta_{k-1}} + \Delta\}, T) : \rho_H(t) \le \theta_k\Big\},$$

where the minimal gap $\Delta > 0$ prevents back-to-back splits in overlapping regions and keeps detections meaningfully separated; if the set is empty, stage k is skipped.

Bug-triggered splitting on a running trial.. If z_{θ_k} exists, the actual split on a running trial i triggers at the first time the trial observes an additional bug,

$$s_{\theta_k, i} \;=\; \min\Big\{t \in (z_{\theta_k}, T] : N_i(t) > N_i(z_{\theta_k})\Big\},$$

skipping the stage if no such time exists. This "zone-then-bug" protocol separates *when splitting becomes statistically justified* (based on $N(\cdot)$) from *when a concrete opportunity to branch arises* (a new bug on a running trial)—key in extremely sparse regimes, where purely time-driven splitting would induce unnecessary branching.

A.2. Online Splitting Implementation (Section 7.4)

Online sparsity score.. To make the ablation independent of any precomputed aggregate signal or prior trials, the sparsity signal uses only *past observations on the running trial* i with bug curve $N_i(t)$: for a window length w on the logging grid,

$$r_{\text{avg},i}(t) = {}^{N_i(t)}\!/_{t+\varepsilon_t}, \qquad r_{\text{win},i}(t) = {}^{N_i(t) - N_i(t-w)}\!/_{w},$$
$$\rho_{\text{on},i}(t) = {}^{r_{\text{win},i}(t)}\!/_{r_{\text{avg},i}(t)+\varepsilon},$$

with regularizers $\varepsilon_t, \varepsilon > 0$; $\rho_{\text{on},i}(t) \ll 1$ indicates the recent discovery rate is small relative to the trial's running average. Persistence uses a *past* horizon:

$$\rho^{-}_{H,i}(t) = \max_{u \in [t-H, t]} \rho_{\text{on},i}(u),$$

Zone entry, ordered stages, & splitting.. Given stage thresholds $\Theta = (\theta_1, \ldots, \theta_K)$, the online variant defines zone-entry times *per running trial* using only running observations. Let S denote the earliest time at which the online statistic becomes well-defined. Concretely, $S = \max\{w, H\}$ is the warm-up time required by the windowed difference w and the persistence horizon H. Then, for each running trial i,

$$z_{\theta_1, i} = \min\{t \ge S : \rho^{-}_{H,i}(t) \le \theta_1\},$$
$$z_{\theta_k, i} = \min\{t \ge z_{\theta_{k-1}, i} + \Delta : \rho^{-}_{H,i}(t) \le \theta_k\} \quad (k \ge 2),$$

with the same minimum gap Δ and stage-skipping rule (when the set is empty). The execution rule remains the same: for a running trial i, the split at stage k is triggered at the first logged time $s_{\theta_k, i}$ after $z_{\theta_k, i}$ when a new bug is observed:

$$s_{\theta_k, i} = \min\{t > z_{\theta_k, i} : N_i(t) > N_i(z_{\theta_k, i})\}.$$

The ablation preserves the splitting mechanism, modifies only the sparsity score, and is broadly deployable.

Appendix B. Spectral Gap Estimation

B.1. Assumptions

Our framework relies on assumptions about the fuzzer's transition process. We discuss their justification and scope here.

Algorithm 3: Spectral Gap Estimation from Trajectory Data

Input: Input trajectory $\{X_R^m(w,t)\}_{t=1}^T$ from M trials;
Step 1: Count transitions between states across all M trials to form the empirical transition matrix $\hat{P}_R$;
Step 2: Compute eigenvalues of $\hat{P}_R$; the second eigenvalue is $\hat{\lambda}_2$;
Step 3: $\hat{\gamma}_{M,R} = 1 - |\hat{\lambda}_2|$;
Output: Estimated spectral gap $\hat{\gamma}_{M,R}$ and relaxation time $\hat{\delta}_R = 1/\hat{\gamma}_{M,R}$

Time-homogeneity (stationarity). We assume the transition matrix P_R^X is fixed across time steps. This holds for mutation-based fuzzers whose mutation rules are fixed in code: given the current input x, the distribution over the next input $R(x)$ depends only on x and the fuzzer's fixed mutation operator, not on the time step t. Coverage-guided fuzzers (e.g., AFL++, LibFuzzer) maintain internal state such as a corpus and scheduling weights, but these are deterministic functions of the sequence of inputs executed so far, i.e., deterministic functions of the Markov chain trajectory $\{X_R(w,s)\}_{s\le t}$, and can be folded into an augmented state. For fuzzers or regimes that genuinely violate stationarity (e.g., time-dependent temperature schedules, LLM-based mutation, or the slow drift of the output surrogate that Section 7.6 measures as easy bugs deplete), we handle the non-stationary case via windowed estimation with random restarting: partition the campaign into approximately stationary windows, estimate within windows, and aggregate. Section 7.6 quantifies the practical extent of this idealization on the output surrogate.

Geometric ergodicity. We assume the output chain Y_R is geometrically ergodic, meaning it converges to its stationary distribution at a geometric rate. This assumption excludes only degenerate "trapped" fuzzers that remain stuck in a single input region indefinitely. In practice, random restarting (a standard feature of most fuzzers) ensures ergodicity. Importantly, ergodicity is *not* required for the Markov chain modeling itself; it is only needed for the theoretical convergence guarantees in Section 5. Our empirical results (Section 7) show consistent behavior across all seven evaluated fuzzers, including those with complex scheduling (AFL++, MOpt).

B.2. Additional Metrics and Results

Additionally, we evaluate (i) how well the computable variance estimates approximate the underlying asymptotic variance, and (ii) the discrepancy between the population variance and the same target:

$$A_R(T) = \left| T \cdot \tilde{S}_R(T) - \sigma^2 \right|, \qquad B_R(T) = \left| T \cdot V_R(T) - \sigma^2 \right|$$

where $\sigma^2 = Var\Big(\lim_{T\to\infty} \frac{1}{\sqrt{T}} \sum_{t=1}^{T} Y_R(w,t)\Big)$ and $T \cdot V_R(T) = T \cdot \mathbb{E}\big[(\tilde{P}_R(T) - \mathbb{E}[\tilde{P}_R(T)])^2\big]$ is the effort-normalized population variance, which characterizes the intrinsic stability of $\tilde{P}_R(T)$ and supports confidence intervals.

We also characterize the mean squared error against the limiting rate $P_R = \lim_{T\to\infty} \tilde{P}_R(T)$ via $M_R(T) = T \cdot \mathbb{E}\big[(\tilde{P}_R(T) - \lim_{T\to\infty} \tilde{P}_R(T))^2\big]$, which decomposes as $M_R(T) = V_R(T) + T \cdot (\mathbb{E}[D_R(T)])^2$ with $D_R(T) = \mathbb{E}[|\tilde{P}_R(T) - \pi(1)|]$—accounting for both variance and bias. The formal evaluation statement follows.

Theorem 4 ($A_R(T)$ and $B_R(T)$)**.** *Suppose the output chains of all fuzzers $R \in \mathcal{R}$ are geometrically ergodic.*

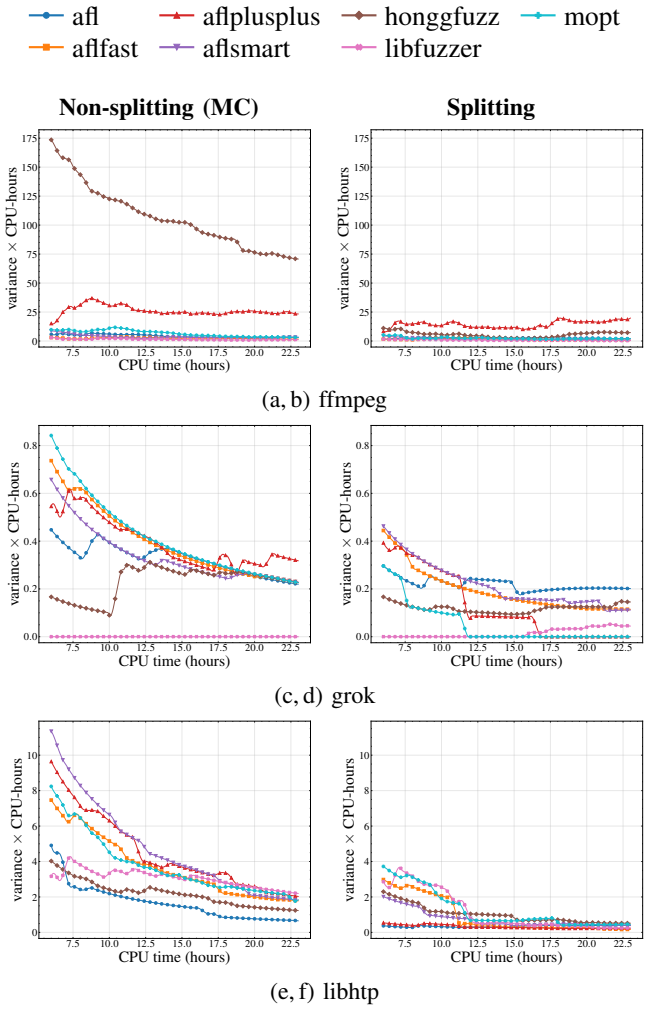
1) *bound on $A_R(T)$. Define $U_{A,R}(T) := \frac{1}{M} + \left(\frac{1}{4} + \frac{1}{16\pi(1)^2}\right) \cdot \frac{(1-\gamma_{M,R})\left(1+(1-\gamma_{M,R})\right)}{\left(1-(1-\gamma_{M,R})\right)^2} + \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{\gamma_{M,R}^2} + 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}$. Then, with probability at least $1 - 2\exp\{-cT^2/M\}$ for an absolute constant $c > 0$, we have $A_R(T) \le U_{A,R}(T)$.*
2) *bound on $B_R(T)$. Define $U_{B,R}(T) := \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{\gamma_{M,R}^2} + 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}$. Then $B_R(T) \le U_{B,R}(T)$.*

By a union bound, with probability at least $1 - 2|\mathcal{R}|\exp\{-cT^2/M\}$ the guarantee in part 1 holds simultaneously for all $R \in \mathcal{R}$. Consequently, the computable effort-normalized sample variance $T \cdot \widetilde{S}_R(T)$ approximates each fuzzer's true robustness σ^2 to within $U_{A,R}(T)$, so ranking fuzzers by $T \cdot \widetilde{S}_R(T)$ ranks them by σ^2 up to these explicit accuracy margins. Fuzzers with larger spectral gap $\gamma_{M,R}$ admit smaller $U_{A,R}(T)$: their robustness can be certified reliably from fewer and shorter trials.

We remark that (i) the second term of $U_{A,R}(T)$ can be sharpened by a factor $1/T$ (it arises as C_2/T in the proof); we state and compute the looser form throughout, which only weakens the bound. (ii) The exponent's M-dependence reflects the deviation radius $1/M$ demanded of the sample variance as M grows.

Appendix C.
Additional Numerical Results

All experiments are conducted on a server running Ubuntu 24.04.2 LTS (Noble) with 2.0 TB of RAM and Intel Xeon Platinum 8480+ processors (56 cores / 112 threads per socket; 2.00 GHz base frequency). We present the full Magma ground-truth table below; the complete per-fuzzer figure grids for Section 7 are provided in the artifact's full version.



(a, b) ffmpeg

(c, d) grok

(e, f) libhtp

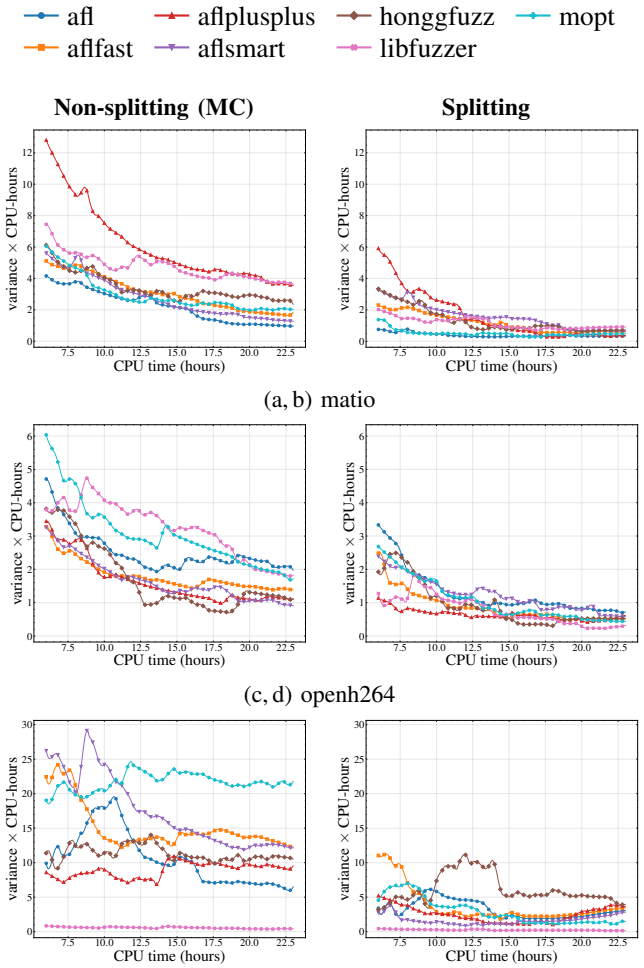
Figure 7: Variance (× CPU-hours) across the seven fuzzers under Monte Carlo (non-splitting, left column) and splitting (right column), for three of the remaining benchmarks (part 1 of 3). Companion to Figure 2.

C.1. Comparison with Monte Carlo Simulation

Next, we compare our algorithm with the vanilla Monte Carlo (MC) simulation from a variance perspective, evaluating the performance of both methods and highlighting the advantages of the proposed splitting approach.

TABLE 7: Magma ground-truth validation (full): five fuzzers × eight Magma programs, aggregated over $M=5$ trials of 12 h, splitting vs. no-split (MC). Each cell is *splitting*/*no-split*: mean unique bugs per trial; total (union) unique bugs over the 5 trials (the Theorem 3 quantity); weighted bug-detection rate $\widehat{P}_R$ (bugs/h); effort-normalized $\mathrm{Var}(\widehat{P}_R)$ (lower is better). **Bold** marks the better value; ties unbolded. Note the saturation in the variance column: the no-split variance is exactly 0 in 17 of 40 cells (identical outcomes across trials), the regime where the variance metric is uninformative (Section 7.5).

Fuzzer	mean uniq (s / n)	total uniq (s / n)	$\widehat{P}_R$ (s / n)	$\mathrm{Var}(\widehat{P}_R)$ (s / n)
libpng				
AFL	**1.2**/1	**2**/1	**4.025**/1	7.23e-08/**0**
AFLFast	1/1	1/1	**4**/1	0/0
AFL++	**3**/2.2	3/3	**5.167**/2	**5.87e-06**/1.16e-05
MOpt	**1.8**/1.2	**3**/2	**4.25**/1.2	**1.63e-06**/4.63e-06
Honggfuzz	**4**/3.6	4/4	**6.55**/3.6	7.52e-06/**6.94e-06**
libtiff				
AFL	1/1	2/2	**1.27**/1	2.72e-05/**0**
AFLFast	**2.25**/2	**5**/3	1.992/**2.000**	**6.41e-06**/2.31e-05
AFL++	**6.4**/5.4	7/7	**5.755**/4.4	3.76e-05/**6.94e-06**
MOpt	**4.8**/3.4	**5**/4	**4.683**/3.4	**3.89e-06**/6.94e-06
Honggfuzz	**4.6**/4	**6**/4	**5.968**/4	1.17e-05/**0**
libsndfile				
AFL	2/2	2/2	**4.16**/2	1.32e-06/**0**
AFLFast	2/2	2/2	**4.06**/2	1.71e-07/**0**
AFL++	7/7	7/7	**8.009**/5.2	**9.28e-06**/1.62e-05
MOpt	7/7	7/7	**7.533**/5	8.69e-05/**9.92e-36**
Honggfuzz	**6.4**/6	**7**/6	**8.969**/6	2.90e-06/**0**
poppler				
AFL	1.8/1.8	**3**/2	**4.05**/1.8	**2.89e-07**/4.63e-06
AFLFast	2/2	2/2	**4.307**/1	1.08e-06/**0**
AFL++	**3.4**/2.2	**5**/3	**5.342**/2.2	**2.78e-06**/4.63e-06
MOpt	**4**/3	**6**/3	**6.175**/2.8	**1.74e-06**/4.63e-06
Honggfuzz	**3**/2.2	3/3	**5.289**/2.2	5.10e-06/**4.63e-06**
sqlite3				
AFL	**0.8**/0.4	**3**/2	**1.158**/0.4	9.98e-05/**1.85e-05**
AFLFast	1.6/1.6	**3**/2	**3.36**/1.6	8.78e-05/**6.94e-06**
AFL++	**3**/1.8	**3**/2	**5.166**/1.8	**1.64e-06**/4.63e-06
MOpt	**2.8**/2.4	**4**/3	**5.285**/2.2	**6.88e-06**/1.62e-05
Honggfuzz	**3.4**/2.8	**5**/3	**5.828**/2.8	**1.33e-06**/4.63e-06
openssl				
AFL	**2**/1.2	2/2	**4.8**/1.2	**6.51e-07**/4.63e-06
AFLFast	**2**/1.6	2/2	**4.85**/1.6	**4.34e-07**/6.94e-06
AFL++	1/1	1/1	**3.825**/1	4.07e-07/**0**
MOpt	**2**/1.8	2/2	**4.592**/1.8	**4.30e-07**/4.63e-06
Honggfuzz	**1.2**/1	**2**/1	**3.749**/1	5.10e-07/**0**
libxml2				
AFL	**1.2**/1	**2**/1	**4**/1	0/0
AFLFast	**1.4**/1	**2**/1	**4.029**/1	6.86e-08/**0**
AFL++	4/4	4/4	**6.775**/4	6.15e-07/**0**
MOpt	**3.6**/3	4/4	**5.35**/3	**9.77e-07**/2.31e-05
Honggfuzz	**2.2**/1.4	**3**/2	**4.25**/1.4	**5.43e-07**/6.94e-06
php				
AFL	3/3	3/3	**5.996**/3	5.79e-06/**0**
AFLFast	3/3	3/3	**6.45**/3	1.13e-05/**0**
AFL++	**3**/2	3/3	**4.864**/2	1.46e-06/**0**
MOpt	3/3	3/3	**6.775**/2.6	1.74e-05/**6.94e-06**
Honggfuzz	1/1	1/1	**4**/1	0/0



(a, b) matio

(c, d) openh264

(e, f) poppler

Figure 8: Variance (× CPU-hours) across the seven fuzzers under Monte Carlo (non-splitting, left column) and splitting (right column), for three of the remaining benchmarks (part 2 of 3). Companion to Figure 2.

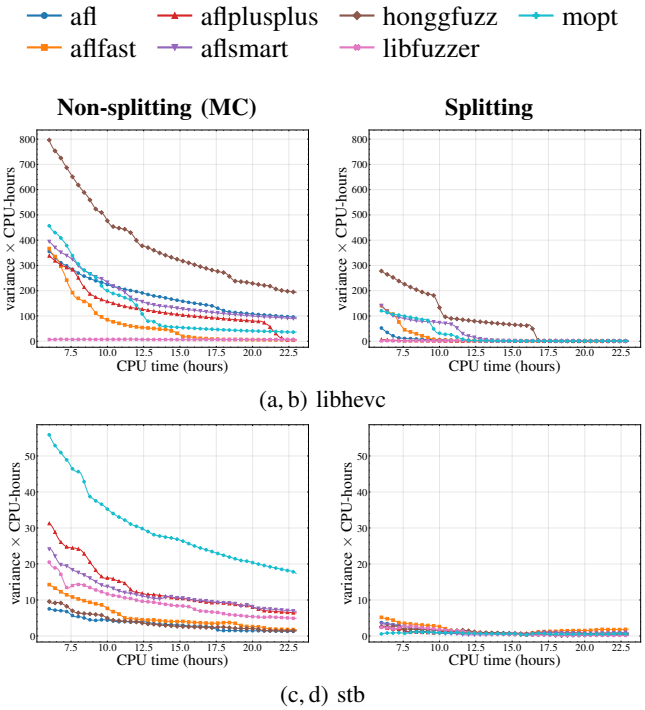
Specifically, we compare the variance of the estimator produced by the splitting algorithm

$$\hat{P}_R^m(T) = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{k_t^m}\sum_{s=1}^{k_t^m} Y_{s,t},$$
$$\hat{P}_R(T) = \frac{1}{M}\sum_{m=1}^{M}\hat{P}_R^m(T),$$

which we refer to as the *splitting estimator*, to that of the estimator produced by the vanilla MC method,

$$\tilde{P}_R(T) = \frac{1}{T}\cdot\frac{1}{M}\sum_{t=1}^{T}\sum_{m=1}^{M} Y_{m,t},$$

which we refer to as the *MC estimator*.



(a, b) libhevc

(c, d) stb

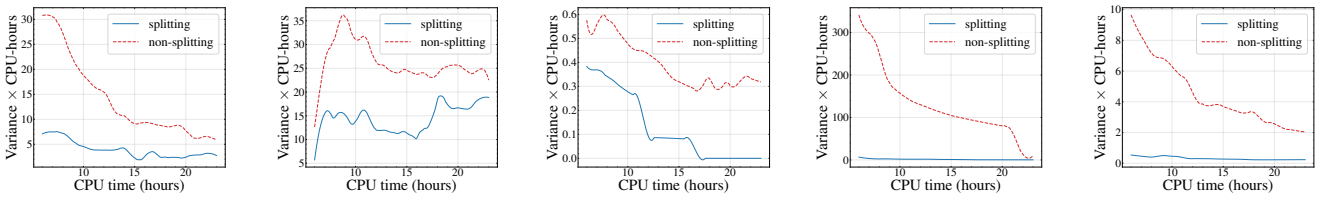
Figure 9: Variance (× CPU-hours) across the seven fuzzers under Monte Carlo (non-splitting, left column) and splitting (right column), for the last two remaining benchmarks (part 3 of 3). Companion to Figure 2.

For the MC estimator, we first consider the case where $M = 1$. Based on the aforementioned derivations, we obtain that

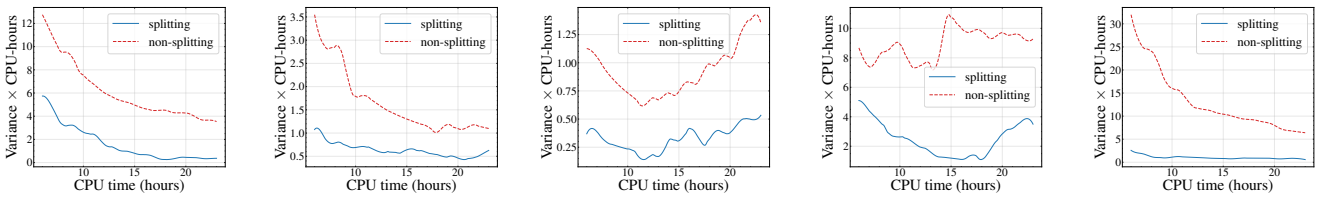
$$\begin{aligned}
&Var(\sqrt{T}\tilde{P}_R(T))\\
&= \sigma^2 - 2\sum_{t=1}^{\infty}\gamma_t + 2\sum_{t=1}^{(T-1)}(1-\frac{t}{T})\gamma_t\\
&= \sigma^2 - 2\sum_{t=1}^{T-1}\gamma_t - 2\sum_{t=T}^{\infty}\gamma_t + 2\sum_{t=1}^{(T-1)}(1-\frac{t}{T})\gamma_t\\
&= \sigma^2 - 2\sum_{t=1}^{(T-1)}(\frac{t}{T})\gamma_t - 2\sum_{t=T}^{\infty}\gamma_t.
\end{aligned}$$

Moreover, for a geometrically ergodic Markov chain, the lag-t autocovariance obeys

$$|\gamma_t| \le \frac{\pi(1)(1-\gamma_{M,R})^t}{2\max\{\pi(0),\pi(1)\}}$$

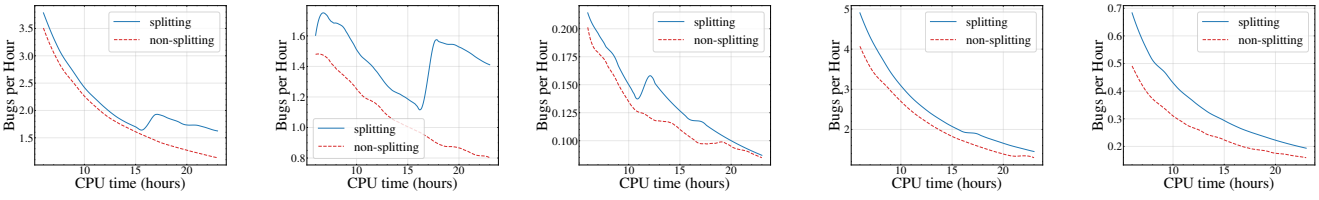


(a) variance of arrow (b) variance of ffmpeg (c) variance of grok (d) variance of libhevc (e) variance of libhtp

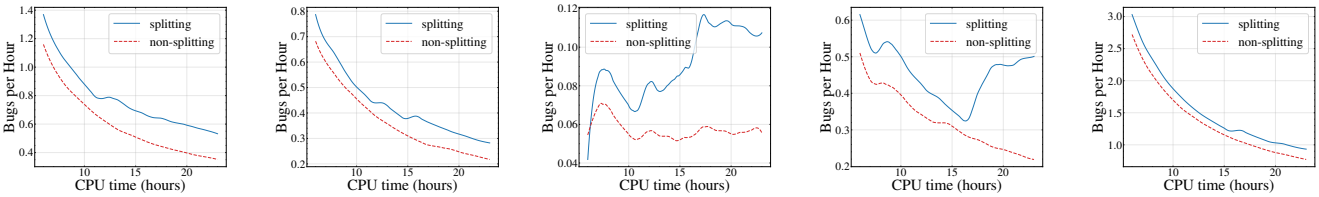


(f) variance of matio (g) variance of openh264 (h) variance of php (i) variance of poppler (j) variance of stb

Figure 10: Comparisons of variance across all 10 benchmarks for fuzzer aflplusplus (full version of Figure 3).

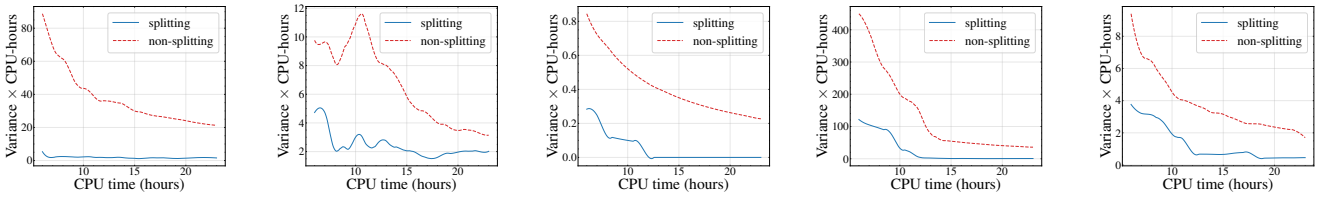


(a) BDR of arrow (b) BDR of ffmpeg (c) BDR of grok (d) BDR of libhevc (e) BDR of libhtp

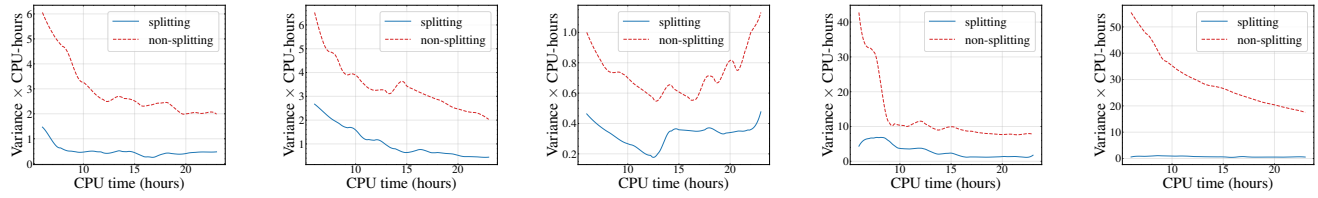


(f) BDR of matio (g) BDR of openh264 (h) BDR of php (i) BDR of poppler (j) BDR of stb

Figure 11: Comparisons of bug detection rate across all 10 benchmarks for fuzzer aflplusplus (full version of Figure 4).

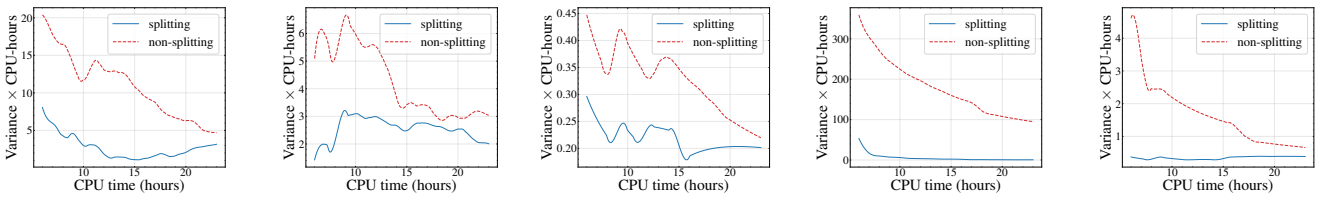


(a) variance of arrow (b) variance of ffmpeg (c) variance of grok (d) variance of libhevc (e) variance of libhtp

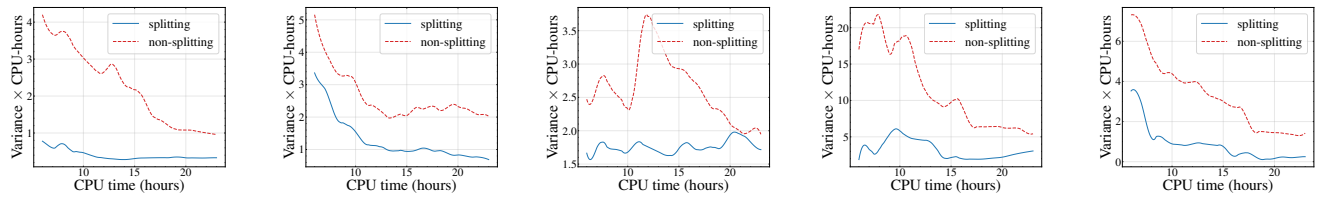


(f) variance of matio (g) variance of openh264 (h) variance of php (i) variance of poppler (j) variance of stb

Figure 12: Comparisons of variance across 10 benchmarks for fuzzer MOpt.

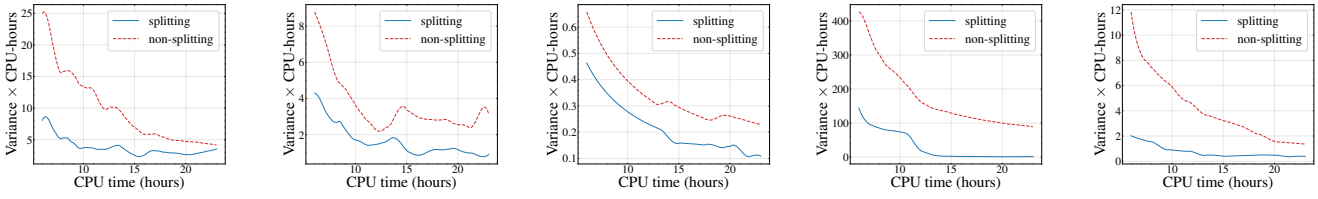


(a) variance of arrow (b) variance of ffmpeg (c) variance of grok (d) variance of libhevc (e) variance of libhtp

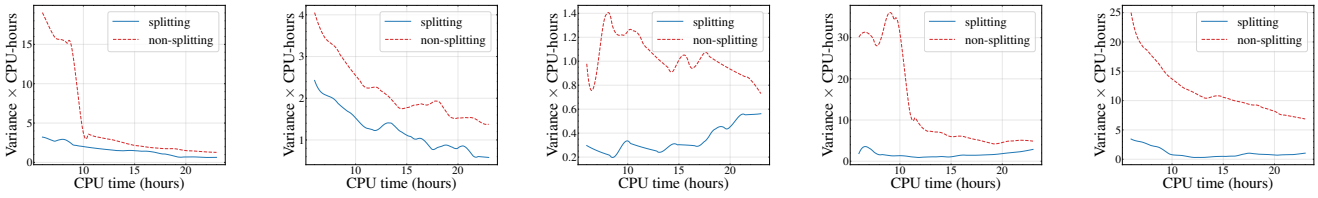


(f) variance of matio (g) variance of openh264 (h) variance of php (i) variance of poppler (j) variance of stb

Figure 13: Comparisons of variance across 10 benchmarks for fuzzer AFL.

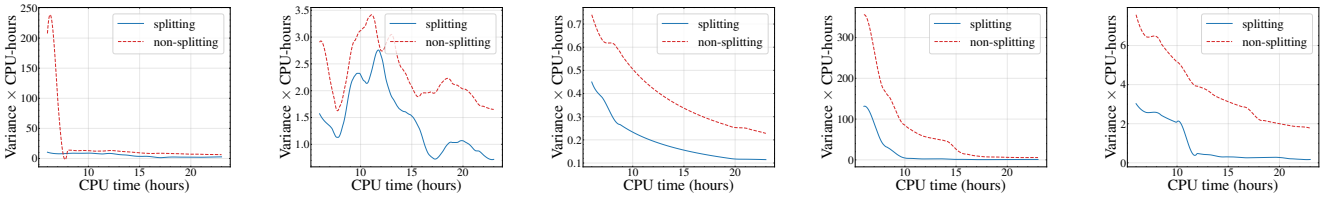


(a) variance of arrow (b) variance of ffmpeg (c) variance of grok (d) variance of libhevc (e) variance of libhtp

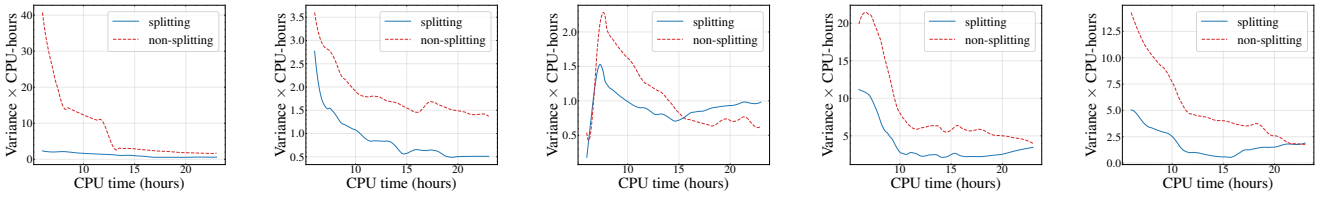


(f) variance of matio (g) variance of openh264 (h) variance of php (i) variance of poppler (j) variance of stb

Figure 14: Comparisons of variance across 10 benchmarks for fuzzer AFLSmart.

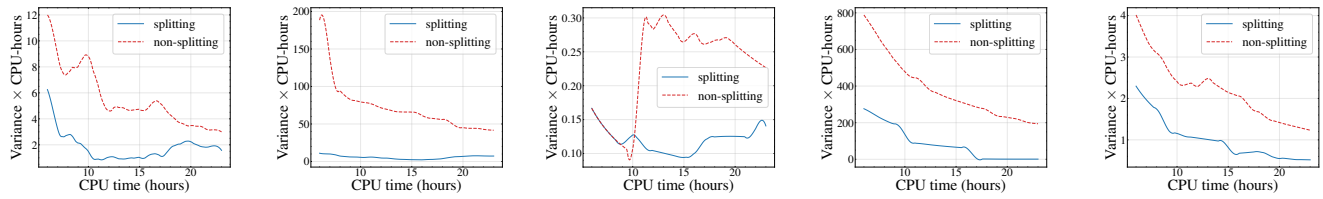


(a) variance of arrow (b) variance of ffmpeg (c) variance of grok (d) variance of libhevc (e) variance of libhtp

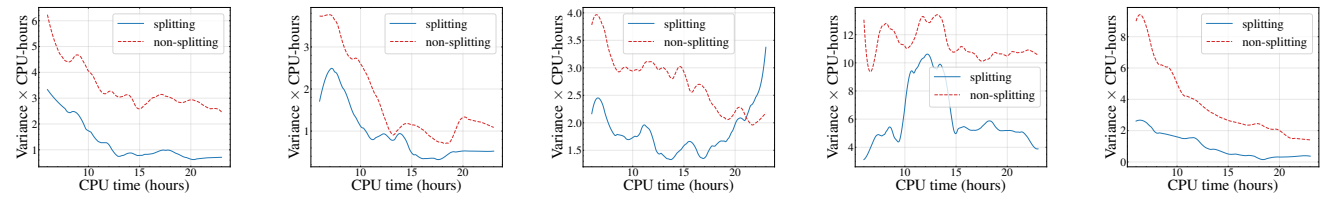


(f) variance of matio (g) variance of openh264 (h) variance of php (i) variance of poppler (j) variance of stb

Figure 15: Comparisons of variance across 10 benchmarks for fuzzer AFLFast.

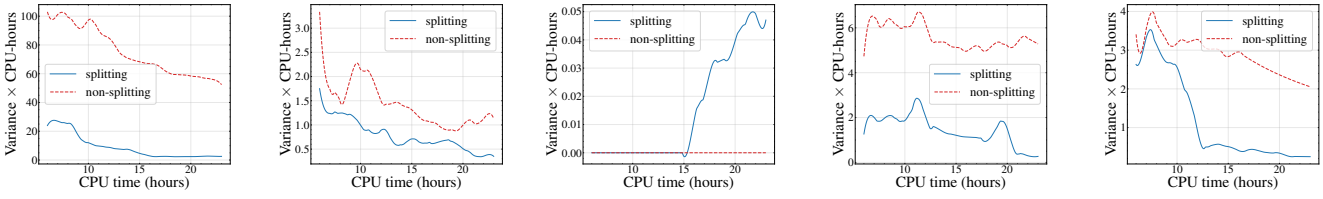


(a) variance of arrow (b) variance of ffmpeg (c) variance of grok (d) variance of libhevc (e) variance of libhtp

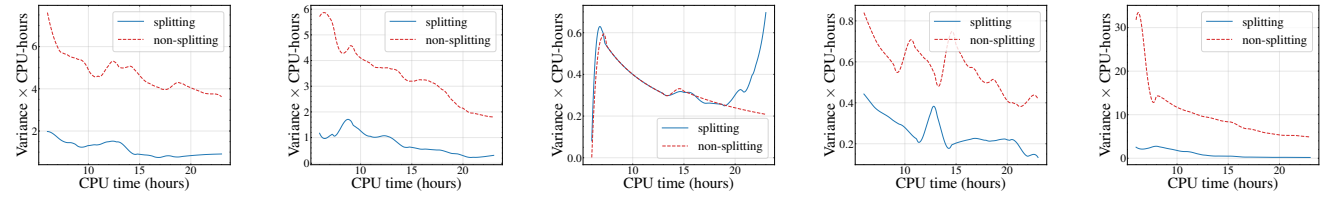


(f) variance of matio (g) variance of openh264 (h) variance of php (i) variance of poppler (j) variance of stb

Figure 16: Comparisons of variance across 10 benchmarks for fuzzer Honggfuzz.

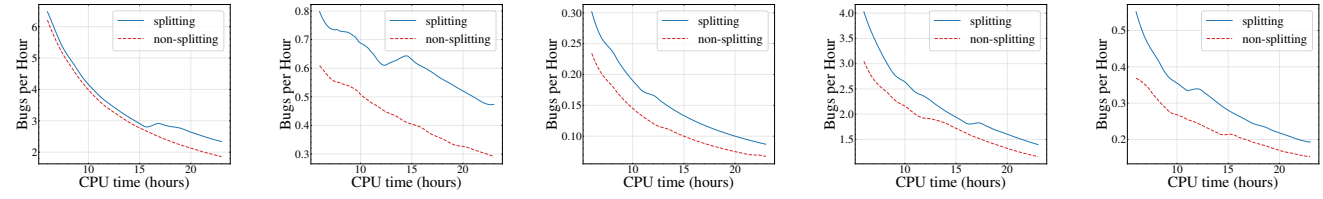


(a) variance of arrow (b) variance of ffmpeg (c) variance of grok (d) variance of libhevc (e) variance of libhtp

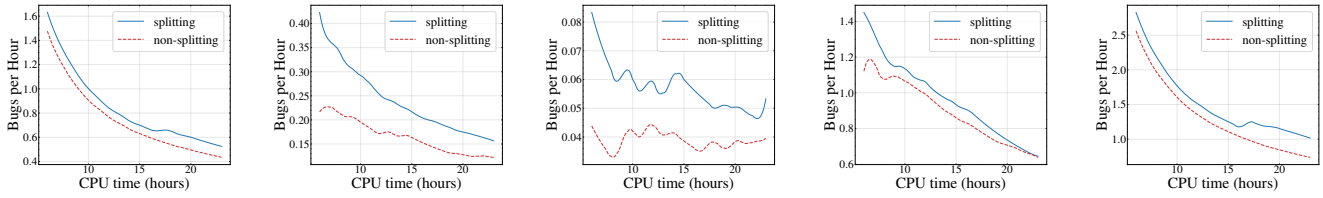


(f) variance of matio (g) variance of openh264 (h) variance of php (i) variance of poppler (j) variance of stb

Figure 17: Comparisons of variance across 10 benchmarks for fuzzer LibFuzzer.

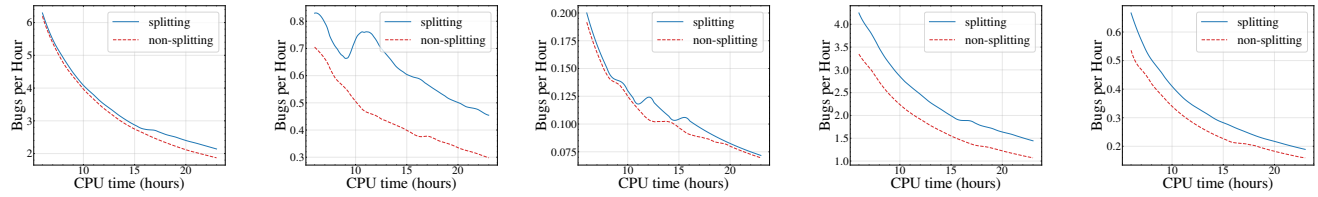


(a) BDR of arrow (b) BDR of ffmpeg (c) BDR of grok (d) BDR of libhevc (e) BDR of libhtp

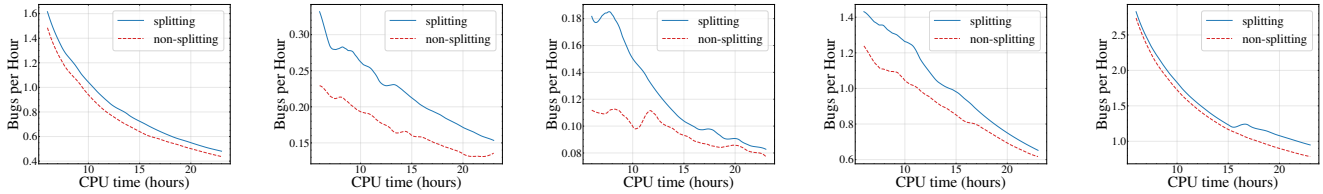


(f) BDR of matio (g) BDR of openh264 (h) BDR of php (i) BDR of poppler (j) BDR of stb

Figure 18: Comparisons of bug detection rate across 10 benchmarks for fuzzer MOpt.

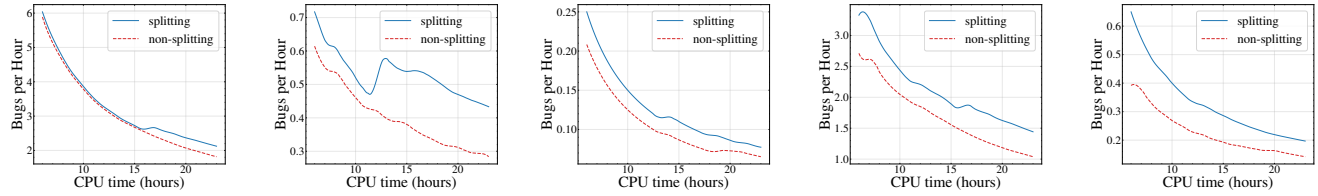


(a) BDR of arrow (b) BDR of ffmpeg (c) BDR of grok (d) BDR of libhevc (e) BDR of libhtp

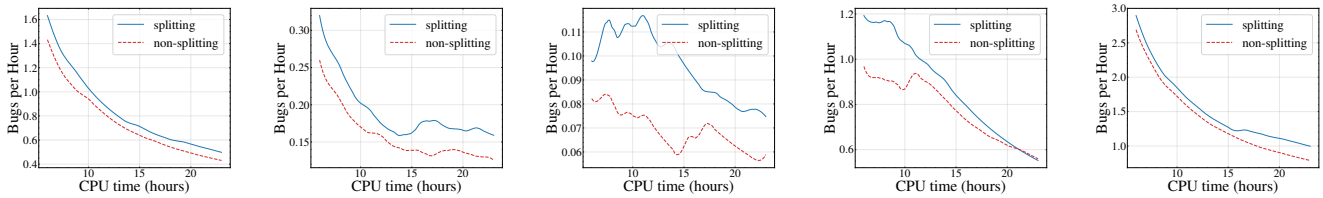


(f) BDR of matio (g) BDR of openh264 (h) BDR of php (i) BDR of poppler (j) BDR of stb

Figure 19: Comparisons of bug detection rate across 10 benchmarks for fuzzer AFL.

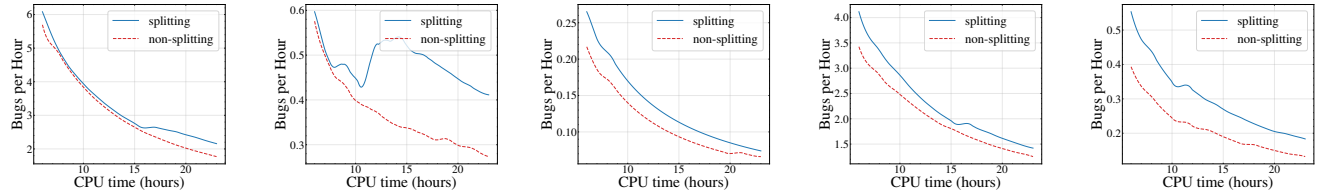


(a) BDR of arrow (b) BDR of ffmpeg (c) BDR of grok (d) BDR of libhevc (e) BDR of libhtp

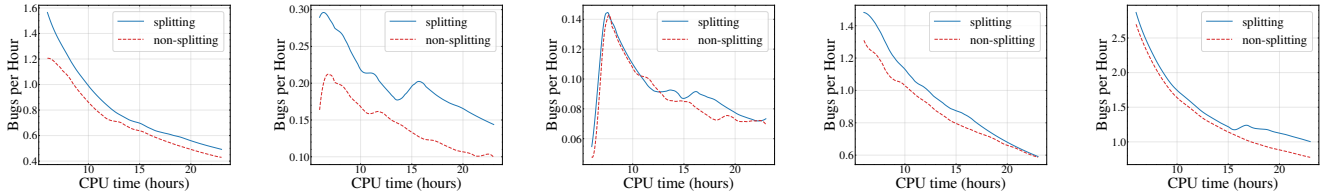


(f) BDR of matio (g) BDR of openh264 (h) BDR of php (i) BDR of poppler (j) BDR of stb

Figure 20: Comparisons of bug detection rate across 10 benchmarks for fuzzer AFLSmart.

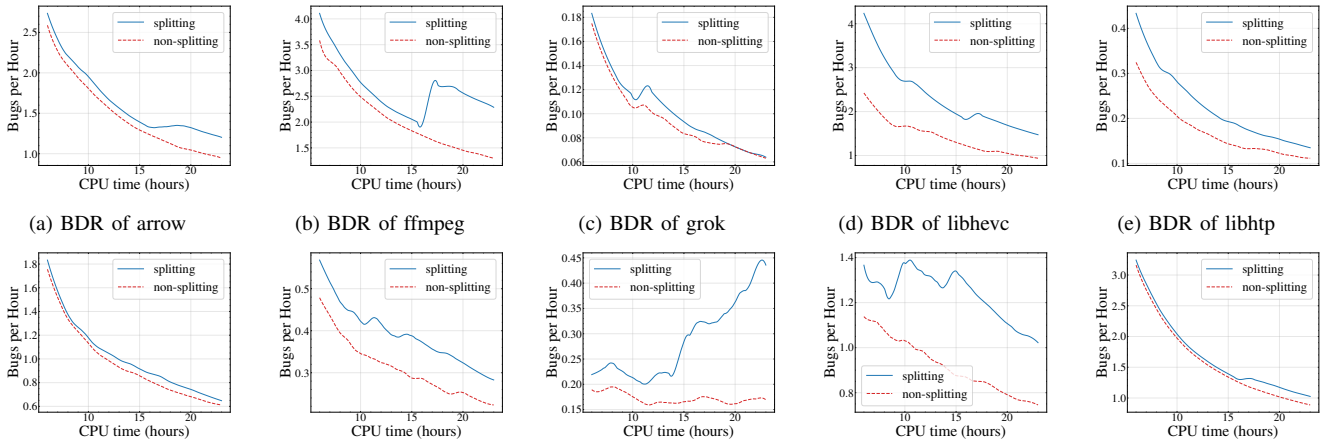


(a) BDR of arrow (b) BDR of ffmpeg (c) BDR of grok (d) BDR of libhevc (e) BDR of libhtp



(f) BDR of matio (g) BDR of openh264 (h) BDR of php (i) BDR of poppler (j) BDR of stb

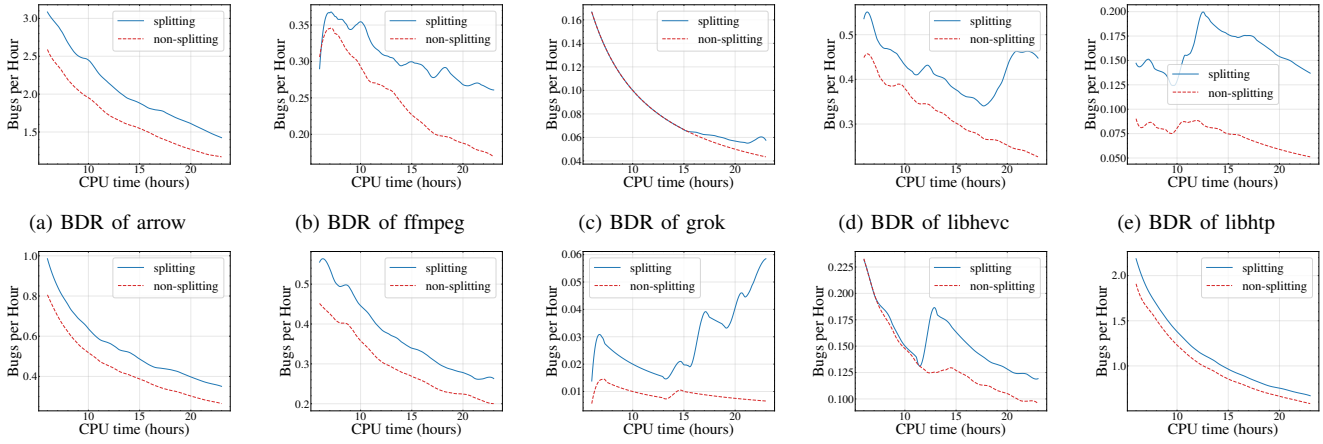
Figure 21: Comparisons of bug detection rate across 10 benchmarks for fuzzer AFLFast.



(a) BDR of arrow (b) BDR of ffmpeg (c) BDR of grok (d) BDR of libhevc (e) BDR of libhtp

(f) BDR of matio (g) BDR of openh264 (h) BDR of php (i) BDR of poppler (j) BDR of stb

Figure 22: Comparisons of bug detection rate across 10 benchmarks for fuzzer Honggfuzz.



(a) BDR of arrow (b) BDR of ffmpeg (c) BDR of grok (d) BDR of libhevc (e) BDR of libhtp

(f) BDR of matio (g) BDR of openh264 (h) BDR of php (i) BDR of poppler (j) BDR of stb

Figure 23: Comparisons of bug detection rate across 10 benchmarks for fuzzer LibFuzzer.

which immediately implies that

$$\begin{aligned}
&\frac{2}{T}\sum_{t=1}^{\infty} t\cdot|\gamma_t| \\
&\le \frac{2}{T}\sum_{t=1}^{\infty} t\cdot\frac{\pi(1)(1-\gamma_{M,R})^t}{2\max\{\pi(0),\pi(1)\}} \\
&= \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2}
\end{aligned}$$

and

$$\begin{aligned}
&\sum_{t=T}^{\infty}|\gamma_t| \\
&\le \sum_{t=T}^{\infty}\frac{\pi(1)(1-\gamma_{M,R})^t}{2\max\{\pi(0),\pi(1)\}} \\
&= \frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}
\end{aligned}$$

Subsequently, we arrive at

$$\begin{aligned}
&\left|Var(\sqrt{T}\tilde{P}_R(T))-\sigma^2\right| \\
&\le \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2}+ \\
&\quad 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}
\end{aligned}$$

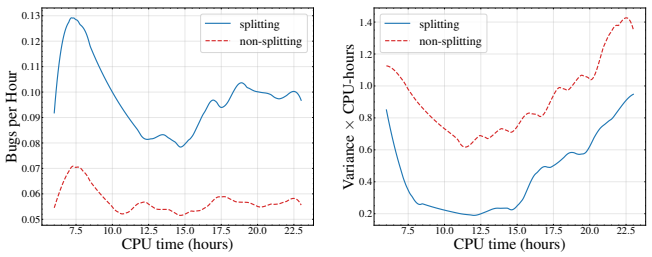
and let us denote

$$\sigma^2_{MC} = Var(\tilde{P}_R(T)).$$

For the splitting algorithm with a single split, the estimator variance satisfies

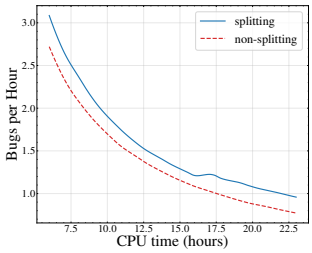
$$\mathrm{Var}\big(\hat{P}_R(T)\big) \;=\; \mathrm{Var}\left(\frac{1}{T}\sum_{t=1}^{T}\frac{1}{k_t^m}\sum_{s=1}^{k_t^m}Y_{s,t}\right).$$

As before, we first focus on the case $M = 1$, i.e., a single initial trial. Suppose the run splits at time τ and then

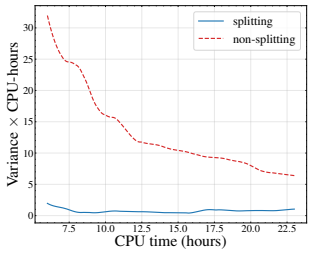


(a) BDR of php

(b) variance of php

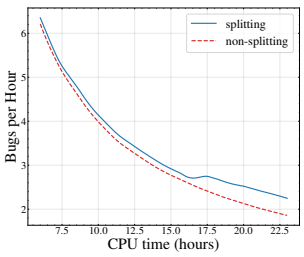


(c) BDR of stb

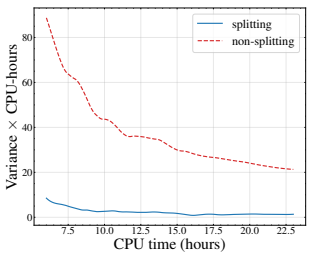


(d) variance of stb

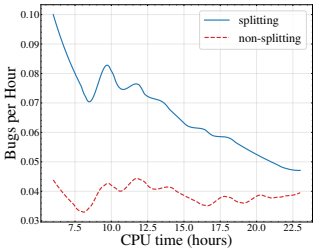
Figure 24: Ablation study for fuzzer aflplusplus on php and stb (companion to Figure 5).



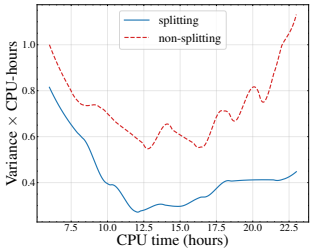
(a) BDR of arrow



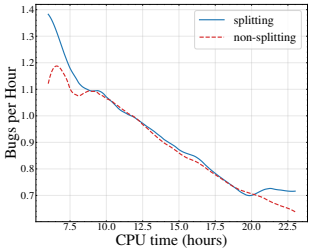
(b) variance of arrow



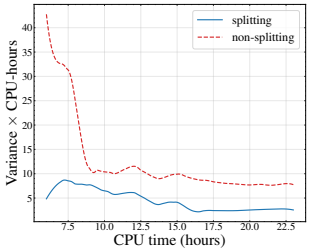
(c) BDR of php



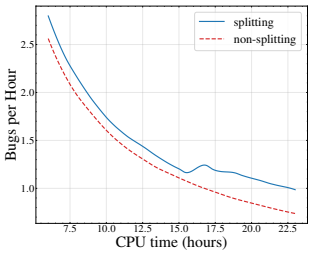
(d) variance of php



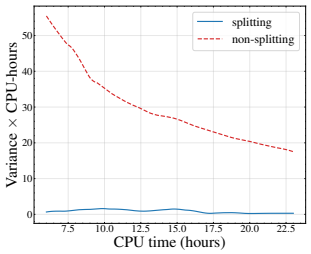
(e) BDR of poppler



(f) variance of poppler



(g) BDR of stb



(h) variance of stb

Figure 25: Ablation study for fuzzer MOpt on arrow, php, poppler and stb.

evolves with two active paths thereafter; specifically, $k_t = 1$ for $t \le \tau$ and $k_t = 2$ for $t > \tau$. This corresponds to a single binary split (as opposed to multiple splits). In the multi-replication setting, the split time τ need not be synchronized across replications.

C.1.1. Assumptions. To characterize the variance of the *splitting estimator*, we adopt the following assumptions. We model the fuzzing process as a (possibly non-stationary) Markov chain on a state space with a designated bug (event) set B (denoted a in the proof). In particular, after a split, we treat the descendant paths as independent paths initiated from the bug-triggering state, while allowing dependence within each descendant path.

A1 (Fuzzer dynamics: AR (1) model).. The state evolution within each post-split replication is modeled by an autoregressive (order-one) update of the form

$$U_{k,t} \;=\; V(U_{k,t-1}) \;+\; W_{k,t}, \qquad k \in \{1,2\},$$

where $V(\cdot)$ represents the (possibly randomized) masking/selection operation applied to the previous state, and $W_{k,t}$ represents the injected mutation/input at time t. Specifically,

$$X_{1,t} = \phi(X_{1,t-1}, W_{1,t}, Z_{1,t}) = \phi(X_{1,t-1}) \cdot W_{1,t} + Z_{1,t} \cdot W_{1,t},$$

with $\phi = (\phi_1, \ldots, \phi_l)$ denoting masking operators over locations and $W_{k,t}$ denoting location selections.

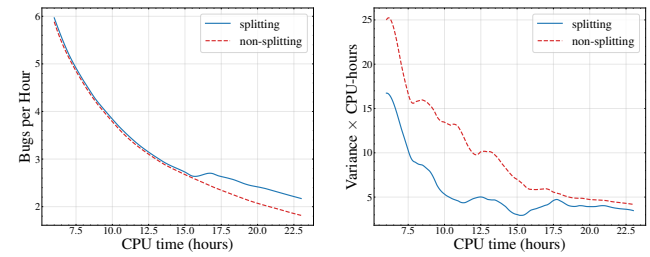
A2 (Domain-set structures).. We treat the bug set B as a discrete set of inputs/states. In addition, we assume that any bit in the input string can potentially trigger a bug, and that each bug is associated with a specific bit/value pattern.

A3 (Correlation control via a multiplicative bound).. We assume that

$$\mathbb{P}(X_{2,t} \in B,\; X_{1,t} \in B) \;\le\; (1-\rho)\,\mathbb{P}(X_{1,t} \in B)\,\mathbb{P}(X_{2,t} \in B).$$

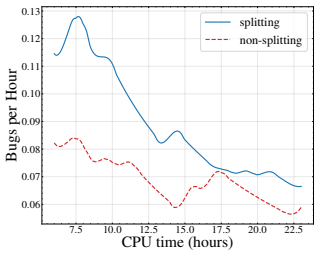
which can be meet by the construction of the AR (1) model by the following lemma.

Lemma 5 (Sufficient condition for Assumption A3 under a coupled-noise AR(1) construction)**.** *Consider the post-split inputs $\{X_{k,t}\}_{t \ge \tau+1}$, $k \in \{1,2\}$ evolving under the AR(1)-type update in Assumption A1, where the mutation at time t selects a location J_t with distribution $\mathbb{P}(J_t = j) = P(j)$ and then injects a value in the domain D_j according to a pmf $f_j(\cdot)$ (allowing non-uniform mutation). Let the bug set B be discrete (Assumption A2), and suppose that membership in B admits the following bitwise description: for each location j there is a (finite) set $V_j \subseteq D_j$ such that "being buggy due to location j" corresponds to $X_t[j] \in V_j$.*

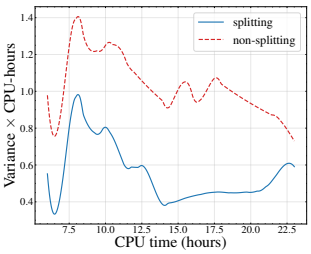


(a) BDR of arrow

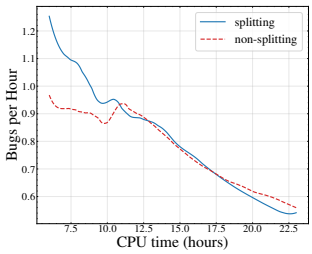
(b) variance of arrow



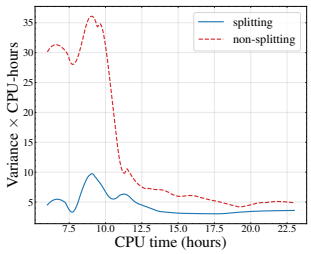
(c) BDR of php



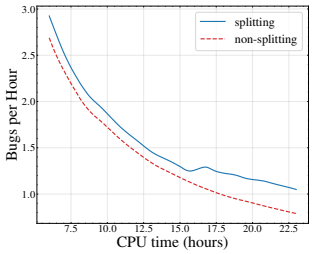
(d) variance of php



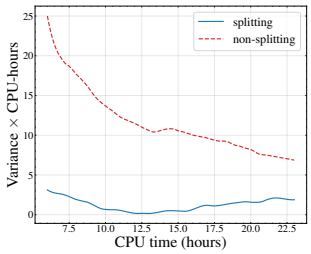
(e) BDR of poppler



(f) variance of poppler



(g) BDR of stb



(h) variance of stb

Figure 26: Ablation study for fuzzer AFLSmart on arrow, php, poppler and stb.

Assume the coupled-noise construction satisfies the following exclusion property: conditional on $X_{1,t}$ and on selecting $J_t = j$, the second replica draws

$$X_{2,t}[j] \sim f_j(\cdot) \text{ restricted to } D_j \setminus \{X_{1,t}[j]\},$$

i.e., $X_{2,t}[j] \neq X_{1,t}[j]$ almost surely at the mutated location (all other coordinates evolve as in A1).

Then there exists $\rho \in (0, 1]$ such that for every t,

$$\mathbb{P}(X_{2,t} \in B,\, X_{1,t} \in B) \;\le\; (1-\rho)\,\mathbb{P}(X_{1,t} \in B)\,\mathbb{P}(X_{2,t} \in B),$$

i.e., Assumption A3 holds. Moreover, one may take

$$\rho \;=\; 1 - \frac{\sum_j P(j) \sum_{\ell \in V_j \setminus \{x[j]\}} f_j(\ell)}{\sum_j P(j) \sum_{\ell \in V_j} f_j(\ell)},$$

where $x[j]$ denotes the (bug-triggering) value at location j realized by $X_{1,t}$ on the event $\{X_{1,t} \in B\}$. In the special case $|V_j| = 1$ for each j (at most one buggy value per location), one can take $\rho = 1$.

Proof. By the definition of conditional probability,

$$\mathbb{P}(X_{2,t} \in B,\, X_{1,t} \in B) = \mathbb{P}(X_{1,t} \in B)\,\mathbb{P}(X_{2,t} \in B \mid X_{1,t} \in B).$$

Fix t. Under the bitwise description, the (marginal) probability that $X_{2,t}$ is buggy can be written as

$$\mathbb{P}(X_{2,t} \in B) \;=\; \sum_j P(j) \sum_{\ell \in V_j} f_j(\ell).$$

On the event $\{X_{1,t} \in B\}$, at the bug-relevant location j the coupled-noise exclusion removes the realized value $X_{1,t}[j]$ from the support for $X_{2,t}[j]$. Hence, conditional on $J_t = j$,

$$\mathbb{P}(X_{2,t}[j] \in V_j \mid X_{1,t} \in B,\, J_t = j) \;\le\; \sum_{\ell \in V_j \setminus \{X_{1,t}[j]\}} f_j(\ell),$$

which is strictly smaller than $\sum_{\ell \in V_j} f_j(\ell)$ whenever $f_j(X_{1,t}[j]) > 0$ and $X_{1,t}[j] \in V_j$. Averaging over j yields

$$\begin{aligned}\mathbb{P}(X_{2,t} \in B \mid X_{1,t} \in B) \;&\le\; \sum_j P(j) \sum_{\ell \in V_j \setminus \{X_{1,t}[j]\}} f_j(\ell) \\ &=\; (1-\rho)\,\mathbb{P}(X_{2,t} \in B)\end{aligned}$$

for ρ as defined in the statement. Multiplying by $\mathbb{P}(X_{1,t} \in B)$ gives

$$\mathbb{P}(X_{2,t} \in B,\, X_{1,t} \in B) \;\le\; (1-\rho)\,\mathbb{P}(X_{1,t} \in B)\,\mathbb{P}(X_{2,t} \in B),$$

which is exactly Assumption A3. For the special case $|V_j| = 1$, the exclusion implies $\mathbb{P}(X_{2,t} \in B \mid X_{1,t} \in B) = 0$, so $\rho = 1$ works. □

A4 (Geometric ergodicity and mixing approximation).. Assume that the Markov chain model is geometrically ergodic (under the coupled-noise construction). Consequently, after a burn-in period ℓ that exceeds the mixing time, the marginal probabilities admit approximations of the following form (up to exponentially small errors):

$$\mathbb{P}(U_{1,t} \in B) \approx \mathbb{P}(U_{2,t} \in B) \approx \pi(1).$$

A5 (Rare-bug regime).. Assume that $\pi(1) \ll \pi(0)$, so bugs occur with low probability, consistent with practical settings.

C.1.2. Stationary Markov Chains with One Split. In this section, we formally compare the *splitting estimator* with the vanilla *MC estimator* for stationary Markov chains under a single split.

Variance comparison.. The variance of the splitting estimator satisfies

$$\begin{aligned}\sigma^2_{\text{Splitting}} &= V_R(T) \\ &= \frac{1}{2T^2}\,\mathrm{Var}\Big(\sum_{t=\tau+1}^{T} Y_{1,t}\Big) + \frac{1}{T^2}\,\mathrm{Var}\Big(\sum_{t=1}^{\tau-1} Y_{s,t}\Big) \\ &\qquad - \frac{\rho}{2T^2} \cdot \sum_{t=\tau+1}^{T} \mathbb{P}(X_{1,t} \in B)\,\mathbb{P}(X_{2,t} \in B).\end{aligned}$$

Moreover, the difference $\sigma^2_{\text{Splitting}} - \sigma^2_{MC}$ is approximately of the order

$$G_T = \frac{\sigma_f^2}{2T} - \frac{\rho}{2T} = \frac{1}{2T}\big(\sigma_f^2 - \rho\big),$$

where

$$\sigma_f^2 = \lim_{T\to\infty} \mathrm{Var}\Big(\frac{1}{\sqrt{T}} \sum_{t=1}^{T} \mathbf{1}_{\{X_t \in B\}}\Big),$$

and ρ is defined through the correlation control condition

$$\mathbb{P}(X_{2,t} \in B,\ X_{1,t} \in B) \le (1-\rho)\cdot\mathbb{P}(X_{1,t} \in B)\cdot\mathbb{P}(X_{2,t} \in B).$$

Correlation model and the origin of ρ.. We model the fuzzing process as an autoregressive system of the form

$$\begin{aligned} X_{1,t} &= \phi(X_{1,t-1}, W_{1,t}, Z_{1,t}) = \phi(X_{1,t-1}) \cdot W_{1,t} + Z_{1,t} \cdot W_{1,t} \\ &= \sum_{j=1}^{l} \phi_j(X_{1,t-1}) W_{1,t}^j + \sum_{j=1}^{l} Z_{1,t^l} W_{1,t}^l. \end{aligned} \tag{1}$$

This formulation implies the correlation bound

$$\mathbb{P}(X_{2,t} \in B,\ X_{1,t} \in B) \le (1-\rho)\cdot\mathbb{P}(X_{1,t} \in B)\cdot\mathbb{P}(X_{2,t} \in B).$$

Here, *masking* is represented by $\phi = (\phi_1, \ldots, \phi_l)$, where each ϕ_j sets location j to `NULL`. The *selections* are given by $W_{k,t} = (W_{k,t}^1, \ldots, W_{k,t}^l)$ for $k \in \{1, 2\}$, where $W_{k,t}^j \sim$ Bernoulli indicates selection of location j. Finally, the *inputs* $Z_{1,t}$ and $Z_{2,t}$ are identically distributed (with CDF F and PDF f) but dependent (via $U_{1,t}$ and $1 - U_{1,t}$).

Closed-form expressions.. Consequently, we obtain the following expressions for σ_f^2 and ρ:

$$\sigma_f^2 \ = \ \pi(1)\pi(0) \ + \ \frac{\pi(1)}{\pi(0)} \cdot \frac{1-\gamma_{M,R}}{\gamma_{M,R}}, \qquad \rho \ = \ \min\Big\{1,\ \frac{1}{|D|\,\pi(1)}\Big\},$$

where the clamp at 1 keeps $1 - \rho \in [0, 1)$, as required by the correlation-control condition A3; the unclamped value $1/(|D|\pi(1))$ does not exceed 1 whenever $\pi(1) \ge 1/|D|$, i.e., whenever the bug set is non-empty under the uniform reference measure. Note that ρ depends on the chain only through its stationary law, not on the detailed transition structure of P_R. In particular, the dominating terms satisfy

$$\sigma_f^2 \approx \frac{\pi(1)}{\gamma_{M,R}}, \qquad \rho \approx \frac{1}{|D|\,\pi(1)}.$$

When does splitting increase or reduce variance?. We compare σ_f^2 and ρ through the threshold condition

$$\pi(1) \ \gtrless \ \sqrt{\gamma_{M,R}/|D|}.$$

If $\pi(1) > \sqrt{\gamma_{M,R}/|D|}$ (i.e., $\sigma_f^2 \ge \rho$), then splitting yields a slight variance increase. If $\pi(1) \le \sqrt{\gamma_{M,R}/|D|}$ (small underlying bug rate), then splitting yields smaller variance, demonstrating its advantage in variance reduction. Thus, splitting is particularly beneficial when the program's bug rate $\pi(1)$ is small relative to $\sqrt{\gamma_{M,R}/|D|}$.

Sample complexity advantage.. Finally, we compare sample complexities. Without splitting, the sample complexity is $2T$, whereas with splitting it is $2T - \tau$. Since $2T - \tau < 2T$, splitting has lower sample complexity than non-splitting. Overall, splitting can reduce variance in the low-bug-rate regime and also improves sample complexity, supporting further development and adoption of the approach.

C.1.3. Non-stationary Markov Chains with One Split. We next extend the variance comparison between the splitting-based estimator and the vanilla Monte Carlo (MC) estimator to the setting of non-stationary Markov chains under a single split.

Assumption in the non-stationary regime: controlled deviation and regularity.. For the non-stationary extension, the proof assumes the deviation between the stationary distribution π and the distribution induced by the (time-varying) kernel over a burn-in horizon can be controlled by a total-variation-type bound q. The proof further assumes a regularity condition on the mapping that defines the correlation constant (e.g., $b(\cdot)$ is Lipschitz, motivated by smoothness), allowing the distributional deviation to translate into a deviation bound on ρ of the form $|\rho - \rho_{\text{ref}}| \le L\delta$, where L depends on the transition kernel P_R and δ quantifies the magnitude of non-stationarity.

Closed-form expressions and bounds.. Consequently, we obtain the following expressions for σ_f^2 and bounds for ρ:

$$\sigma_f^2 \ = \ \pi(1)\pi(0) \ + \ \frac{\pi(1)}{\pi(0)} \cdot \frac{1-\gamma_{M,R}}{\gamma_{M,R}},$$

and

$$\left|\rho - \frac{1}{|D|\,|\pi(1)|}\right| \ \le \ L \cdot \delta,$$

where L depends on the transition kernel P_R.

In particular, the dominating term in σ_f^2 satisfies

$$\sigma_f^2 \approx \frac{\pi(1)}{\gamma_{M,R}},$$

and the bounds on ρ can be equivalently written as

$$\frac{1}{|D|\,|\pi(1)|} - L\delta \ \le \ \rho \ \le \ \frac{1}{|D|\,|\pi(1)|} + L\delta.$$

Variance comparison across three regimes.. Let the benchmark term be $\frac{1}{|D|\,|\pi(1)|}$ with uncertainty radius $L\delta$. Then, the comparison between σ_f^2 and ρ yields three regimes.

First, if

$$\frac{\pi(1)}{\gamma_{M,R}} \ > \ \frac{1}{|D|\,|\pi(1)|} + L\delta \qquad (\text{i.e., } \sigma_f^2 \ge \rho),$$

then splitting yields a *slight increase* in variance.

Second, if

$$\frac{\pi(1)}{\gamma_{M,R}} \ < \ \frac{1}{|D|\,|\pi(1)|} - L\delta,$$

then the splitting algorithm has *smaller variance*, demonstrating its advantage in the low-bug-rate regime.

Third, if

$$\frac{1}{|D|\,|\pi(1)|} - L\delta \ < \ \frac{\pi(1)}{\gamma_{M,R}} \ < \ \frac{1}{|D|\,|\pi(1)|} + L\delta,$$

then σ_f^2 and ρ are *comparable*. In this indeterminate middle regime, we cannot definitively claim which variance is larger, and neither splitting nor non-splitting exhibits a significant variance advantage based on these bounds alone. The uncertainty term $L\delta$ (driven by P_R) therefore creates a middle region in which variance ordering is not identifiable from bounds alone.

Sample complexity advantage.. Regardless of the variance ordering, splitting improves sample complexity. Specifically, the non-splitting method has sample complexity $2T$, whereas the splitting method has sample complexity $2T - \tau$. Since $2T - \tau < 2T$, splitting has *lower sample complexity* than non-splitting. In other words, even when variance benefits are unclear (i.e., in the indeterminate middle regime), splitting still provides a consistent improvement through reduced sample complexity. A more rigorous statement requires computing the overall work efficiency by jointly accounting for variance and sample complexity.

Appendix D. Discussions on the Assumptions

We stress that, although our analysis is built on an ergodicity assumption, this condition plays an important practical role: it rules out poorly performing fuzzers whose mutation processes can become trapped in restricted regions of the input space. Notably, our empirical results align closely with the theoretical predictions even when this assumption is not enforced. Similar assumptions are standard in the Markov chain literature. While it is certainly of interest to derive stronger results that relax ergodicity—and we hope this work motivates such future efforts—any rigorous analysis must begin with a baseline. In this sense, our work constitutes a first step, providing the initial analytical framework for studying Markov chain behavior in the context of fuzzing.

We further assume that the Markov chain is time-homogeneous, i.e., its transition matrix does not vary over time. This assumption is satisfied by most existing fuzzers, since their mutation rules are fixed and implemented as static code, which uniquely determines the transition matrix.

Finally, although our analysis focuses on the case of a single input seed, scenarios with multiple seeds can be accommodated naturally. If multiple seeds are executed in parallel, the process is equivalent to running multiple independent trials, which is already covered by our framework. If, instead, seeds are fixed in advance and cycled over time, this can be modeled by augmenting the transition matrix to include deterministic transitions to new seeds, followed by a restart under the original transition dynamics—effectively inducing a regeneration structure in the Markov chain.

Appendix E. Case Study of Splitting

Consider the case where D is a grid of integers in the set $[0, 9] \times [0, 9]$ in two-dimensional space. Suppose the set of bugs forms the sub-grid $[3, 5] \times [3, 5]$, i.e., a compact and convex subset of the input space. In this case, the path will split whenever it enters $[3, 5] \times [3, 5]$.

Assume the fuzzer is AFL, in that it randomly selects a location between 1 and 2 and modifies the value at the selected location with a random value. Notably, AFL is reversible which allows us to utilize the conclusion above. Without splitting, then we can write down the transition matrix as

$$P(x, y) = \frac{1}{10} 1_{x_1=y_1, x_2=y_2} + \\ (\frac{1}{20}) 1_{x_1=y_1, x_2 \neq y_2} + (\frac{1}{20}) 1_{x_1 \neq y_1, x_2=y_2}$$

Step 1: define a state-dependent autocovariance weight.. Let $f = \mathbf{1}_B$ and $p := \pi(B)$. Define the centered function $g := f - p$. For a Markov chain with kernel P, define the state-dependent (signed) deviation

$$\Delta_\ell(x) \;:=\; (P^\ell f)(x) - p \;=\; \mathbb{P}_x(X_\ell \in B) - \pi(B),$$

and define the local weight

$$c(x) \;:=\; g(x)^2 \;+\; 2 \sum_{\ell \geq 1} \big| g(x)\, \Delta_\ell(x) \big|.$$

This quantity is the natural refinement of a global "integrated autocovariance": it measures how strongly the event $X_t \in B$ remains informative about future membership in B, starting from state x. In particular, for any time t, one may take $c_t := c(X_t)$ in the bound

$$\mathrm{Var}(\hat{\mu}_T) \;\leq\; \frac{1}{T^2} \sum_{t=1}^{T} \frac{c(X_t)}{k_t},$$

which makes clear that times/states with larger $c(\cdot)$ contribute more and are precisely the ones you want larger k_t on.

Step 2: compute π and $p = \pi(B)$ for the grid AFL kernel.. Let us consider

$$p = \pi(B) = \frac{|B|}{|D|} = \frac{9}{100}.$$

Moreover,

$$g(x) = \begin{cases} 1 - p, & x \in B, \\ -p, & x \notin B. \end{cases}$$

Step 3: compute $\Delta_\ell(x) = \mathbb{P}_x(X_\ell \in B) - p$ explicitly.. Let $m := |[3, 5]| = 3$, so $m/10 = 3/10$. For any initial state $x = (x_1, x_2)$, define

$$a_i(x) := \mathbf{1}\{x_i \in [3, 5]\}, \qquad i = 1, 2.$$

Key observation for this kernel: at each step you resample exactly one coordinate uniformly in $\{0, \ldots, 9\}$. Hence, after ℓ steps, a given coordinate i is *never refreshed* with probability $(1/2)^\ell$, and is refreshed at least once with probability $1 - (1/2)^\ell$. Using a partition on whether one coordinate is never

refreshed (which occurs iff all ℓ updates chose the other coordinate), one obtains for every $\ell \geq 1$:

$$\mathbb{P}_x(X_\ell \in B) = \underbrace{\Big(1-2^{1-\ell}\Big)\Big(\frac{m}{10}\Big)^2}_{\text{both coordinates refreshed at least once}} + \underbrace{2^{-\ell}\Big(\frac{m}{10}\Big)\big(a_1(x)+a_2(x)\big)}_{\text{exactly one coordinate never refreshed}}.$$

Since $p = (m/10)^2$, this implies the exact deviation formula

$$\Delta_\ell(x) = \mathbb{P}_x(X_\ell \in B) - p = 2^{-\ell}\Big(\frac{m}{10}\Big)\Big(a_1(x)+a_2(x)-2\frac{m}{10}\Big).$$

Specialization to the clustered-bug case $x \in B$.. If $x \in B$, then $a_1(x) = a_2(x) = 1$, so for all $\ell \geq 1$,

$$\Delta_\ell(x) = 2^{-\ell}\Big(\frac{m}{10}\Big)\Big(2-2\frac{m}{10}\Big) = 2^{1-\ell}\Big(\frac{m}{10}\Big)\Big(1-\frac{m}{10}\Big).$$

For $m = 3$, this becomes

$$\Delta_\ell(x) = 0.42 \cdot 2^{-\ell}, \qquad x \in B.$$

This is an explicit *persistence/return characterization*: starting inside the bug cluster, the excess probability of still being in B after ℓ steps decays exactly as $2^{-\ell}$, with amplitude $(m/10)(1-m/10)$ controlled by the cluster width m. (Heuristically, $2^{-\ell}$ is the probability that the update schedule has failed to "refresh away" at least one coordinate.)

Step 4: compute $c(x)$ explicitly for $x \in B$.. For $x \in B$, $g(x) = 1-p$ and $\Delta_\ell(x) \geq 0$, so the absolute values drop:

$$c(x) = (1-p)^2 + 2(1-p)\sum_{\ell \geq 1}\Delta_\ell(x), \qquad x \in B.$$

Using $\sum_{\ell \geq 1} 2^{-\ell} = 1$ and the formula above, for $x \in B$

$$\sum_{\ell \geq 1}\Delta_\ell(x) = \sum_{\ell \geq 1} 2^{1-\ell}\Big(\frac{m}{10}\Big)\Big(1-\frac{m}{10}\Big) = 2\Big(\frac{m}{10}\Big)\Big(1-\frac{m}{10}\Big),$$

hence

$$c(x) = (1-p)^2 + 4(1-p)\Big(\frac{m}{10}\Big)\Big(1-\frac{m}{10}\Big), \qquad x \in B.$$

Plugging $m = 3$ and $p = 9/100$ yields for $x \in B$,

$$\begin{aligned} c(x) &= (0.91)^2 + 4(0.91)(0.3)(0.7) \\ &= 0.8281 + 0.7644 = 1.5925. \end{aligned}$$

This implies that, for $x \in B$, the contribution term satisfies $\frac{c(x)}{k_t} = \frac{1.5925}{2} < 1$, which demonstrates the advantage of splitting at the bug-triggering point.

Appendix F. Additional Theoretical Guarantee of Splitting

F.0.1. Measure on Splitting Performance. Consistent with the existing work on splitting, we assess the effectiveness of the proposed splitting strategy τ by comparing the variance of the resulting estimator to that under the optimal strategy τ^*. Specifically, we consider the relative efficiency

$$R_v \;=\; \frac{\mathcal{D}(\tau^*)}{\mathcal{D}(\tau)},$$

where

$$\mathcal{D}(\tau) \;=\; \int \tau^{-1}(x)\,\mathrm{Var}\big(\hat{P}_R(T)\big)\,\pi^2(x)\,dx,$$

and τ is defined by

$$\tau(x) \;=\; \frac{\beta(x)\pi(x)}{\int \beta(u)\pi(u)\,du}.$$

Here, $\beta(x)$ denotes the splitting kernel associated with state x.

Under our splitting algorithm, natural choices include $\beta(x) = \mathbb{P}\big(R(x) \in B\big)$ or $\beta(x) = \mathbf{1}\{R(x) \in B\}$. These choices lead to different implications for the efficiency of the resulting fuzzers.

Theorem 6 (efficiency)**.** *Let us assume that there exists δ, such that $1 \geq \delta_1 \geq P(R(x) \in B) \geq \delta > 0$ for any x. Let us also assume that $P(R(x) \in B)$ is computable. Let us assume that $\beta(x) = P(R(x) \in B)$. Then we have*

$$R_v \geq V,$$

where $V = \frac{\delta}{\delta_1}$.

Theorem 7 (efficiency)**.** *Let us assume that there exists δ, such that $P(R(x) \in B) > \delta$ for any x. Let us also assume that $\omega(x) = 1_{R(x) \in B}$ and $\beta(x) = \min(\omega(x), a)$. Then we have*

$$R_v \geq V$$

where $V = \max_a \min_x a \cdot P(R(x) \in B) \cdot P(R(x) \notin B) + P(R(x) \in B)^2$.

This implies that the resulting variance is at most $\frac{1}{V}$ times the optimal variance, where the optimum corresponds to the best achievable (but generally unknown) splitting strategy. Notably, the bound holds under mild conditions on the fuzzer, such as requiring the bug-reaching probability from any input to be uniformly bounded away from 0.

Appendix G. Additional Theoretical Results

In this section, we provide a theoretical study of the proposed performance estimators—both asymptotically and in finite time—and, consequently, of the newly introduced evaluation measures. We consider both asymptotic and finite-time regimes with respect to the fuzzing horizon T and the number of replications M. The motivation for explicitly incorporating M is as follows.

For each replication $m \in \{1, 2, \ldots, M\}$, the estimator

$$\tilde{P}_R^{(m)}(T) \;:=\; \frac{1}{T}\sum_{t=1}^{T}\mathbf{1}_{\{X_t^m \in B\}}$$

is computed from an independent simulation run (labeled by m). Since these replications are generated independently and are initialized from the same distribution, the collection $\{\tilde{P}_R^{(m)}(T)\}_{m=1}^M$ is (conditionally) i.i.d. across m. This experimental design aligns with the constrained super-Markov chain perspective formalized in [21], which advocates using many short chains rather than a single long chain, which is a recommendation that fits naturally in fuzz testing. Under this framework, the variability of $\tilde{P}_R^{(m)}(T)$ is governed by two factors: the convergence and mixing behavior of a single Markov chain as T increases, and the reduction in Monte Carlo error obtained by averaging across replications as M increases. Intuitively, an appropriate choice of M can reduce the required horizon T, enabling more efficient fuzzing in parallel computing environments.

We present the asymptotic analytical results in Section G.1, the finite-time results in Section G.2, and the corresponding evaluation framework in Section G.3.

G.1. Asymptotic analyses

In what follows, we analyze the asymptotic behavior of the estimator $\tilde{S}_R(T)$ and the corresponding variance of $\tilde{P}_R(T)$, which defines the evaluation measure $V_R(T)$, as well as the measures $A_R(T)$, $B_R(T)$, and $M_R(T)$.

Lemma 8. *For any ergodic Markov chain P_R, we obtain* $\lim_{T\to\infty} \tilde{P}_R(T) = P_R = \pi(1)$.

Proof. Fix a trial m. Since the chain $\{X_t^m\}_{t\geq 0}$ is ergodic on the finite state space D with stationary distribution π_R^X, Birkhoff's ergodic theorem (equivalently, the strong law of large numbers for ergodic Markov chains; see, e.g., [30]) applies to the bounded function $f = \mathbf{1}_{\{x\in B\}}$ and yields, for any initial state w,

$$\lim_{T\to\infty} \frac{N_T^m}{T} = \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}_{\{X_t^m\in B\}} = \sum_{x\in B} \pi_R^X(x) = \pi(1)$$

The estimator is the average over the M independent trials,

$$\tilde{P}_R(T) = \frac{1}{M}\sum_{m=1}^{M} \frac{N_T^m}{T},$$

a finite average of terms each converging almost surely to $\pi(1)$; hence $\tilde{P}_R(T) \to \pi(1)$ almost surely as $T\to\infty$. Since $0 \leq \tilde{P}_R(T) \leq 1$, the bounded convergence theorem gives convergence of the expectations as well, which establishes the desired claim and completes the proof.

We remark that the renewal description of bug arrivals offers a complementary view: the successive inter-arrival times between visits to B form a regenerative sequence whose mean return time, by Kac's formula, equals $1/\pi(1)$ when the chain is started inside B under stationarity; the renewal-reward theorem then recovers $N_T/T \to \pi(1)$. The ergodic-theorem argument above is the shortest self-contained route and requires no regeneration structure. □

Theorem 9 (Asymptotic property of sample variance (as $T\to\infty$))**.** *Let us assume that the transition matrix is ergodic. Then we have as T goes to ∞, the sample variance reads as*

$$\lim_{T\to\infty} \tilde{S}_R(T) = 0.$$

Theorem 10 (Asymptotic property of sample variance (as $M\to\infty$))**.** *Let us assume that the transition matrix is ergodic. Then we have as M goes to ∞, the sample variance reads as*

$$\lim_{M\to\infty} \tilde{S}_R(T) = S_R(T).$$

Theorem 11 (Asymptotic property of population variance (as $T\to\infty$))**.** *Let us assume that the transition matrix is ergodic. Then we have that as T goes to ∞, the population variance reads as*

$$\lim_{T\to\infty} V_R(T) = 0.$$

Theorem 12 (Asymptotic property of population variance (as $M\to\infty$))**.** *Let us assume that the transition matrix is ergodic. Then we have that as M goes to ∞, the population variance reads as*

$$\lim_{M\to\infty} V_R(T) = 0.$$

This implies that, with high probability, the variances of the estimator $\tilde{P}_R(T)$—including both the sample variance and the population variance—approach zero as time goes to infinity. For any fixed T, as $M\to\infty$, both the sample variance $S_R(T)$ and the population variance $V_R(T)$ converge to the finite-time performance measure $S_R(T)$. This holds for every fuzzer $R\in\mathcal{R}$.

Theorem 13 (Asymptotic property of $M_R(T)$ (as $T\to\infty$))**.** *Let us assume that the transition matrix is ergodic. Then we have that as T goes to ∞,* a.s.

$$\lim_{T\to\infty} M_R(T) = 0.$$

Consequently, we conclude that the mean squared error of $\tilde{P}_R(T)$ also converges to zero asymptotically with respect to T.

Likewise, we consider the asymptotic behavior of $M_R(T)$ when M goes to ∞.

Theorem 14 (Asymptotic property of $M_R(T)$ (as $M\to\infty$))**.** *Let us assume that the transition matrix is ergodic. Then we have that as M goes to ∞,*

$$\lim_{M\to\infty} M_R(T) = (P_R(T) - \pi(1))^2.$$

Thus far, convergence with respect to the fuzzing duration T implies convergence to zero. Naturally, we are also interested in asymptotic convergence toward a non-zero value, which characterizes the rate of convergence. This is particularly important for establishing the convergence of $A_R(T)$ and $B_R(T)$. To this end, we present the central limit theorem for Markov chains [30], which enables us to

characterize the non-zero asymptotic behavior of the variance. The formal statement is as follows.

Theorem 15 (Central Limit Theorem [30])**.** *Let $\{X_t\}_{t\geq 0}$ be a Markov chain with stationary distribution π, and let $f : \mathcal{X} \to \mathbb{R}$ be a function such that $\mathbb{E}_\pi[f(X_0)^2] < \infty$. Assume the Markov chain is* geometrically ergodic*, i.e., there exist constants $C < \infty$ and $0 < \lambda_{R,2} < 1$ such that for all $x \in \mathcal{X}$ and $t \geq 0$,*

$$\|P^t(x,\cdot) - \pi(\cdot)\|_{TV} \leq C\lambda_{R,2}^t.$$

Define the sample mean:

$$\bar{f}_T = \frac{1}{T}\sum_{t=1}^{T} f(X_k).$$

Then the Central Limit Theorem holds:

$$\sqrt{T}\left(\bar{f}_T - \mathbb{E}_\pi[f]\right) \xrightarrow{d} \mathcal{N}(0, \sigma_f^2),$$

where

$$\begin{aligned}\sigma_f^2 &= \lim_{T\to\infty} \mathrm{Var}\left(\frac{1}{\sqrt{T}}\sum_{t=1}^{T} f(X_t)\right) \\ &= \mathrm{Var}_\pi(f) + 2\sum_{k=1}^{\infty} \mathrm{Cov}_\pi(f(X_0), f(X_k)).\end{aligned}$$

In our context, $\bar{f}_T$ corresponds to $\tilde{P}_R(T)$ and $f = \mathbf{1}_{\{X\in B\}}$. We also assume that the Markov chain is geometrically ergodic. As a result, we leverage the central limit theorem to characterize robustness and to estimate P_R.

The above theorem establishes that $T \cdot V_R(T)$ and $T \cdot S_R(T)$ converge to $\sigma_f^2 = \sigma^2$, as defined previously. Equivalently, $\lim_{T\to\infty} A_R(T) = 0$, which in turn implies the following asymptotic result for $A_R(T)$.

Corollary 1 (Asymptotic property of $A_R(T)$)**.** *Let us assume that the transition matrix is ergodic. Then, as T goes to ∞, we have*

$$\lim_{T\to\infty} A_R(T) = 0.$$

Meanwhile, based on the aforementioned result for $V_R(T)$, we obtain the following asymptotic property of $A_R(T)$ with respect to M.

Corollary 2 (Asymptotic property of $A_R(T)$)**.** *Let us assume that the transition matrix is ergodic. Then, as M goes to ∞, we have*

$$\lim_{M\to\infty} A_R(T) = \left|T \cdot S_R(T) - \sigma^2\right|.$$

For the deviation of $\tilde{S}_R(T)$ from σ^2, we obtain the following asymptotic convergence result for $B_R(T)$.

Corollary 3 (Asymptotic property of $B_R(T)$)**.** *Let us assume that the transition matrix is ergodic. Then, as T goes to ∞, we have*

$$\lim_{T\to\infty} B_R(T) = \left|\lim_{T\to\infty} T \cdot V_R(T) - \sigma^2\right| = 0.$$

Additionally, the convergence result for $S_R(T)$ implies the following asymptotic property of $B_R(T)$ with respect to M.

Corollary 4 (Asymptotic property of $B_R(T)$)**.** *Let us assume that the transition matrix is ergodic. Then, as M goes to ∞, we have*

$$\lim_{M\to\infty} B_R(T) = \left|T \cdot S_R(T) - \sigma^2\right|.$$

Essentially, the convergence of $A_R(T)$, $B_R(T)$, and $M_R(T)$ is established via the corresponding results for $S_R(T)$ and $V_R(T)$, which substantially simplifies the analyses. We carry this observation forward in our finite-time analyses.

Next, it remains essential to understand the finite-time behavior of the estimators and to examine how closely they approximate their steady-state counterparts. This understanding is key to validating our methodology for assessing variability in the bug detection rate, constructing confidence intervals based on the central limit theorem, and evaluating the robustness of bug detection rates across different fuzzers.

G.2. Finite-time Analyses

In this section, we examine the finite-time behavior of the proposed performance estimator and evaluation measures. Specifically, we characterize the finite-time properties of variances $S_R(T)$ and $V_R(T)$. Building on these results, we extend the finite-time analysis to the error metrics $A_R(T)$ and $B_R(T)$, as well as the mean squared error $M_R(T)$.

More precisely, we begin by studying how the sample variance $S_R(T)$ and the population variance $V_R(T)$ converge to σ^2, which allows us to derive corresponding upper bounds on $A_R(T)$ and $B_R(T)$. We then establish finite-time upper bounds on $\tilde{S}_R(T)$ and $V_R(T)$, which in turn enable us to obtain an upper bound on $M_R(T)$.

Theorem 16 (Convergence of Sample Variance)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic. Let us assume that the simulation runs for T time steps. Then We have with probability at least $1 - 2\exp\{-cT^2/M\}$, where $c > 0$ is an absolute constant, that*

$$\begin{aligned}&|T \cdot \tilde{S}_R(T) - \sigma^2| \\ &\leq \frac{1}{M} + \left(\frac{1}{4} + \frac{1}{16\pi(1)^2}\right) \cdot \frac{(1-\gamma_{M,R})\big(1+(1-\gamma_{M,R})\big)}{\big(1-(1-\gamma_{M,R})\big)^2} \\ &\quad + \frac{2}{T}\,\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\,\frac{1-\gamma_{M,R}}{\gamma_{M,R}^2} \\ &\quad + 2\,\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\,\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}\end{aligned}$$

where $\gamma_{M,R}$ denotes the spectral gap constant of the Markov chain. In the non-reversible case, $\gamma_{M,R}$ is defined as the spectral gap of a suitable reversibilization of the chain. Specifically, let $P_{M,R}$ be a reversible kernel with respect

to π_R obtained from P_R (i.e., a reversibilization). We then define

$$\gamma_{M,R} \;:=\; 1-\lambda_2\big(P_{M,R}\big).$$

Theorem 17 (Convergence of Population Variance)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic. Let us assume that the simulation runs for T time steps. Then We have that*

$$\begin{aligned}
&|T\cdot V_R(T)-\sigma^2|\\
&=|Var(\sqrt{T}\tilde{P}_R(T))-\sigma^2|\\
&\le \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2}+\\
&\qquad 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}
\end{aligned}$$

Based on the above convergence results, we derive the following straight-forward corollary on the upper bound of $A_R(T)$ and $B_R(T)$.

Corollary 5 (Upper bounds on $A_R(T)$ and $B_R(T)$)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic. Let us assume that the simulation runs for T time steps. Then We have that*

$$\begin{aligned}
&A_R(T)\\
&\le \frac{1}{M}+\Big(\frac{1}{4}+\frac{1}{16\pi(1)^2}\Big)\cdot\frac{(1-\gamma_{M,R})\big(1+(1-\gamma_{M,R})\big)}{\big(1-(1-\gamma_{M,R})\big)^2}\\
&\quad+\frac{2}{T}\,\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\,\frac{1-\gamma_{M,R}}{\gamma_{M,R}^2}\\
&\quad+2\,\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\,\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}
\end{aligned}$$

and

$$\begin{aligned}
&B_R(T)\\
&\le \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2}+\\
&\qquad 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}.
\end{aligned}$$

Additionally, we establish finite-time upper bounds on $\tilde{S}_R(T)$ and $V_R(T)$ that hold for all T. These results yield conservative guarantees when $\tilde{P}_R(T)$ must be computed using only empirical estimators. We present bounds for both reversible and non-reversible Markov chains, since the resulting expressions depend on whether the chain is reversible; moreover, the analytical techniques required in the two cases differ substantially.

We begin with the reversible setting, where the transition kernel $P_R(\cdot,\cdot)$ satisfies the *detailed balance* (reversibility) condition: there exists a stationary distribution π_R such that, for all states $x, y \in \mathcal{X}$,

$$\pi_R(x)\,P_R(x,y) \;=\; \pi_R(y)\,P_R(y,x).$$

Reversible Markov chains.

Theorem 18 (Sample variance)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic, reversible, and positive. Let us assume that the simulation runs for T time steps. Then We have*

$$\begin{aligned}
\sqrt{\tilde{S}_R(T)} &\le \sqrt{2\frac{Var(\sum_{t=1}^{T}1_{X_t^m\in B})}{M}+(\frac{4\delta_R}{T})^2}\\
&\le \sqrt{2\frac{K_R\cdot(\pi(1)(1-\pi(1)))}{MT^2}+(\frac{4\delta_R}{T})^2}
\end{aligned}$$

where K_R is defined as in [31].

Subsequently, we establish an upper bound on the population variance, stated in the following result for $V_R(T)$.

Theorem 19 (Population variance)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic, reversible, and positive. Let us assume that the simulation runs for T time steps. Then we obtain that*

$$\begin{aligned}
V_R(T) \le \min(&2(1-\pi(1))^2\frac{1}{T^2}\frac{1}{\gamma_R}+\\
&2(\sqrt{\frac{\log T}{T}}+\sqrt{\frac{4\delta_R}{T}})^2,\frac{K_R\cdot(\pi(1)(1-\pi(1)))}{MT^2}).
\end{aligned}$$

where K_R is defined as in [31].

The above results establish robustness guarantees for the fuzzers, but they apply only to reversible Markov chains, which can be restrictive in practice. We therefore turn to general non-reversible chains and derive corresponding finite-time upper bounds on the sample and population variances.

Non-reversible Markov chains. For general geometrically ergodic Markov chains (without assuming reversibility), prior work derives variance bounds by relating the chain to a Metropolis-Hastings (M-H) *reversibilization* and its spectral gap. In what follows, we adopt the framework of [3] and begin by defining the M-H spectral gap under M-H *reversibilization*.

Definition 1 (M-H spectral gap)**.** *Let us assume that M_1 and M_2 are the first and second Metropolis-Hastings (M-H) Markov chains associated with P_R. Then, the M-H spectral gap of P_R is defined as follows:*

$$\gamma_R^{MH} = 1-\sup_{n\in\mathbb{N}}\{L(M_2(P_R^n))^{\frac{1}{n}},U(M_1(P_R^n))^{\frac{1}{n}}\}$$

where L and U represents the operators on a transition matrix P

$$\begin{aligned}
L(P) &= \inf_{f\in L_0^2(\pi),||f||_\pi=1} < Pf, f >_\pi\\
U(P) &= \sup_{f\in L_0^2(\pi),||f||_\pi=1} < Pf, f >_\pi
\end{aligned}$$

Additionally, we define C and the complement of C as C^c as follows. Formally, we define a feasible region C as

$$C = \{n\in\mathbb{N} : L(M_2(P_R^n)) \le 1, U(M_1(P_R^n)) \le 1\}$$

and

$$C^c = \mathbb{N} \backslash C.$$

With these definitions and tools for the non-reversible setting in place, we proceed to establish upper bounds on both the sample variance and the true variance via Metropolis-Hastings reversibilization for general Markov chains. The formal statements are as follows.

Theorem 20 (Sample variance)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic, and positive. Let us assume that the simulation runs for T time steps. We have*

$$\tilde{S}_R(T) \leq \frac{2}{M}(\pi(1)(1-\pi(1)(|C|^c + \frac{2}{\gamma_R^{MH}}) + (\frac{4\delta_R}{T})^2).$$

where δ_R is the corresponding relaxation time, defined as $\frac{1}{\gamma_R^{MH}}$.

Subsequently, we establish the following upper bound on the population variance $V_R(T)$.

Theorem 21 (Population variance)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic, and positive. Let us assume that the simulation runs for T time steps. Then we obtain that*

$$V_R(T) \leq \pi(1)(1-\pi(1))(|C|^c + \frac{2}{\gamma_R^{MH}}).$$

Next, we turn to the finite-time analysis of the mean squared error for a fixed simulation horizon T. Specifically, we derive finite-time upper bounds on the MSE to compare worst-case performance across different fuzzers. As before, we present results for both reversible and non-reversible Markov chains, reflecting the distinct structural properties of their transition kernels.

We emphasize that, given our bounds on $V_R(T)$ and the bias term, an upper bound on the mean squared error $M_R(T)$ follows directly. We begin with the reversible case in Theorem 22 and then extend the analysis to general non-reversible chains in Theorem 23.

Theorem 22 (Mean Square Error)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic, reversible, and positive. Let us assume that the simulation runs for T time steps. Then we derive that*

$$\begin{aligned} M_R(T) \leq \min(&2(1-\pi(1))^2 \frac{1}{T^2}\frac{1}{\gamma_R} \\ &+ 2(\sqrt{\frac{\log T}{T}} + \sqrt{\frac{4\delta_R}{T}})^2, \frac{K_R \cdot (\pi(1)(1-\pi(1)))}{MT^2}) \\ &+ (\min(C_R \frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}))^2. \end{aligned}$$

where C_R, α_R, K_R are defined as: $C_R = \alpha^{-r}, \alpha_R = \theta^{\frac{1}{r}}$ are constants associated with the transition matrix P_R^X, with $\theta = 1 - \min_{x_1,x_2}(\frac{(P_R^X)^r(x_1,x_2)}{\pi(x_2)}, r = \min_t (P_R^X)^t(x_1,x_2) > 0$ for any x_1, x_2; $(P_R^X)^t$ denotes P_R^X to the power of t; K_R is defined as in Theorem 19 following [31].

Theorem 23 (Mean Square Error)**.** *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic, and positive. Let us assume that the simulation runs for T time steps. We hence have*

$$\begin{aligned} M_R(T) \leq &\frac{\pi(1)(1-\pi(1)(|C|^c + \frac{2}{\gamma_R^{MH}})}{MT} \\ &+ (\min(C_R \frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}))^2. \end{aligned}$$

G.3. Evaluation Results

G.3.1. $V_R(T)$**-based Evaluation.** We next present the evaluation based on $V_R(T)$.

Theorem 24. *For each fuzzer $R \in \mathcal{R}$, define an upper bound $U_R(T)$ on $V_R(T)$ as follows.*

1) ***Reversible Markov chains.*** *Define $U_R^{\mathrm{rev}}(T) := \min\Big(2(1-\pi(1))^2 \frac{1}{T^2}\frac{1}{\gamma_R} + 2\big(\sqrt{\frac{\log T}{T}} + \sqrt{\frac{4\delta_R}{T}}\big)^2, \frac{K_R \cdot \big(\pi(1)(1-\pi(1))\big)}{MT^2}\Big)$, where $K_R \leq 1 + \frac{2\lambda_{R,2}}{T(1-\lambda_{R,2})^2}$.*
2) ***Non-reversible Markov chains.*** *Define $U_R^{\mathrm{nonrev}}(T) := \pi(1)\big(1-\pi(1)\big)\Big(|C|^c + \frac{2}{\gamma_R^{\mathrm{MH}}}\Big)$.*

Selecting any $\hat{R} \in \arg\min_{R \in \mathcal{R}} U_R^{\mathrm{rev}}(T)/U_R^{\mathrm{nonrev}}(T)$ yields a fuzzer that minimizes this upper bound on $V_R(T)$, and thus identifies the best fuzzer under this criterion.

G.3.2. $S_R(T)$**-based Evaluation.**

Theorem 25. *Let T be the simulation horizon and M the number of trials. For each fuzzer $R \in \mathcal{R}$, define an upper bound $U_R(T)$ on the deviation $\tilde{S}_R(T)$ as follows.*

1) ***Reversible Markov chains.*** *Define $U_R^{\mathrm{rev}}(T) := 2\frac{K_R \cdot \big(\pi(1)(1-\pi(1))\big)}{MT^2} + \big(\frac{4\delta_R}{T}\big)^2$, where $K_R \leq 1 + \frac{2\lambda_{R,2}}{T(1-\lambda_{R,2})^2}$.*
2) ***Non-reversible Markov chains.*** *Define $U_R^{\mathrm{nonrev}}(T) := \frac{2}{M}\big(\pi(1)\big(1-\pi(1)\big)\big(|C^c| + \frac{2}{\gamma_R^{\mathrm{MH}}}\big) + (4\delta_R)^2\big)$.*

Selecting any $\hat{R} \in \arg\min_{R \in \mathcal{R}} U_R^{\mathrm{rev}}(T)/U_R^{\mathrm{nonrev}}(T)$ yields a fuzzer with the smallest deviation $\tilde{S}_R(T)$, and thus selects the best fuzzer under this criterion.

G.3.3. $M_R(T)$**-based Evaluation.** Lastly, we turn to an evaluation based on $M_R(T)$, which incorporates both variance and bias information and is widely used in machine learning and statistics.

Theorem 26. *For each fuzzer $R \in \mathcal{R}$, define an upper bound $U_{M,R}(T)$ on $M_R(T)$ as follows.*

1) ***Reversible Markov chains.*** *Define $U_{M,R}^{\mathrm{rev}}(T) := \min\Big(2(1-\pi(1))^2 \frac{1}{T^2}\frac{1}{\gamma_R} + 2\big(\sqrt{\frac{\log T}{T}} + \sqrt{\frac{4\delta_R}{T}}\big)^2, \frac{K_R \cdot \big(\pi(1)(1-\pi(1))\big)}{MT^2}\Big) +$*

$\Big(\min\big(C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}\big)\Big)^2$, *where* $K_R \leq 1+\frac{2\lambda_{R,2}}{T(1-\lambda_{R,2})^2}$.

2) ***Non-reversible Markov chains.*** *Define* $U^{\mathrm{nonrev}}_{M,R}(T) := \frac{\pi(1)\big(1-\pi(1)\big)\big(|C|^c+\frac{2}{\gamma^{\mathrm{MH}}_R}\big)}{MT} + \Big(\min\big(C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}\big)\Big)^2$.

Selecting any $\hat{R} \in \arg\min_{R\in\mathcal{R}} U^{\mathrm{rev}}_{M,R}(T)/U^{\mathrm{nonrev}}_{M,R}(T)$ *yields a fuzzer that minimizes this upper bound on* $M_R(T)$*, and thus selects the best fuzzer under this criterion.*

Appendix H. Proof

H.1. Proofs for the Asymptotic Analyses (Section G.1)

Proof of Theorem 9. We note the following key observation (by definition):

$$\lim_{T\to\infty}\frac{1}{T}\sum_{t=1}^{T}1_{X_t^m\in B} = P_R(T).$$

Consequently, we obtain

$$\begin{aligned}
&\lim_{T\to\infty}\tilde{S}_R(T)\\
&=\lim_{T\to\infty}\sqrt{\frac{1}{M}\sum_{m=1}^{M}\Big(\tilde{P}_R(T)-\frac{1}{T}\sum_{t=1}^{T}1_{X_t^m\in B}\Big)^2}\\
&=\lim_{T\to\infty}\sqrt{\frac{1}{M}\sum_{m=1}^{M}\Big(P_R(T)-\frac{1}{T}\sum_{t=1}^{T}1_{X_t^m\in B}\Big)^2}\\
&=\lim_{T\to\infty}\sqrt{\frac{1}{M}\sum_{m=1}^{M}\big(P_R(T)-P_R(T)\big)^2}\\
&=0,
\end{aligned}$$

which completes the proof. □

Proof of Theorem 10. By definition,

$$\tilde{S}_R(T) = \frac{1}{M}\sum_{m=1}^{M}\Big(\tilde{P}_R(T)-\frac{1}{T}\sum_{t=1}^{T}1_{X_t^m\in B}\Big)^2,$$

where $\frac{1}{T}\sum_{t=1}^{T}1_{X_t^m\in B}$ are independently and identically distributed based on the MC simulation process. Then, by the strong law of large numbers, we obtain

$$\begin{aligned}
\lim_{M\to\infty}\tilde{S}_R(T) &= E\Big[\Big(\tilde{P}_R(T)-\frac{1}{T}\sum_{t=1}^{T}1_{X_t^m\in B}\Big)^2\Big]\\
&= Var\Big(\frac{1}{T}\sum_{t=1}^{T}1_{X_t^m\in B}\Big) = S_R(T),
\end{aligned}$$

which concludes the proof. □

Proof of Theorem 11. By definition, we obtain

$$\begin{aligned}
&\lim_{T\to\infty}V_R(T)\\
&=\lim_{T\to\infty}\sqrt{\frac{1}{M}\sum_{m=1}^{M}\big(E[\tilde{P}_R(T)]-\tilde{P}_R(T)\big)^2}\\
&=\lim_{T\to\infty}\sqrt{\frac{1}{M}\sum_{m=1}^{M}\big(\pi(1)-\tilde{P}_R(T)\big)^2}\\
&=\lim_{T\to\infty}\sqrt{\frac{1}{M}\sum_{m=1}^{M}\big(\pi(1)-\pi(1)\big)^2}\\
&=0,
\end{aligned}$$

which finishes the proof. □

Proof of Theorem 12. Based on

$$V_R(T) = E\big[(\tilde{P}_R(T)-E[\tilde{P}_R(T)])^2\big],$$

we have that

$$\begin{aligned}
\lim_{M\to\infty}V_R(T) &= E\Big[\lim_{M\to\infty}\big(\tilde{P}_R(T)-E[\tilde{P}_R(T)]\big)^2\Big]\\
&= E\big[(E[\tilde{P}_R(T)]-E[\tilde{P}_R(T)])^2\big]\\
&= 0,
\end{aligned}$$

which completes the proof. □

Proof of Theorem 13. Again, by definition,

$$\lim_{T\to\infty}M_R(T) = \lim_{T\to\infty}V_R(T) + \big(\mathbb{E}[D_R(T)]\big)^2.$$

Meanwhile, by Theorem 11 and by Lemma 8,

$$\lim_{T\to\infty}V_R(T)=0, \qquad \lim_{T\to\infty}D_R(T)=0,$$

and the claim follows by exchanging the limit and expectation. □

Proof of Theorem 14. Notably,

$$\begin{aligned}
\lim_{M\to\infty}M_R(T) &= \lim_{M\to\infty}E\Big(\tilde{P}_R(T)-\lim_{T\to\infty}P_R(x\in I_R^T : x\in H_R^T)\Big)^2\\
&= E\big[(P_R(T)-P_R)^2\big]\\
&= (P_R(T)-P_R)^2 = (P_R(T)-\pi(1))^2.
\end{aligned}$$

□

H.2. Proofs for the Finite-time Analyses (Section G.2)

Proof. Theorem 16 Let us recall that $\lambda_{R,2}$ denotes the second largest eigenvalue of P_R. For example, in the two-state model, we derive that $\lambda_{R,2} = 1-P_R(0,1)-P_R(1,0)$.

First, by definition, we arrive at

$$\begin{aligned}&Var(TP_R(T))\\&=\frac{1}{M}\sum_{i=1}^{T}\sum_{j=1}^{T}Cov(1_{X_i^m\in B},1_{X_j^m\in B}))\\&=\sum_{t=-(T-1)}^{(T-1)}(T-t)\gamma_t\end{aligned}$$

where $\gamma_t = Cov(1_{X_0^m\in B}, 1_{X_t^m\in B})$.

Notably, $\gamma_t = \gamma_{-t}$, and $\gamma_0 = Var(1_{X_0^m\in B})$, we obtain

$$\begin{aligned}&Var(\sqrt{T}P_R(T))\\&=\frac{1}{T}Var(TP_R(T))\\&=\gamma_0+2\sum_{t=1}^{(T-1)}(1-\frac{t}{T})\gamma_t\end{aligned}$$

Meanwhile, by definition,

$$\sigma^2=\gamma_0+2\sum_{t=1}^{\infty}\gamma_t$$

Subsequently, we derive

$$\begin{aligned}&Var(\sqrt{T}P_R(T))\\&=\sigma^2-2\sum_{t=1}^{\infty}\gamma_t+2\sum_{t=1}^{(T-1)}(1-\frac{t}{T})\gamma_t\\&=\sigma^2-2\sum_{t=1}^{T-1}\gamma_t-2\sum_{t=T}^{\infty}\gamma_t+2\sum_{t=1}^{(T-1)}(1-\frac{t}{T})\gamma_t\\&=\sigma^2-2\sum_{t=1}^{(T-1)}(\frac{t}{T})\gamma_t-2\sum_{t=T}^{\infty}\gamma_t\end{aligned}$$

It is worth noting that for a geometrically ergodic markov chain, we have

$$\begin{aligned}&|2\sum_{t=T}^{\infty}\gamma_t|\le 2\sum_{t=T}^{\infty}|\gamma_t|\\&=2\sum_{t=T}^{\infty}|Cov(1_{X_0^m\in B},1_{X_t^m\in B})|\\&\le 2\sum_{t=T}^{\infty}|P(X_t^m\in B,X_0^m\in B)-P(X_t^m\in B)P(X_0^m\in B)|\\&=2\sum_{t=T}^{\infty}|(P(X_t^m\in B|X_0^m\in B)-P(X_t^m\in B))P(X_0^m\in B)|\end{aligned}$$

Notably, we have the following decomposition

$$\begin{aligned}&|(P(X_t^m\in B|X_0^m\in B)-P(X_t^m\in B))|\\&\le|(P(X_t^m\in B|X_0^m\in B)-\pi(1)+\pi(1)-P(X_t^m\in B))|\\&\le\frac{(1-\gamma_{M,R})^t}{4\pi(1)}+\frac{(1-\gamma_{M,R})^t}{4\max\{\pi(0),\pi(1)\}}\\&\le\frac{(1-\gamma_{M,R})^t}{2\max\{\pi(0),\pi(1)\}}\end{aligned}$$

This immediately gives us that

$$\begin{aligned}&|2\sum_{t=T}^{\infty}\gamma_t|\\&\le 2\sum_{t=T}^{\infty}|(P(X_t^m\in B|X_0^m\in B)-P(X_t^m\in B))P(X_0^m\in B)||\\&\le 2\pi(1)\sum_{t=T}^{\infty}\frac{(1-\gamma_{M,R})^t}{2\max\{\pi(0),\pi(1)\}}\\&=2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}\end{aligned}$$

Consequently, we obtain

$$\begin{aligned}|Var(\sqrt{T}P_R(T))-\sigma^2|\le&\left|\frac{1}{T}\sum_{t=1}^{\infty}t\cdot\gamma_t\right|\\&+2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}.\end{aligned}$$

where $\gamma_t = Cov(1_{X_0^m\in B}, 1_{X_t^m\in B})$

More precisely, based on the above derivation, we have that for a geometrically ergodic Markov chain, the lag-t autocovariance obeys the bound

$$|\gamma_t|\le\frac{\pi(1)(1-\gamma_{M,R})^t}{2\max\{\pi(0),\pi(1)\}}$$

which immediately implies that

$$\begin{aligned}&\frac{2}{T}\sum_{t=1}^{\infty}t\cdot|\gamma_t|\\&\le\frac{2}{T}\sum_{t=1}^{\infty}t\cdot\frac{\pi(1)(1-\gamma_{M,R})^t}{2\max\{\pi(0),\pi(1)\}}\\&=\frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2}\end{aligned}$$

Putting them together, we derive that

$$\begin{aligned}|Var(\sqrt{T}P_R(T))-\sigma^2|\le&\frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2}\\&+2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}.\end{aligned}$$

Next, we examine the difference between $Var(\sqrt{T}P_R(T))$ and $S_R(T)$. Notably,

$$E[S_R(T)]=Var_{\pi_0}(\sqrt{T}P_R(T))$$

Meanwhile,

$$\begin{aligned}&|Var_{\pi_0}(\sqrt{T}P_R(T))-Var(\sqrt{T}P_R(T))|\\&\le\frac{C_2}{T}\end{aligned}$$

We next establish an upper bound on C_2. From total variation distance estimates and standard covariance perturbation bounds (see), we use the following general result that

holds for bounded functions f and geometrically ergodic Markov chains, and is stated for example in Lemma 2.1 of [7]. Although their primary focus is on batch means estimators, this bound itself applies more broadly and is independent of the specific variance estimation method. Formally, we have

$$\begin{aligned}&|\mathrm{Cov}_{\pi_0}(f(X_i), f(X_j)) - \mathrm{Cov}_{\pi}(f(X_i), f(X_j))| \\ &\leq (\frac{1}{4} + \frac{1}{16\pi_1^2}) \cdot ((1-\gamma_{M,R}))^{\max(i,j)}\end{aligned}$$

where

$$f(x) = 1_{x \in B}.$$

This holds true by the following observation. Note that we assume that $i \leq j$ without loss of generalizability

$$\begin{aligned}
&|\mathrm{Cov}_{\pi_0}(f(X_i), f(X_j)) - \mathrm{Cov}_{\pi}(f(X_i), f(X_j))| \\
&\quad\leq \Big| P(X_i \in B, X_j \in B | X_0 \sim \pi_0) \\
&\qquad - P(X_i \in B | X_0 \sim \pi_0) \cdot P(X_j \in B | X_0 \sim \pi_0) \\
&\qquad - P(X_i \in B, X_j \in B | X_0 \sim \pi) \\
&\qquad + P(X_i \in B | X_0 \sim \pi) \cdot P(X_j \in B | X_0 \sim \pi) \Big| \\
&\quad\leq \Big| P(X_i \in B, X_j \in B | X_0 \sim \pi_0) \\
&\qquad - P(X_i \in B | X_0 \sim \pi_0) \cdot P(X_j \in B | X_0 \sim \pi_0) \\
&\qquad - P(X_i \in B, X_j \in B | X_0 \sim \pi) \\
&\qquad + P(X_i \in B | X_0 \sim \pi) \cdot P(X_j \in B | X_0 \sim \pi) \Big| \\
&\quad\leq \Big| P(X_i \in B | X_0 \sim \pi_0) \cdot P(X_j \in B | X_i \in B) \\
&\qquad - P(X_i \in B | X_0 \sim \pi_0) \cdot P(X_j \in B | X_0 \sim \pi_0) \\
&\qquad - P(X_j \in B | X_i \in B) P(X_i \in B | X_0 \sim \pi) \\
&\qquad - P(X_i \in B | X_0 \sim \pi) \cdot P(X_j \in B | X_0 \sim \pi) \Big| \\
&\quad\leq P(X_j \in B | X_i \in B) \Big| P(X_i \in B | X_0 \sim \pi_0) - P(X_i \in B | X_0 \sim \pi) \Big| \\
&\qquad + \Big| P(X_i \in B | X_0 \sim \pi_0) \cdot P(X_j \in B | X_0 \sim \pi_0) \\
&\qquad - P(X_i \in B | X_0 \sim \pi) \cdot P(X_j \in B | X_0 \sim \pi) \Big| \\
&\quad\leq P(X_j \in B | X_i \in B) ||\mu_i - \pi||_{TV} \\
&\qquad - P(X_i \in B | X_0 \sim \pi_0) \cdot P(X_j \in B | X_0 \sim \pi_0) \\
&\qquad - P(X_i \in B | X_0 \sim \pi_0) \cdot P(X_j \in B | X_0 \sim \pi) \\
&\qquad + P(X_i \in B | X_0 \sim \pi_0) \cdot P(X_j \in B | X_0 \sim \pi) \\
&\qquad - P(X_i \in B | X_0 \sim \pi) \cdot P(X_j \in B | X_0 \sim \pi) | \\
&\quad\leq P(X_j \in B | X_i \in B) ||\mu_i - \pi||_{TV} \\
&\qquad - P(X_j \in B | X_0 \sim \pi) ||\mu_i - \pi||_{TV} \\
&\qquad + P(X_j \in B | X_0 \sim \pi) ||\mu_j - \pi||_{TV} \\
&\quad= (P(X_j \in B | X_i \in B) - P(X_j \in B | X_0 \sim \pi)) ||\mu_i - \pi||_{TV} \\
&\qquad + P(X_i \in B | X_0 \sim \pi) ||\mu_j - \pi||_{TV} \\
&\quad= ||\mu_{j-i} - \pi||_{TV} ||\mu_i - \pi||_{TV} \\
&\qquad + P(X_i \in B | X_0 \sim \pi) ||\mu_j - \pi||_{TV}
\end{aligned}$$

where $\mu_t = \mathcal{L}(X_t \mid X_0 \sim \pi_0)$.

Based on the total variation distance result:

$$\|\mu_t - \pi\|_{\mathrm{TV}} = \frac{1}{2} \left| \mathbb{P}(X_t = 1) - \pi(1) \right| = \frac{1}{4\pi_1}(1-\gamma_{M,R})^t,$$

where $\pi_1 = \frac{P_R(0,1)}{P_R(0,1)+P_R(1,0)}$ is the stationary probability of state 1.

Subsequently, we derive that

$$\begin{aligned}
&|\mathrm{Cov}_{\pi_0}(f(X_i), f(X_j)) - \mathrm{Cov}_{\pi}(f(X_i), f(X_j))| \\
&\leq \frac{1}{4\pi_1}(1-\gamma_{M,R})^{j-i} \frac{1}{4\pi_1}(1-\gamma_{M,R})^{i} + \pi(1) \cdot \frac{1}{4\pi_1}(1-\gamma_{M,R})^{j} \\
&\leq (\frac{1}{4} + \frac{1}{16\pi_1^2})(1-\gamma_{M,R})^{j}
\end{aligned}$$

Likewise, we can obtain the same result for $i > j$, and thus validate the claim.

Hence,

$$\begin{aligned}&|\mathrm{Cov}_{\mu}(f(X_i), f(X_j)) - \mathrm{Cov}_{\pi}(f(X_i), f(X_j))| \\ &\qquad \leq \left(\frac{1}{4} + \frac{1}{16\pi_1^2}\right) \cdot \left(1-\gamma_{M,R}\right)^{\max(i,j)}.\end{aligned}$$

where

$$f(x) = 1_{x \in B}$$

This yields:

$$|C_2| \leq \sum_{i=1}^{\infty} \sum_{j=1}^{\infty} (\frac{1}{4} + \frac{1}{16\pi_1^2})(1-\gamma_{M,R})^{\max(i,j)}.$$

We now compute the sum:

$$\begin{aligned}
&\sum_{i=1}^{\infty} \sum_{j=1}^{\infty} (1-\gamma_{M,R})^{\max(i,j)} \\
&= 2 \sum_{k=1}^{\infty} k(1-\gamma_{M,R})^k - \sum_{k=1}^{\infty} (1-\gamma_{M,R})^k \\
&= 2 \cdot \frac{(1-\gamma_{M,R})}{(1-(1-\gamma_{M,R}))^2} - \frac{(1-\gamma_{M,R})}{1-(1-\gamma_{M,R})} \\
&= \frac{(1-\gamma_{M,R})(1+(1-\gamma_{M,R}))}{(1-(1-\gamma_{M,R}))^2}.
\end{aligned}$$

Therefore,

$$\boxed{C_2 \leq (\frac{1}{4} + \frac{1}{16\pi_1^2}) \cdot \frac{(1-\gamma_{M,R})(1+(1-\gamma_{M,R}))}{(1-(1-\gamma_{M,R}))^2}}$$

Likewise, if we start from $X_0 = 0$, then we replace $\pi(1)$ with $\pi(0)$, and everything follows the same logic. Henceforth, we skip the proof steps herein. As a result, we establish that

$$\boxed{C_2 \leq (\frac{1}{4} + \frac{1}{16\pi_1^2}) \cdot \frac{(1-\gamma_{M,R})(1+(1-\gamma_{M,R}))}{(1-(1-\gamma_{M,R}))^2}.}$$

Lastly, we apply a bounded-differences (McDiarmid) concentration argument to $S_R(T)$. The estimator $S_R(T)$ is a function of the M independent trials, and each per-trial summand is almost surely bounded by S^2, the deterministic

upper bound on the sample variance established in Theorem 18; consequently, replacing any single trial changes $S_R(T)$ by at most $4S^2/M$. McDiarmid's inequality then yields, for any $\varepsilon > 0$,

$$P(|S_R(T) - E[S_R(T)]| \geq \varepsilon) \leq 2\exp\Big(-\frac{M\varepsilon^2}{8S^4}\Big).$$

We instantiate $\varepsilon = \frac{1}{M}$, which gives deviation probability at most $2\exp\{-\frac{1}{8MS^4}\}$.

Putting everything together, we obtain

$$\begin{aligned}
&|S_R(T) - \sigma^2| \\
&= |S_R(T) - E[S_R(T) + E[S_R(T)] - Var_{\pi_0}(\sqrt{T}P_R(T)) + \\
&\qquad Var_{\pi_0}(\sqrt{T}P_R(T)) - Var(\sqrt{T}P_R(T)) + \\
&\qquad Var(\sqrt{T}P_R(T)) - \sigma^2| \\
&\leq |S_R(T) - E[S_R(T)| + |E[S_R(T)] - Var_{\pi_0}(\sqrt{T}P_R(T))| \\
&\qquad + |Var_{\pi_0}(\sqrt{T}P_R(T)) - Var(\sqrt{T}P_R(T))| + \\
&\qquad |Var(\sqrt{T}P_R(T)) - \sigma^2|
\end{aligned}$$

With probability at least $1 - 2\exp\{-\frac{1}{8MS^4}\}$, we have that

$$\begin{aligned}
&|S_R(T) - \sigma^2| \\
&\leq |S_R(T) - E[S_R(T)| + |E[S_R(T)] - Var_{\pi_0}(\sqrt{T}P_R(T))| \\
&\qquad + |Var_{\pi_0}(\sqrt{T}P_R(T)) - Var(\sqrt{T}P_R(T))| + \\
&\qquad |Var(\sqrt{T}P_R(T)) - \sigma^2| \\
&\leq \frac{1}{M} + 0 + \frac{C_2}{T} + \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2} + \\
&\qquad 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}
\end{aligned}$$

Plugging in $C_2 \leq \frac{(P_R(0,1)+P_R(1,0))(1-\gamma_{M,R})(1+(1-\gamma_{M,R}))}{\min\{P_R(1,0),P_R(0,1)\}(1-(1-\gamma_{M,R}))^2}$, we arrive at with probability at least $1 - 2\exp\{-\frac{1}{8MS^4}\}$

$$\begin{aligned}
&|S_R(T) - \sigma^2| \\
&\leq \frac{1}{M} + \frac{C_2}{T} + \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2} + \\
&\qquad 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}} \\
&\leq \frac{1}{M} + (\frac{1}{4} + \frac{1}{16\pi_1^2}) \cdot \frac{(1-\gamma_{M,R})(1+(1-\gamma_{M,R}))}{(1-(1-\gamma_{M,R}))^2} + \\
&\qquad \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2} + \\
&\qquad 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}
\end{aligned}$$

Based on Theorem 18, we obtain that

$$S^2 \leq O(\frac{1}{T})$$

so the deviation probability is at most $2\exp\{-cT^2/M\}$ for an absolute constant $c > 0$, which completes the proof.

□

Proof. Theorem 17 By definition, again, we have

$$\begin{aligned}
&|Var(\sqrt{T}P_R(T)) - \sigma^2| \\
&\leq |\frac{1}{T}\sum_{t=1}^{\infty} t \cdot \gamma_t| + 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}
\end{aligned}$$

where $\gamma_t = Cov(1_{X_0^m \in B}, 1_{X_t^m \in B})$

More precisely,for for a geometrically ergodic Markov chain, we derive that

$$\gamma_t = \frac{\pi(1)(1-\gamma_{M,R})^T}{2\max\{\pi(0),\pi(1)\}}$$

and subsequently

$$\begin{aligned}
&\frac{2}{T}\sum_{t=1}^{\infty} t \cdot \gamma_t \\
&= \frac{2}{T}\sum_{t=1}^{\infty} t \cdot \frac{\pi(1)(1-\gamma_{M,R})^T}{2\max\{\pi(0),\pi(1)\}} \\
&= \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2}
\end{aligned}$$

Putting them together, we derive that

$$\begin{aligned}
&|Var(\sqrt{T}P_R(T)) - \sigma^2| \\
&\leq \frac{2}{T}\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{1-\gamma_{M,R}}{(\gamma_{M,R})^2} + \\
&\qquad 2\frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\frac{(1-\gamma_{M,R})^T}{\gamma_{M,R}}
\end{aligned}$$

which concludes the proof.

□

Proof. Theorem 18

Firstly, by decomposing the sample variance, we derive that

$$\begin{aligned}
\sqrt{\tilde{S}_R(T)} &= \sqrt{\frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - \frac{1}{T}\sum_{t=1}^{T} 1_{X_t^m \in B})^2} \\
&= \sqrt{\begin{aligned}&\frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - E[\tilde{P}_R(T)] + E[\tilde{P}_R(T)] \\ &- \frac{1}{T}\sum_{t=1}^{T} 1_{X_t^m \in B})^2\end{aligned}} \\
&\leq \sqrt{\begin{aligned}&2\frac{1}{M}\sum_{m=1}^{M}\big(\tilde{P}_R(T) - \mathbb{E}[\tilde{P}_R(T)]\big)^2 \\ &+ \Big(\mathbb{E}[\tilde{P}_R(T)] - \frac{1}{T}\sum_{t=1}^{T}\mathbf{1}_{\{X_t^m \in B\}}\Big)^2\end{aligned}} \\
&\leq \sqrt{2Var(\tilde{P}_R(T)) + (\frac{4\delta_R}{T})^2} \\
&\leq \sqrt{2\frac{Var(\sum_{t=1}^{T} 1_{X_t^m \in B})}{MT^2} + (\frac{4\delta_R}{T})^2}
\end{aligned}$$

Noting that there is a dependency on the true variance of $\tilde{P}_R(T)$ and the Markov chain we consider is reversible, we leverage the variance bounding technique for reversible Markov chains. It is known that for reversible, positive, geometrically ergodic Markov chains [31],

$$Var(\sum_{t=1}^{T} 1_{X_t^m \in B}) \leq K \cdot Var_\pi(1_{X_t^m \in B}) = K_R \cdot (\pi(1)(1-\pi(1)))$$

This immediately indicates that

$$\begin{aligned}\sqrt{\tilde{S}_R(T)} &\leq \sqrt{2\frac{Var(\sum_{t=1}^{T} 1_{X_t^m \in B})}{M} + (\frac{4\delta_R}{T})^2} \\ &\leq \sqrt{2\frac{K_R \cdot (\pi(1)(1-\pi(1)))}{MT^2} + (\frac{4\delta_R}{T})^2}\end{aligned}$$

which completes the first part of the statement.

Meanwhile, based on the following lemma, we derive the upper bound on K_R

Lemma 27. *Let us assume that the transition matrices of all fuzzers P_R for $R \in \mathcal{R}$ are geometrically ergodic, reversible, and positive. Then we have*

$$K_R \leq 1 + \frac{2\lambda_{R,2}}{T(1-\lambda_{R,2})^2}$$

where $\lambda_{R,2}$ represents the spectral radius of the transition matrix P_R, denoted by the second largest eigenvalue of P_R.

Proof. The proof is straightforward by the decomposition of $Var(\sum_{t=1}^{T} 1_{X_t^m \in B})$. More precisely, we note that

$$\begin{aligned}&Var(T1_{X_t^m \in B})) \\ &= \sum_{i=1}^{T}\sum_{j=1}^{T} Cov(1_{X_i^m \in B}, 1_{X_j^m \in B})) \\ &= T \cdot \gamma_0 + 2\sum_{t=1}^{(T-1)} (T-t)\gamma_t\end{aligned}$$

where $\gamma_t = Cov(1_{X_0^m \in B}, 1_{X_t^m \in B})$.

Based on the spectral property of reversible Markov chain,

$$\gamma_t \leq \lambda_{R,2}^t \frac{1}{\pi}$$

Plugging in, we derive that

$$\begin{aligned}&Var(\sqrt{T}1_{X_t^m \in B})) \\ &= \gamma_0 + 2\frac{1}{T}\sum_{t=1}^{(T-1)} (T-t)\lambda_{R,2}^t \frac{1}{\pi} \\ &\leq 1 + \frac{2\lambda_{R,2}}{T(1-\lambda_{R,2})^2}\end{aligned}$$

□

We observe that the upper bound on $S_R(T)$ is monotone decreasing in the variance bound constant K_R and δ_R, as well as being monotone decreasing in $\lambda_{R,2}$. Therefore, selecting the fuzzer with the smallest $\lambda_{R,2}$ and δ_R minimizes the upper bound, which finishes the second part of statement.

□

Proof. Theorem 19 The population variance reads as

$$\begin{aligned}Var(\tilde{P}_R(T)) &= \frac{1}{M}\sum_{m=1}^{M}(E[\tilde{P}_R(T)] - \tilde{P}_R(T))^2 \\ &= \frac{1}{M}\sum_{m=1}^{M}(E[\tilde{P}_R(T)] - \pi(1) + \pi(1) - \tilde{P}_R(T))^2 \\ &\leq \frac{2}{M}\sum_{m=1}^{M}(E[\tilde{P}_R(T)] - \pi(1))^2 + (\pi(1) - \tilde{P}_R(T))^2\end{aligned}$$

Recall that we previously establish that

$$\begin{aligned}&|E[P_R(x \in I_R^T : x \in H_R^T)] - \pi(1)|| \\ &\leq (1-\pi(1))\frac{1}{T}\frac{1}{\gamma_R}\end{aligned}$$

and

$$|\tilde{P}_R(T) - \pi(1)| \leq \sqrt{\frac{\log T}{T}} + \sqrt{\frac{4\delta_R}{T}}$$

This immediately implies that the population variance has a finite-time upper bound on the population variance, which reads as

$$\begin{aligned}&\frac{1}{M}\sum_{m=1}^{M}(E[\tilde{P}_R(T)] - \tilde{P}_R(T))^2 \\ &\leq 2(1-\pi(1))^2\frac{1}{T^2}\frac{1}{\gamma_R} + 2(\sqrt{\frac{\log T}{T}} + \sqrt{\frac{4\delta_R}{T}})^2\end{aligned}$$

Meanwhile, based on the derivation of the sample variance, we obtain that

$$\begin{aligned}&Var(\tilde{P}_R(T)) \\ &\leq \frac{Var(\sum_{t=1}^{T} 1_{X_t^m \in B})}{MT^2} \\ &\leq \frac{K_R\pi(1)(1-\pi(1))}{MT^2}\end{aligned}$$

Consequently, we derive the following upper bound on the true variance

$$\begin{aligned}V_R(T) \leq \min(&2(1-\pi(1))^2\frac{1}{T^2}\frac{1}{\gamma_R} + \\ &2(\sqrt{\frac{\log T}{T}} + \sqrt{\frac{4\delta_R}{T}})^2, \frac{K_R\pi(1)(1-\pi(1))}{MT^2})\end{aligned}$$

By selecting the fuzzer with parameters $gamma_R$, δ_R, and K_R that minimizes the above quantity, we minimize the upper bound on the variance and thus select the best fuzzer. This completes the proof.

□

Proof. Theorem 20 By the aforementioned decomposition of $\tilde{S}_R(T)$, we obtain that

$$\tilde{S}_R(T) = \frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - \frac{1}{T}\sum_{t=1}^{T}1_{X_t^m \in B})^2$$
$$\leq 2\frac{Var(\sum_{t=1}^{T}1_{X_t^m \in B})}{MT^2} + (\frac{4\delta_R}{T})^2$$

Theorem 6.1 in [3] directly suggests that

$$Var(\sum_{t=1}^{T}1_{X_t^m \in B})$$
$$\leq T \cdot Var(1_{X_t^m \in B}) \cdot (|C|^c + \frac{2}{\gamma_R^{MH}})$$
$$\leq \pi(1)(1-\pi(1)(|C|^c + \frac{2}{\gamma_R^{MH}}).$$

Consequently, we derive that

$$\tilde{S}_R(T) \leq 2\frac{\pi(1)(1-\pi(1)(|C|^c + \frac{2}{\gamma_R^{MH}})}{MT^2} + (\frac{4\delta_R}{T})^2$$

Thus far, selecting the fuzzer that maximizes this upper bound provides the choice of best fuzzer, which completes the proof.

□

Proof. Theorem 21 We derive the following upper bound on the variance of the Markov chain associated with P_R through the M-H spectral gap, as introduced in [3]. Based on Theorem 6.1 in [3], we obtain the upper bound on the population variance as

$$Var(\sum_{t=1}^{T}1_{X_t^m \in B})$$
$$\leq T \cdot Var(1_{X_t^m \in B}) \cdot (|C|^c + \frac{2}{\gamma_R^{MH}})$$
$$\leq \pi(1)(1-\pi(1)(|C|^c + \frac{2}{\gamma_R^{MH}}).$$

Subsequently, the upper bound on the true variance of the estimator $\tilde{P}_R(T)$ reads as

$$Var(\tilde{P}_R(T))$$
$$\leq \frac{Var(\sum_{t=1}^{T}1_{X_t^m \in B})}{MT^2}$$
$$\leq \frac{T \cdot Var(1_{X_t^m \in B}) \cdot (|C|^c + \frac{2}{\gamma_R^{MH}})}{MT^2}$$
$$\leq \frac{\pi(1)(1-\pi(1)(|C|^c + \frac{2}{\gamma_R^{MH}})}{MT}$$

□

Proof. Theorem 22 It is worth noting that the mean square error can be decomposed as follows

$$M_R(T) = \frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - \lim_{T\to\infty}P_R(x \in I_R^T : x \in H_R^T))^2$$
$$= \frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - E[\tilde{P}_R(T)] + E[\tilde{P}_R(T)] -$$
$$\lim_{T\to\infty}P_R(x \in I_R^T : x \in H_R^T))^2$$
$$= \frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - E[\tilde{P}_R(T)])^2 + (E[\tilde{P}_R(T)] -$$
$$\lim_{T\to\infty}P_R(x \in I_R^T : x \in H_R^T))^2$$
$$= Var(\tilde{P}_R(T)) + (E[\tilde{P}_R(T)] - \lim_{T\to\infty}P_R(x \in I_R^T : x \in H_R^T))^2$$

This essentially indicates that $M_R(T) = V_R(T) + B_R(T)$ where $B_R(T)$ represents the bias of the estimator $\tilde{P}_R(T)$.

Based on the aforementioned analysis, we obtain that

$$\lim_{T\to\infty}P_R(x \in I_R^T : x \in H_R^T) = \pi(1)$$

and

$$|E[\tilde{P}_R(T)] - \pi(1)||$$
$$= |\frac{1}{T}\sum_{t=1}^{T}P_R^t(1) - \pi(1)|$$
$$\leq \frac{1}{T}\sum_{t=1}^{T}C_R\alpha_R^t$$

and

$$\frac{1}{T}\sum_{t=1}^{T}C_R\alpha_R^t \leq \min(C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}})$$

Subsequently, we derive that

$$M_R(T) \leq Var(\tilde{P}_R(T)) + (\min(C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}))^2$$

which depends on the true variance.

Notably, for the true variance, we establish that for reversible Markov chains in Theorem 19:

$$Var(\tilde{P}_R(T)) \leq \min(2(1-\pi(1))^2\frac{1}{T^2}\frac{1}{\gamma_R} +$$
$$2(\sqrt{\frac{\log T}{T}} + \sqrt{\frac{4\delta_R}{T}})^2, \frac{K_R \cdot (\pi(1)(1-\pi(1)))}{MT^2}).$$

Subsequently, we have that for reversible Markov chains, the mean square error of $\tilde{P}_R(T)$ has the following upper bound

$$M_R(T)^2 \leq Var(\tilde{P}_R(T)) + (\min(C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}))^2$$
$$\leq \min(2(1-\pi(1))^2\frac{1}{T^2}\frac{1}{\gamma_R} + 2(\sqrt{\frac{\log T}{T}} + \sqrt{\frac{4\delta_R}{T}})^2,$$
$$\frac{K_R \cdot (\pi(1)(1-\pi(1)))}{MT^2})$$
$$+ (\min(C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}))^2$$

Consequently, selecting the fuzzer based on the minimal value of the upper bound leads to optimal fuzzer selection with respect to the mean square error.

This completes the proof.

□

Proof. Theorem 23 Likewise, we have the decomposition of $M_R(T)$ and the same upper bound on the second bias term $B_R(T)$.

$$M_R(T) = \frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - \lim_{T\to\infty} P_R(x \in I_R^T : x \in H_R^T))^2$$
$$= \frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - E[\tilde{P}_R(T)] + E[\tilde{P}_R(T)] -$$
$$\lim_{T\to\infty} P_R(x \in I_R^T : x \in H_R^T))^2$$
$$= \frac{1}{M}\sum_{m=1}^{M}(\tilde{P}_R(T) - E[\tilde{P}_R(T)])^2 + (E[\tilde{P}_R(T)] -$$
$$\lim_{T\to\infty} P_R(x \in I_R^T : x \in H_R^T))^2$$
$$= Var(\tilde{P}_R(T)) + (E[\tilde{P}_R(T)] - \lim_{T\to\infty} P_R(x \in I_R^T : x \in H_R^T))^2$$

Subsequently, we derive that

$$M_R(T) \leq Var(\tilde{P}_R(T)) + (\min(C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}))^2$$

which depends on the true variance.

Notably, for the true variance, we establish that for non-reversible Markov chains in Theorem 21:

$$Var(\tilde{P}_R(T)) \leq \frac{\pi(1)(1-\pi(1)(|C|^c + \frac{2}{\gamma_R^{MH}})}{MT}.$$

Consequently, putting everything together, we derive the following upper bound on the estimator $M_R(T)$

$$M_R(T) = Var(\tilde{P}_R(T)) +$$
$$\Big(\min\Big\{C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}\Big\}\Big)^2$$
$$\leq \frac{\pi(1)\Big(1-\pi(1)\big(|C|^c + \frac{2}{\gamma_R^{MH}}\big)\Big)}{MT} +$$
$$\Big(\min\Big\{C_R\frac{1}{T}\frac{1}{1-\alpha_R}, \sqrt{\frac{4\delta_R}{T}}\Big\}\Big)^2.$$

□

H.3. Proofs for the Splitting Efficiency Guarantees (Appendix F)

Proof. Theorem 6

Define the (diagonal) second-moment term

$$d_{ii}(x) := \mathbb{E}\big[\hat{h}_{ii}(x)^2\big] - \Big(\mathbb{E}\big[\hat{h}_{ii}(x)\big]\Big)^2. \tag{2}$$

Define the normalized weight

$$\tau(x) := \frac{\pi(x)\,\bar{w}(x)}{\int \pi(u)\,\beta(u)\,\bar{w}(u)\,du}. \tag{3}$$

Let the branching density be

$$\beta(x) := \frac{1}{\bar{w}(x)} \quad (\text{or } 0 \text{ if } w=\infty), \qquad w(x) = \min_j \hat{p}_j \cdot \hat{P}_R(x \in B). \tag{4}$$

With the above notation,

$$R_v := \frac{\det\big(\mathcal{D}(\tilde{w})(T)\big)}{\det\big(\mathcal{D}(\mathbf{1})(T)\big)} = \frac{\int \tau^{-1}(x)\,d_{ii}(x)\,\pi(x)\,dx}{\int d_{ii}(x)\,\pi(x)\,dx}. \tag{5}$$

When $\pi(x) = 1/|D|$, (3) becomes

$$\tau(x) = \frac{\pi(x)\,\bar{w}(x)}{\int \pi(u)\,\frac{1}{w(u)}\,du} = \frac{w(x)}{|D|}\int \frac{1}{w(u)}\,du. \tag{6}$$

Moreover, if $d_{ii}(x) = Var_T(\hat{h}_{ii}(x))$ is independent of x, i.e.,

$$d_{ii}(x) = Var_T\big(\hat{h}_{ii}(x)\big) = Var\Big(\int g\,p(x)\Big), \quad \text{independent of } x, \tag{7}$$

then (5) reduces to

$$R_v = \int \tau^{-1}(x)\,\pi^{-1}(x)\,dx = \frac{1}{|D|^2}\int \tau^{-1}(x)\,dx$$
$$= \frac{\int \beta(x)\,dx}{|D|^2}\int \frac{1}{\beta(x)}\,dx. \tag{8}$$

Consequently, we derive that

$$R_v = \frac{\int \beta(x)\,dx}{|D|^2}\int \frac{1}{\beta(x)}\,dx$$
$$= \frac{\int \frac{1}{w(x)}\,dx}{|D|^2}\int w(x)\,dx.$$

where $w(x) = \frac{1}{\beta(x)}$.

Let us consider that

$$w(x) = \mathbb{P}\big(R(x,w) \in B\big).$$

Assume $\quad \delta \leq \mathbb{P}\big(R(x,w)\in B\big) \leq \delta_1,\ \forall x, \qquad$ (hence $\delta \leq \delta_1$).

$$\int \Big(\frac{1}{\mathbb{P}(R(x,w)\in B)}\Big)^a dx \geq \frac{1}{|\delta_1|^a}.$$

Notably, assuming $\mathbb{P}\big(R(x,w) \in B\big) \geq \delta$ allows us to define $\beta(x) := \frac{1}{\mathbb{P}\left(R(x,w)\in B\right)}$, and this implies that

$$R_v \geq \frac{\delta}{\delta_1}.$$

□

Proof. Theorem 7 By the definition of R_v, we have that

$$\begin{aligned}
R_v &= \frac{\int \beta(x)\,dx \int \frac{1}{\beta(x)}\,dx}{|D|^2} \\
&\geq \frac{\int \beta(x)\,dx \int \omega(x)\,dx}{|D|^2} \\
&\geq \frac{\int \beta(x)\,dx \int \mathbf{1}_{\{R(x,w)\in B\}}\,dx}{|D|^2} \\
&= \frac{\int \beta(x)\,dx\, \mathbb{P}\big(R(x,w)\in B\big)}{|D|} \\
&= \Big[a\,\mathbb{P}\big(R(x,w)\notin B\big) + \mathbb{P}\big(R(x,w)\in B\big)\Big]\,\mathbb{P}_x\big((x,w)\in B\big) \\
&= a\,\mathbb{P}\big(R(x,w)\notin B\big)\,\mathbb{P}_x\big((x,w)\in B\big) \;+\; \mathbb{P}\big(R(x,w)\in B\big)^2.
\end{aligned}$$

Choose a and x such that

$$\begin{aligned}
&a\,\mathbb{P}\big(R(x,w)\notin B\big)\,\mathbb{P}_x\big((x,w)\in B\big) \;+\; \mathbb{P}\big(R(x,w)\in B\big)^2 \\
&= \max_a \min_x a\,\mathbb{P}\big(R(x,w)\notin B\big)\,\mathbb{P}_x\big((x,w)\in B\big) \;+\; \mathbb{P}\big(R(x,w)\in B\big)^2
\end{aligned}$$

This gives us that

$$R_v \geq V,$$

where

$$\begin{aligned}
V = \max_a \min_x\, & a\,\mathbb{P}\big(R(x,w)\notin B\big)\,\mathbb{P}_x\big((x,w)\in B\big) \\
&+\; \mathbb{P}\big(R(x,w)\in B\big)^2.
\end{aligned}$$

This demonstrates the efficiency effectiveness of the splitting algorithm compared to the optimal splitting measure. □

H.4. Proofs for the Splitting Comparison (Section 6)

Setup.. Recall $Y_{k,t} = \mathbf{1}\{X_{k,t} \in B\}$ for post-split replications $k \in \{1,2\}$ and $Y_{s,t} = \mathbf{1}\{X_{s,t} \in B\}$ for the shared pre-split trajectory. Under one split at time τ, we have $N_t = 1$ for $t \leq \tau$ and $N_t = 2$ for $t > \tau$, so the splitting estimator can be written as

$$\hat{P}_R(T) = \frac{1}{T}\left(\sum_{t=1}^{\tau} Y_{s,t} + \sum_{t=\tau+1}^{T} \frac{1}{2}\sum_{k=1}^{2} Y_{k,t}\right). \tag{9}$$

At the split time τ, the two post-split trajectories start from the same state (the pre-split state at time τ), and all post-split randomness is generated independently of the pre-split randomness conditional on $X_{s,\tau}$. We view τ as a regeneration point and treat the pre- and post-split contributions as uncorrelated at the level of the variance decomposition.

Step 1: Variance decomposition.. Since $Y_{s,\tau}$ is fixed given the split event, it does not contribute to the variance. Using (9) and the regeneration-cycle viewpoint at τ, we decompose

$$\begin{aligned}
&Var\big(\hat{P}_R(T)\big) \\
&= Var\left(\frac{1}{T}\sum_{t=\tau+1}^{T}\frac{1}{2}\sum_{k=1}^{2} Y_{k,t} + \frac{1}{T}\sum_{t=1}^{\tau-1} Y_{s,t}\right) \\
&= \frac{1}{4T^2} Var\left(\sum_{t=\tau+1}^{T}\big(Y_{1,t}+Y_{2,t}\big)\right) + \frac{1}{T^2} Var\left(\sum_{t=1}^{\tau-1} Y_{s,t}\right).
\end{aligned} \tag{10}$$

Expanding the first term yields

$$\begin{aligned}
&Var\left(\sum_{t=\tau+1}^{T}\big(Y_{1,t}+Y_{2,t}\big)\right) \\
&= Var\left(\sum_{t=\tau+1}^{T} Y_{1,t}\right) + Var\left(\sum_{t=\tau+1}^{T} Y_{2,t}\right) + \\
&\quad 2Cov\left(\sum_{t=\tau+1}^{T} Y_{1,t},\, \sum_{t=\tau+1}^{T} Y_{2,t}\right).
\end{aligned} \tag{11}$$

By symmetry of the two post-split replications, $Var(\sum Y_{1,t}) = Var(\sum Y_{2,t})$. Step 2: Controlling the post-split covariance via the multiplicative correlation bound.. Using the correlation control condition

$$P(X_{1,t}\in B,\; X_{2,t}\in B) \;\leq\; (1-\rho)P(X_{1,t}\in B)P(X_{2,t}\in B),$$

we obtain for each $t > \tau$,

$$\begin{aligned}
&Cov(Y_{1,t}, Y_{2,t}) \\
&= P(X_{1,t}\in B,\; X_{2,t}\in B) - P(X_{1,t}\in B)P(X_{2,t}\in B) \\
&\leq -\rho\cdot P(X_{1,t}\in B)P(X_{2,t}\in B).
\end{aligned} \tag{12}$$

Summing over $t = \tau+1,\ldots,T$ gives

$$\begin{aligned}
&Cov\left(\sum_{t=\tau+1}^{T} Y_{1,t},\, \sum_{t=\tau+1}^{T} Y_{2,t}\right) \\
&= \sum_{t=\tau+1}^{T} Cov(Y_{1,t}, Y_{2,t}) \\
&\leq -\rho \sum_{t=\tau+1}^{T} P(X_{1,t}\in B)P(X_{2,t}\in B).
\end{aligned} \tag{13}$$

Plugging (13) into (10)–(11) yields

$$\begin{aligned}
&Var(\hat{P}_R(T)) \\
&= \frac{1}{2T^2} Var\left(\sum_{t=\tau+1}^{T} Y_{1,t}\right) + \frac{1}{T^2} Var\left(\sum_{t=1}^{\tau-1} Y_{s,t}\right) - \\
&\qquad \frac{\rho}{2T^2}\sum_{t=\tau+1}^{T} P(X_{1,t}\in B)P(X_{2,t}\in B),
\end{aligned} \tag{14}$$

which is the claimed variance comparison form in Section 6.3.2.

Step 3: Asymptotic order and the G_T term.. Let $f(x) = \mathbf{1}\{x \in B\}$ and define the asymptotic variance

$$\sigma_f^2 \;=\; \lim_{T\to\infty} Var\left(\frac{1}{\sqrt{T}}\sum_{t=1}^{T} f(X_t)\right).$$

For a geometrically ergodic chain, $Var(\sum_{t=\tau+1}^{T} Y_{1,t}) = (T-\tau)\sigma_f^2 + o(T)$. If additionally τ exceeds the mixing time, then $P(X_{1,t} \in B) \approx P(X_{2,t} \in B) \approx \pi(1)$ for $t > \tau$, and the negative term in (14) scales as

$$\begin{aligned}&\frac{\rho}{2T^2}\sum_{t=\tau+1}^{T} P(X_{1,t} \in B)P(X_{2,t} \in B)\\ &\approx\; \frac{\rho}{2T^2}(T-\tau)\pi(1)^2 \;=\; O\Big(\frac{\rho}{T}\Big).\end{aligned}$$

Collecting the dominant terms yields the stated G_T scaling:

$$Var(\hat{P}_R(T)) - Var(\tilde{P}_R(T)) \;\approx\; G_T = \frac{1}{2T}\big(\sigma_f^2 - \rho\big).$$

Step 4: Closed-form expressions in the stationary regime.. When the chain is stationary with invariant distribution π, the standard variance formula gives

$$\sigma_f^2 = Var_\pi(f) + 2\sum_{k=1}^{\infty} Cov_\pi\big(f(X_0), f(X_k)\big).$$

For a two-state (bug / non-bug) abstraction with spectral gap $\gamma_{M,R}$ and $f = \mathbf{1}\{X \in B\}$, one obtains

$$\begin{aligned}&Var_\pi(f) = \pi(1)\pi(0),\\ &Cov_\pi\big(f(X_0), f(X_k)\big) = \frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}(1-\gamma_{M,R})^k,\end{aligned}$$

and summing the geometric series yields

$$\sigma_f^2 = \pi(1)\pi(0) + \frac{\pi(1)}{2\max\{\pi(0),\pi(1)\}}\cdot\frac{1-\gamma_{M,R}}{\gamma_{M,R}}.$$

Next, under the AR(1) masking/mutation model, the correlation constant ρ can be interpreted as the fractional decrease in joint bug probability induced by the coupled construction. In the simplified discrete model in which (i) the mutation value is uniform over a finite input domain of size $|D|$, and (ii) the event $\{X_t \in B\}$ is triggered by a single location/value pattern, the resulting expression reduces to

$$\rho = \frac{1}{|D|\cdot\pi(1)}.$$

Step 5: Non-stationary extension (controlled deviation).. When the chain is non-stationary, assume the deviation between the stationary distribution π and the marginal distribution at time t (after a burn-in) is bounded by δ in total variation, and the mapping from this deviation to ρ is L-Lipschitz. Then

$$\big|\rho - \tfrac{1}{|D|\cdot\pi(1)}\big| \le L\delta,$$

which yields the stated bounds on ρ used in Section 6.3.3.

H.5. Variance Comparison between Splitting and MC

Proof of Theorem 2. **Non-splitting (vanilla MC).** Let $\tilde{\mu}_T = \frac{1}{T}\sum_{t=1}^{T} f(X_t)$ with X_t a stationary Markov chain with kernel P. Then

$$\mathrm{Var}(\tilde{\mu}_T) = \frac{\sigma^2_{\mathrm{asym}}(f)}{T} + o(T^{-1}), \qquad \mathrm{CPU}_{\mathrm{MC}}(T) = T,$$

and hence

$$\mathcal{E}_{\mathrm{MC}}(T) = \mathrm{Var}(\tilde{\mu}_T)\cdot T = \sigma^2_{\mathrm{asym}}(f) + o(1).$$

Splitting estimator (two descendants; re-splitting allowed). Let $X_t^{(1)}, \ldots, X_t^{(k_t)}$ be the active paths at time t, and

$$\bar{Z}_t := \frac{1}{k_t}\sum_{i=1}^{k_t} f(X_t^{(i)}), \hat{\mu}_T := \frac{1}{T}\sum_{t=1}^{T}\bar{Z}_t, \mathrm{CPU}_{\mathrm{split}}(T) = \sum_{t=1}^{T} k_t.$$

Define the (finite-horizon) local autocovariance weight

$$\begin{aligned}&c_t^{(T)} \;:=\; \mathrm{Var}\big(g(X_t)\big) \;+\; 2\sum_{\ell=1}^{T-t}\big|\mathrm{Cov}\big(g(X_t), g(X_{t+\ell})\big)\big|,\\ &g := f - \pi(f).\end{aligned}$$

Under Assumption A1, we have

$$\begin{aligned}\mathrm{Var}(\hat{\mu}_T) \;&\le\; \frac{1}{T^2}\sum_{t=1}^{T}\frac{c_t^{(T)}}{k_t},\\ \mathcal{E}_{\mathrm{split}}(T) \;&\le\; \frac{1}{T^2}\Big(\sum_{t=1}^{T} k_t\Big)\Big(\sum_{t=1}^{T}\frac{c_t^{(T)}}{k_t}\Big).\end{aligned}$$

Two-level allocation comparison ($k_t \in \{1,2\}$). Based on our splitting algorithm,

$$k_t = \begin{cases}2, & X_t \in B,\\ 1, & X_t \notin B.\end{cases}$$

Let $S := \{t \in [T] : X_t \in B\}$, and define

$$C_{\mathrm{in}} := \sum_{t\in S} c_t^{(T)}, \qquad C_{\mathrm{out}} := \sum_{t\notin S} c_t^{(T)}.$$

Then we obtain

$$\begin{aligned}&\mathcal{E}_{\mathrm{split}}(T) \;\le\; \frac{T+|S|}{T^2}\Big(C_{\mathrm{out}} + \frac{1}{2}C_{\mathrm{in}}\Big),\\ &\mathcal{E}_{\mathrm{MC}}(T) \text{ (via the same bound with } k_t \equiv 1) \;\le\; \frac{1}{T}\big(C_{\mathrm{out}} + C_{\mathrm{in}}\big).\end{aligned}$$

Therefore,

$$\frac{\mathcal{E}_{\mathrm{split}}(T)}{\mathcal{E}_{\mathrm{MC}}(T)} \;\le\; \frac{T+|S|}{T}\cdot\frac{C_{\mathrm{out}} + \frac{1}{2}C_{\mathrm{in}}}{C_{\mathrm{out}} + C_{\mathrm{in}}}.$$

This implies that splitting improves variance-per-CPU whenever C_{in} (the autocovariance mass contributed by times in B)

dominates C_{out}. In the extreme clustered regime $C_{\text{in}} \gg C_{\text{out}}$, the ratio approaches

$$\frac{\mathcal{E}_{\text{split}}(T)}{\mathcal{E}_{\text{MC}}(T)} \approx \frac{T+|S|}{T} \cdot \frac{1}{2}.$$

Under stationarity, $|S|/T \approx p = \pi(B)$, recovering the heuristic $(1+p)/2$.

□

H.6. Bug Detection Rate of Splitting

Proof of Theorem 3. Let R_{MC} and R_{Split} denote the bug-detection rates of MC and Splitting, respectively. To avoid notational collision, it is convenient to view $\{Y_{s,t}\}$ as the bug indicators produced by Splitting at (outer) time t, where $s \in \{1, \dots, k_t\}$, and to identify the MC indicator at time t with $Y_{1,t}$. Then

$$R_{\text{MC}} = \frac{1}{T}\sum_{t=1}^{T} Y_{1,t} = \frac{|S_{\text{MC}}|}{T}.$$

Under Splitting, since $k_t \in \{1,2\}$ and $k_t = 2$ exactly on the splitting times, define

$$A := \{t \in [T] : k_t = 2\}.$$

Then the Splitting rate can be written as

$$R_{\text{Split}} = \frac{1}{T}\sum_{t=1}^{T}\frac{1}{k_t}\sum_{s=1}^{k_t} Y_{s,t} = \frac{1}{T}\sum_{t\notin A} Y_{1,t} + \frac{1}{2T}\sum_{t\in A}\big(Y_{1,t}+Y_{2,t}\big).$$

Therefore, the increment of Splitting over MC is

$$\begin{aligned} R_{\text{Split}} - R_{\text{MC}} &= \Big[\frac{1}{T}\sum_{t\notin A} Y_{1,t} + \frac{1}{2T}\sum_{t\in A}(Y_{1,t}+Y_{2,t})\Big] - \frac{1}{T}\sum_{t=1}^{T} Y_{1,t} \\ &= \frac{1}{2T}\sum_{t\in A}\big(Y_{2,t} - Y_{1,t}\big). \end{aligned}$$

This is an exact identity.

Using your relation

$$|S_{\text{Splitting}}| - |S_{\text{MC}}| = \sum_{t=1}^{T}\sum_{s=2}^{k_t} Y_{s,t} = \sum_{t\in A} Y_{2,t},$$

we can equivalently rewrite the increment as

$$R_{\text{Split}} - R_{\text{MC}} = \frac{1}{2T}\left(|S_{\text{Splitting}}| - |S_{\text{MC}}| - \sum_{t\in A} Y_{1,t}\right).$$

As a simple consequence (since $0 \le Y_{1,t} \le 1$),

$$\frac{1}{2T}\Big(|S_{\text{Splitting}}| - |S_{\text{MC}}| - |A|\Big) \le R_{\text{Split}} - R_{\text{MC}} \le \frac{1}{2T}\Big(|S_{\text{Splitting}}| - |S_{\text{MC}}|\Big),$$

and $R_{\text{Split}} \ge R_{\text{MC}}$ holds whenever

$$\sum_{t\in A} Y_{2,t} \ge \sum_{t\in A} Y_{1,t}.$$

□